



Git + GitHub

Prof. David Fernandes de Oliveira
Instituto de Computação
UFAM

Controle de Versão

- **Sistemas de Controle de Versão** registram as alterações realizadas em um conjunto de arquivos ao longo do tempo.
- O uso de um sistema de controle de versão permite:
 - restaurar arquivos ou todo o projeto para uma versão anterior
 - comparar as alterações realizadas ao longo do tempo
 - facilitar o trabalho colaborativo de uma equipe
 - registrar a data, a hora, o autor e a descrição de cada alteração



git

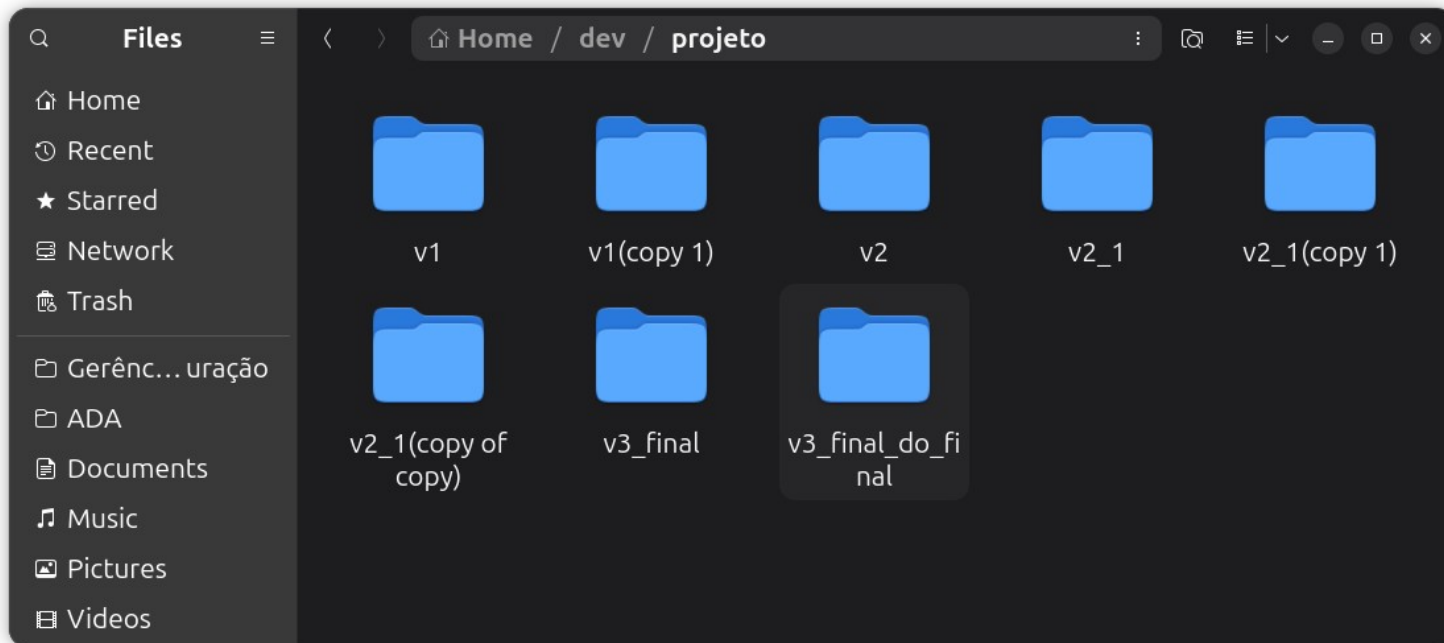


mercurial



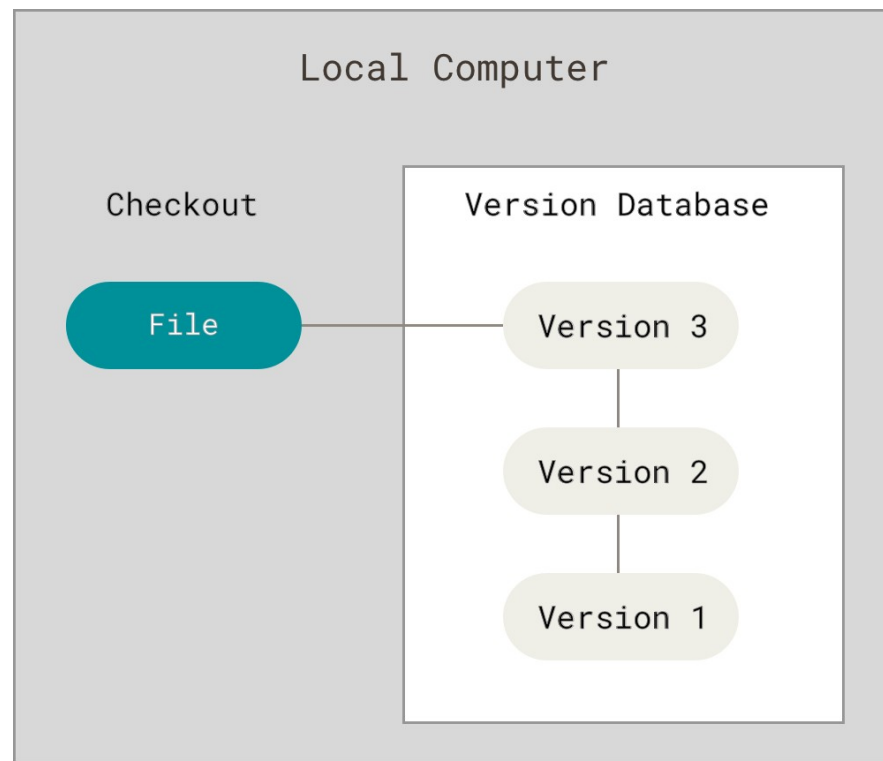
Controle de Versão Local

- Uma das formas mais simples de controlar versões é **criar cópias dos arquivos ou projetos** em diferentes diretórios
- Embora seja uma prática comum, essa abordagem não gera histórico estruturado das alterações e é propensa a erros
 - É difícil saber o que alteriou de uma pasta para outra



Controle de Versão Local

- Para lidar com esse problema, alguns programadores desenvolveram **sistemas de controle de versão locais**
 - Tais sistemas armazenavam todas as alterações dos arquivos em várias versões dos mesmos

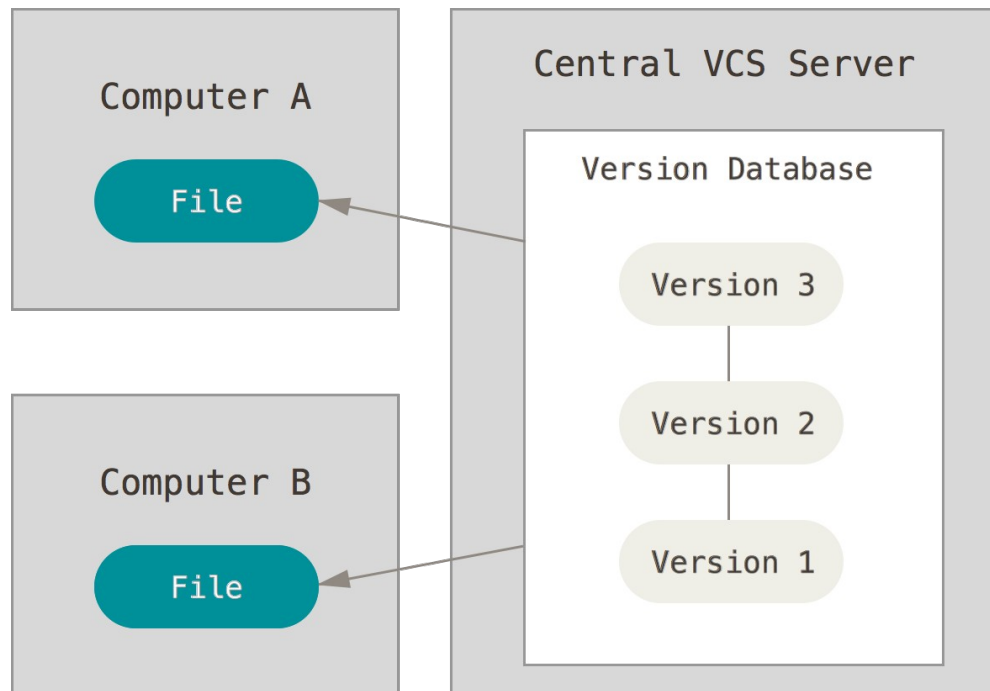


Imagens retiradas do
livro **Pro Git**,
disponível
gratuitamente no
[site oficial do Git](https://git-scm.com/)



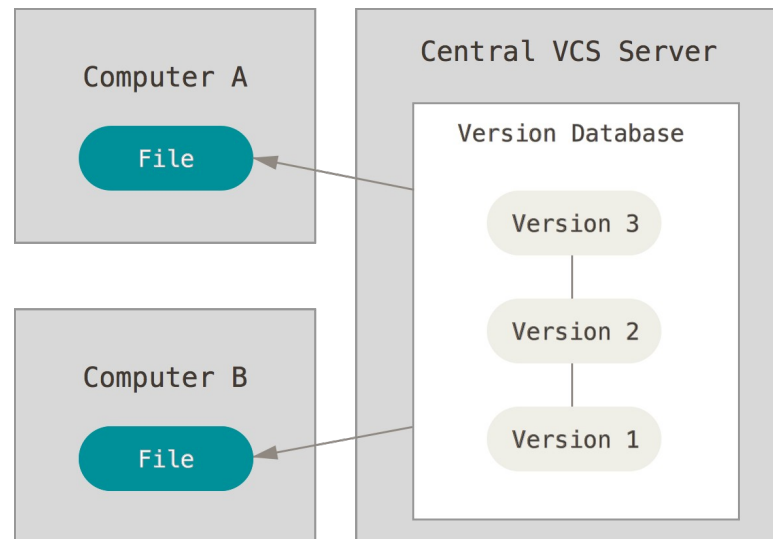
Controle de Versão Centralizado

- Em muitas ocasiões, é necessário trabalhar em conjunto com outros desenvolvedores
- Para lidar com isso, foram desenvolvidos os **sistemas de controle de versão centralizados**



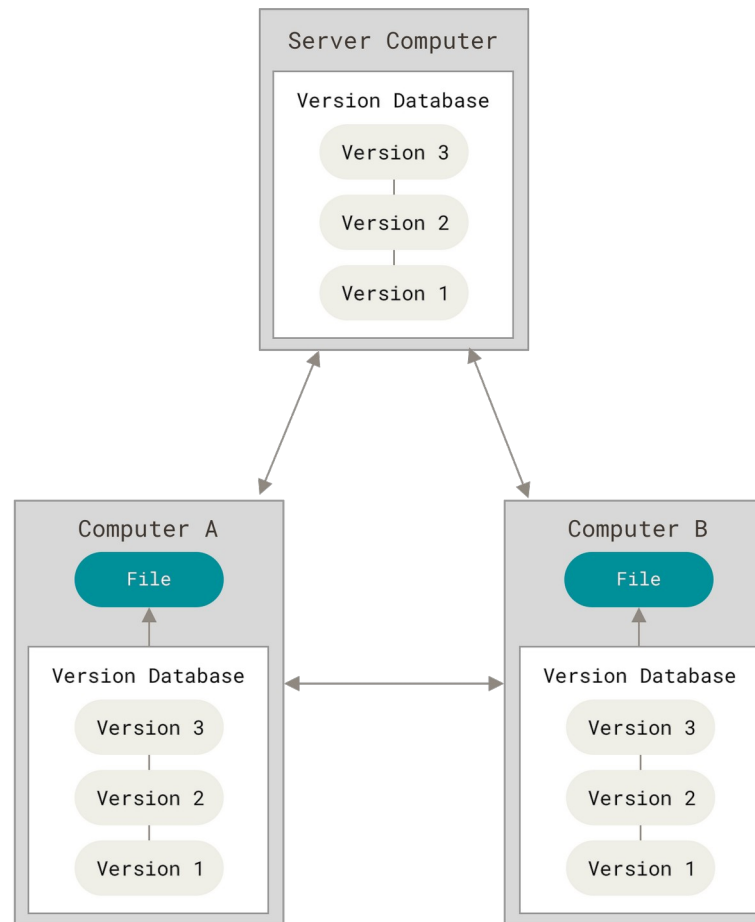
Controle de Versão Centralizado

- Problemas com o controle de versão centralizado:
 - Se o servidor ficar fora do ar, ninguém poderá acessar o mecanismo de versionamento ou salvar seus arquivos
 - Se o servidor for corrompido e não houver backup, perde-se todo o histórico de mudanças do projeto
 - Os desenvolvedores ficam apenas com a versão atual do projeto de suas máquinas



Controle de Versão Distribuídos

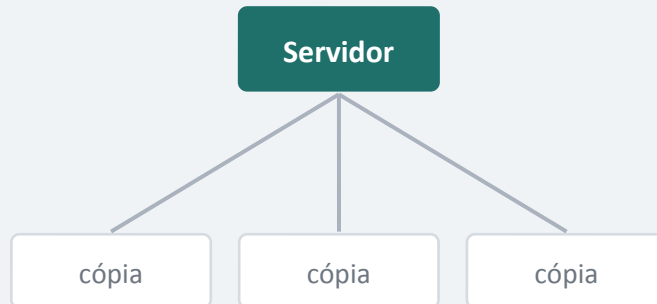
- Na abordagem **distribuída**, as estações de trabalho (clientes) também possuem cópias completas do repositório



Centralizado x Distribuído

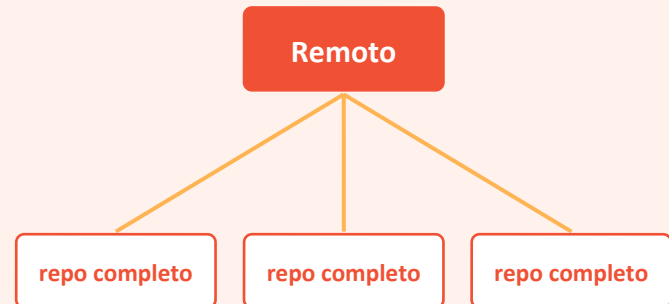
- Uma desvantagem da abordagem centralizada é que quase nada do controle de versão funciona offline

Centralizado (Subversion)



Histórico existe só no servidor. Offline, quase nada funciona.

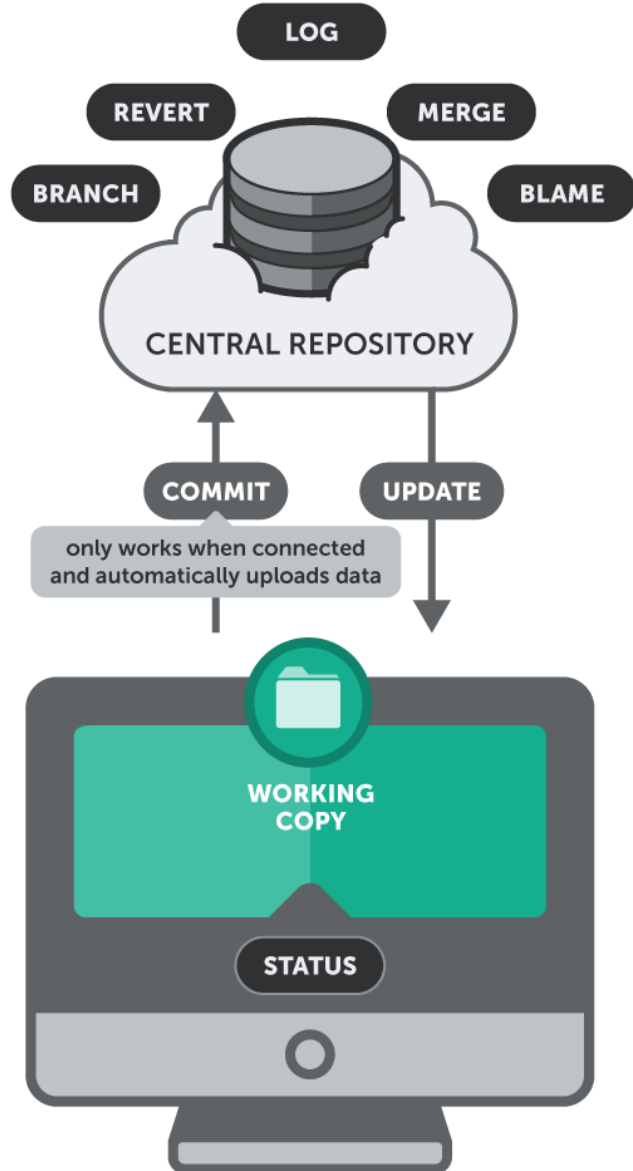
Distribuído (Git)



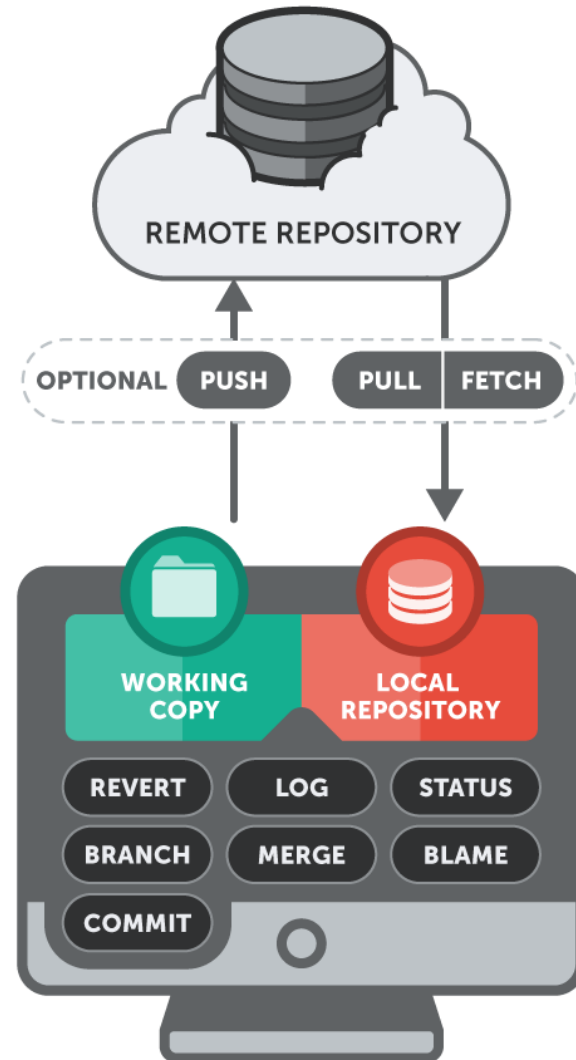
Cada clone tem todo o histórico. Commit, branch e log funcionam offline.



SUBVERSION



GIT



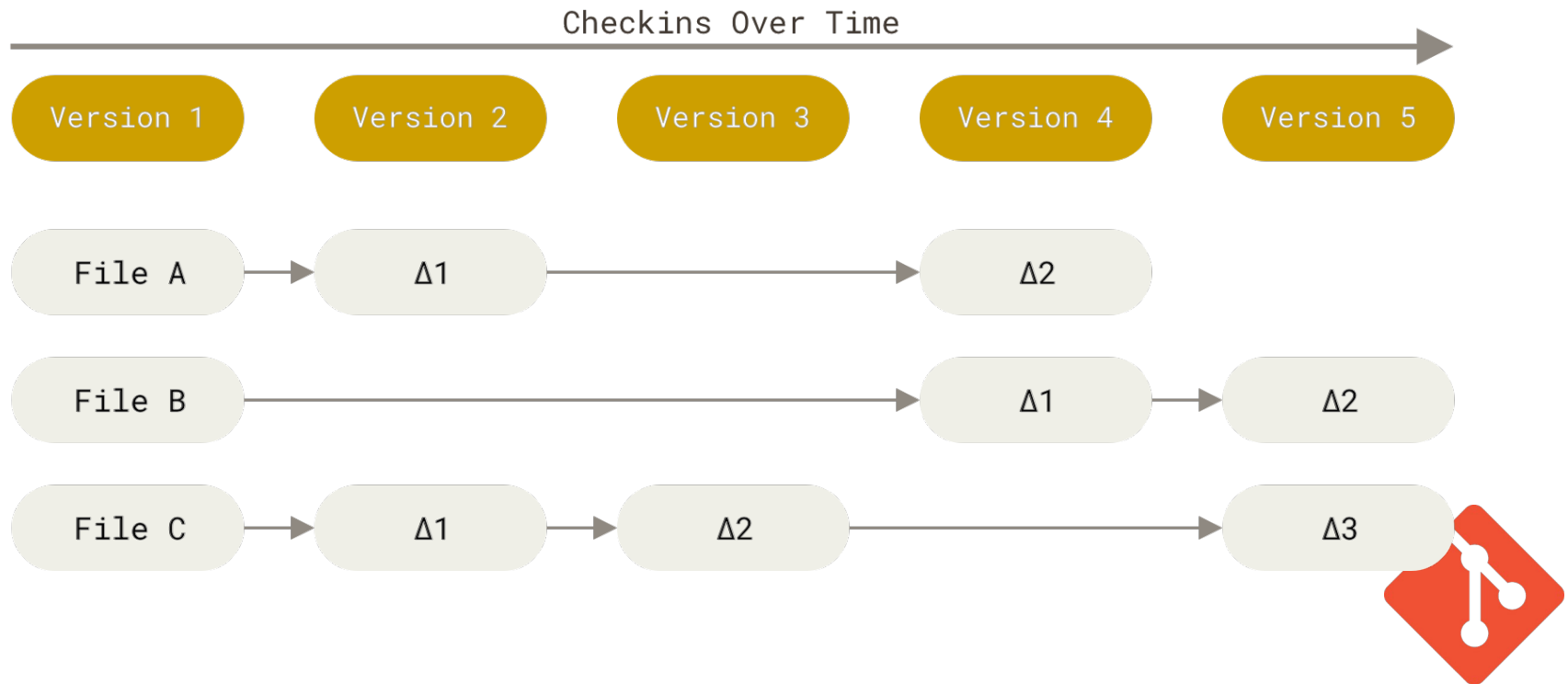
O que é o Git?

- É um **sistema de controle de versão distribuído**, desenvolvido a partir de 2005
- Foi criado em 2005 por **Linus Torvalds**, com algumas contribuições da comunidade de desenvolvedores do kernel do Linux



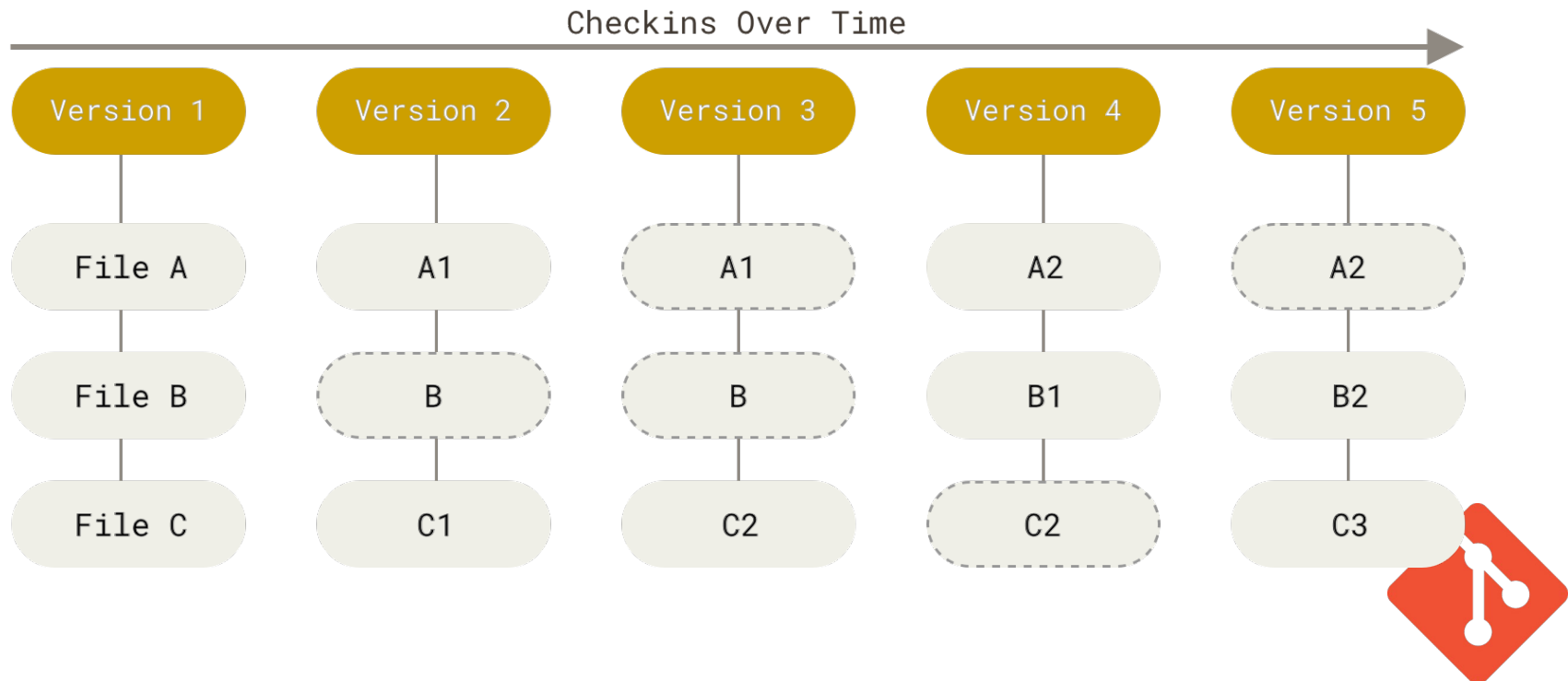
Noções Básicas de Git

- Outra diferença entre Git e outros SCV (Subversion, CVS, etc) está na forma que o Git armazena os dados
- A maior parte dos sistemas armazena informação como uma lista de mudanças por arquivo



Noções Básicas de Git

- O Git, por outro lado, registra cada commit como uma fotografia completa dos arquivos do projeto
- Arquivos que não foram modificados são reutilizados por referência, evitando cópias desnecessárias



Inicializando um Repositório

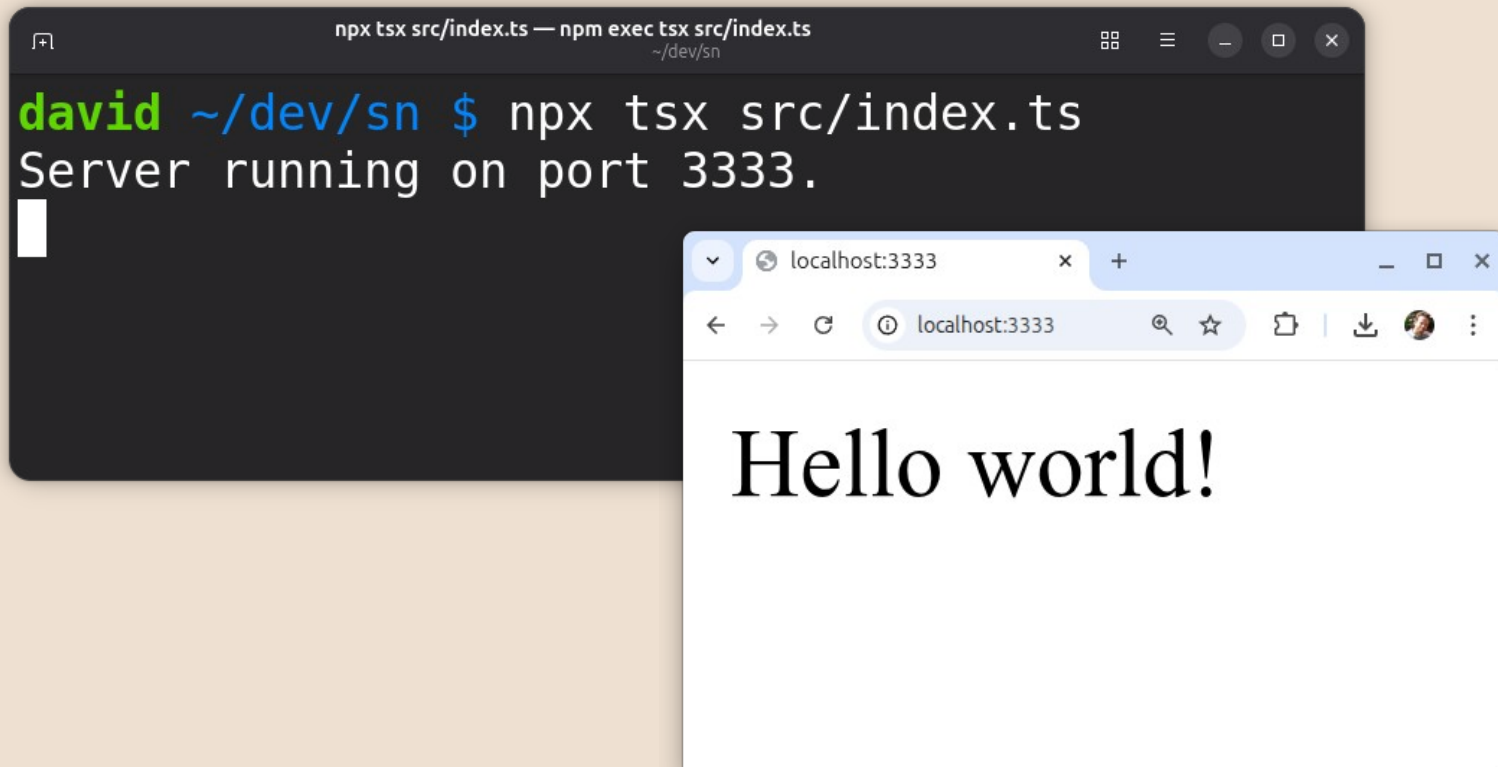
- Para começar a versionar com Git um projeto que já existe, acesse o diretório do projeto pelo terminal e execute:
 - **git init**
 - Esse comando cria um diretório **.git** dentro do projeto, que armazena todos os dados de controle de versões do sistema

```
david@coyote:~/dev/sn
david ~/dev/sn $ ls -a
.  node_modules  package.json  tsconfig.json
.. package-lock.json  src
david ~/dev/sn $ git init
Initialized empty Git repository in /home/david/dev/sn/.git/
david ~/dev/sn on  ↳ main $ ls -a
.  .git  package-lock.json  src
.. node_modules  package.json  tsconfig.json
david ~/dev/sn on  ↳ main $
```



Exercício

- Desenvolva um sistema **Hello World** usando Express/TypeScript e então inicialize o repositório Git usando **git init**
 - Para isso, siga as instruções dos slides sobre o SmartNotes, até a seção **Hello World em Express/TypeScript**



Configuração Inicial do Git

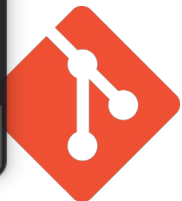
- O Git possui uma ferramenta chamada **git config**, utilizada para definir e consultar variáveis de configuração
- Essas configurações podem ser armazenadas em três níveis:
 - **/etc/gitconfig**: aplica-se a todos os usuários e repositórios do sistema. É configurado com a opção **--system**
 - **~/.gitconfig**: aplica-se a todos os repositórios do usuário atual. É configurado com a opção **--global**
 - **.git/config**: aplica-se apenas ao repositório atual. É configurado com a opção **--local**
- Quando uma mesma configuração é definida em mais de um nível, o nível mais específico tem prioridade:
 - local > global > system



Configuração Inicial do Git

- Após instalar o Git, é importante configurar o **nome** e o **endereço de e-mail** do usuário
 - O Git usa essas informações para identificar o autor de cada commit e registrar quem realizou as alterações no repositório
 - Também é útil definir o **editor** e o **nome da branch** padrões

```
david@coyote:~  
david ~ $ git config --global user.name "David Fernandes"  
david ~ $ git config --global user.email "david@icomp.ufam.edu.br"  
david ~ $ git config --global init.defaultBranch main  
david ~ $ git config --global core.editor "vim"  
david ~ $ cat .gitconfig  
[user]  
    email = david@icomp.ufam.edu.br  
    name = David Fernandes  
[init]  
    defaultBranch = main  
[core]  
    editor = vim  
david ~ $
```

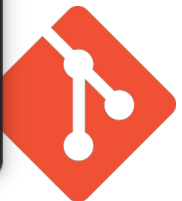


Configuração Inicial do Git

- Após instalar o Git, é importante configurar o **nome** e o **endereço de e-mail** do usuário

Muito importante: É preciso configurar seu nome de usuário e email sempre que for usar uma máquina dos labs para trabalhar nas atividades da disciplina. Isso é importante para viabilizar o acesso ao histórico de commits e analisar a contribuição de cada um no projeto da disciplina.

```
david ~ $ git config --global user.name "David Fernandes"
david ~ $ git config --global user.email "david@icomp.ufam.edu.br"
david ~ $ git config --global init.defaultBranch main
david ~ $ git config --global core.editor "vim"
david ~ $ cat .gitconfig
[user]
    email = david@icomp.ufam.edu.br
    name = David Fernandes
[init]
    defaultBranch = main
[core]
    editor = vim
david ~ $
```



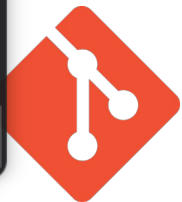
Configuração Inicial do Git

- Após instalar o Git, é importante configurar o **nome** e o **endereço de e-mail** do usuário

Muito importante: É preciso configurar seu nome de usuário e email sempre que for usar uma máquina dos labs para trabalhar nas atividades da disciplina.

Para isso, siga as orientações do README.md do repositório
<https://github.com/dbfernandes/git-config>

```
david ~ $ git config --global user.email "david@icomp.ufam.edu.br"
david ~ $ git config --global init.defaultBranch main
david ~ $ git config --global core.editor "vim"
david ~ $ cat .gitconfig
[user]
    email = david@icomp.ufam.edu.br
    name = David Fernandes
[init]
    defaultBranch = main
[core]
    editor = vim
david ~ $
```



Configuração Inicial do Git

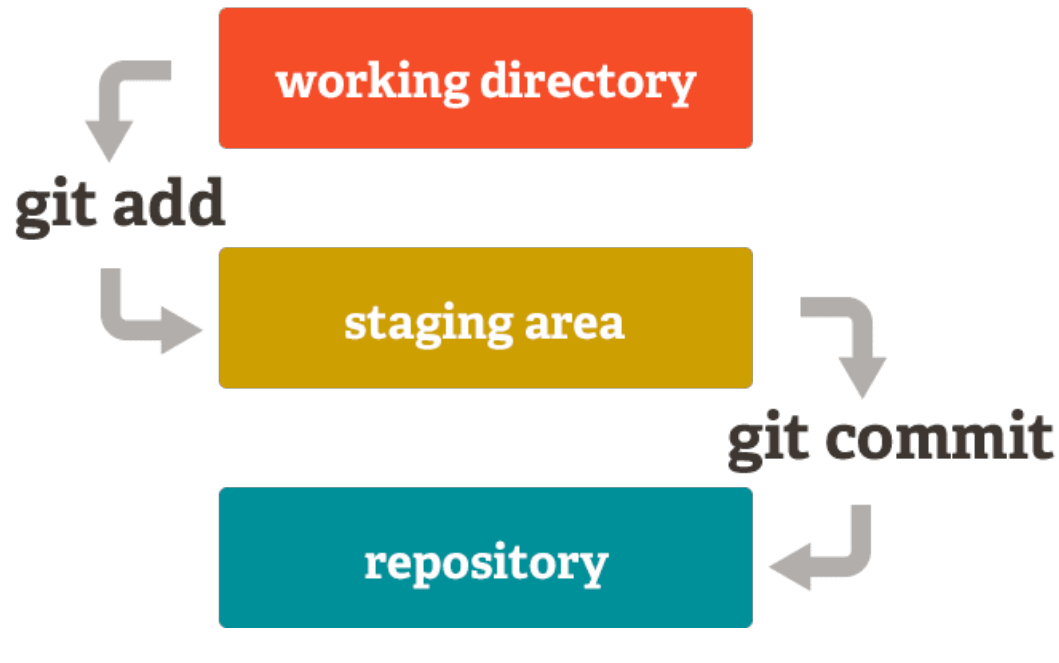
```
david@coyote:~/dev/sn
david ~/dev/sn on main $ npm run git-config
> gc@1.0.0 git-config
> bash scripts/git-config.sh
Configuração do Git não encontrada.
Nome: David Fernandes
E-mail: david@icomp.ufam.edu.br
Arquivo .git-config criado.
Faça commit desse arquivo para reutilizar a configuração em outros clones.
Configurações locais do Git aplicadas:
[user]
  name = David Fernandes
  email = david@icomp.ufam.edu.br
[core]
  editor = vim
david ~/dev/sn on main $
```

took 13s



Fluxo de Trabalho

- Após criar um repositório, o fluxo básico de trabalho consiste em:
 - Modificar os arquivos do projeto
 - Selecionar as alterações que farão parte do próximo commit, utilizando **git add**
 - Registrar essas alterações no histórico, utilizando **git commit**



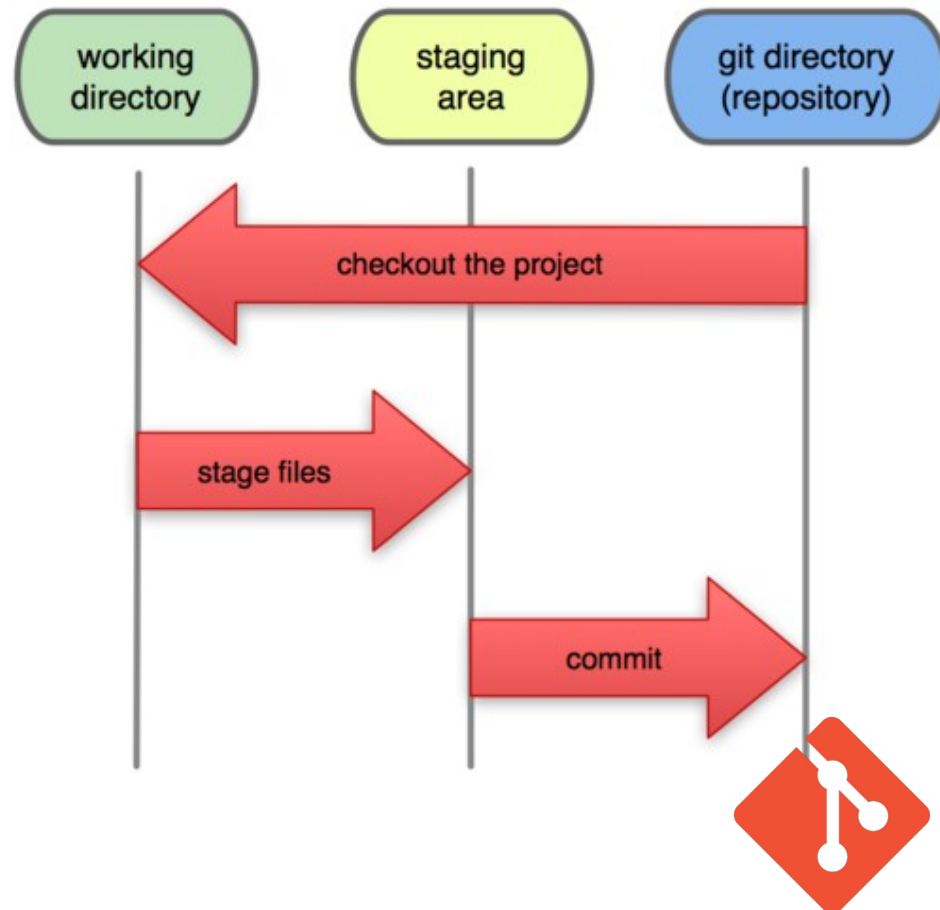
O Primeiro Commit

- Para começar a versionar os arquivos existentes de um projeto, é necessário:
 - adicionar os arquivos desejados à **staging area**
 - **criar um commit** para registrar essas alterações no histórico do repositório

```
david@coyote:~/dev/sn
david ~/dev/sn on  main $ git add scripts/ .git-config *.json src
david ~/dev/sn on  main $ git commit -m "First commit"
[main (root-commit) 7da5585] First commit
6 files changed, 2129 insertions(+)
create mode 100644 .git-config
create mode 100644 package-lock.json
create mode 100644 package.json
create mode 100644 scripts/git-config.sh
create mode 100644 src/index.ts
create mode 100644 tsconfig.json
david ~/dev/sn on  main $
```

Os Três Estados

- Desta forma, o Git faz com que seus arquivos sempre estejam em um dos três estados fundamentais:
 - **Modificado** (modified), quando o arquivo sofreu mudanças mas ainda não foi selecionado
 - **Selecionado** (staged), quando você marca um arquivo modificado para que ele faça parte do próximo commit (consolidação)
 - **Consolidado** (committed), quando os arquivos estão seguramente salvos em seu repositório local



Adicionando um arquivo

- É possível incluir novos arquivos no repositório local normalmente, usando editores ou recursos do SO
- A ferramenta **git status** pode ser usada para identificar os estados dos arquivos do repositório

```
david@coyote:~/dev/sn
david ~/dev/sn on ʘ main $ echo "# SmartNotes" > README.md
david ~/dev/sn on ʘ main $ git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    README.md
    node_modules/

nothing added to commit but untracked files present (use "git add" to track)
david ~/dev/sn on ʘ main $
```





david@coyote:~/dev/sn

~/dev/sn



```
david ~/dev/sn on ʘ main $ echo "# SmartNotes" > README.md
```

```
david ~/dev/sn on ʘ main $ git status
```

On branch main

Untracked files:

(use "git add <file>..." to include in what will be committed)

README.md
node_modules/

nothing added to commit but untracked files present (use "git add" to track)

```
david ~/dev/sn on ʘ main $ git add README.md
```

```
david ~/dev/sn on ʘ main $ git status
```

On branch main

Changes to be committed:

(use "git restore --staged <file>..." to unstage)

new file: README.md

Untracked files:

Untracked files:

(use "git add <file>..." to include in what will be committed)

node_modules/

```
david ~/dev/sn on ʘ main $ git commit -m "Create README.md file"
```

```
[main 933f520] Create README.md file
```

1 file changed, 1 insertion(+)

create mode 100644 README.md

```
david ~/dev/sn on ʘ main $
```



Variações de add e commit

`git add <arquivo>`

Adiciona um arquivo específico à staging area.

`git add .`

Adiciona tudo no diretório atual e subdiretórios. Use com atenção.

`git add -p`

Interativo: escolhe trecho a trecho o que entra no commit.

`git commit -m "..."`

Commit com mensagem na própria linha de comando.

`git commit`

Abre o editor, permitindo escrever corpo de mensagem completo.

`git commit -a -m "..."`

Adiciona automaticamente arquivos já rastreados e comita.

`git commit --amend`

Substitui o último commit. Nunca use em commits já publicados.



Mensagens de Commit

- A mensagem deve deixar claro o propósito do commit no histórico de mudanças do código

Mensagens de commit pouco úteis

ajustes
fix
asdf
funcionou agora
commit

Mensagens de commit informativas

Corrige cálculo de juros em faturas vencidas
Adiciona validação de CPF no cadastro
Remove dependência não utilizada de moment.js

- Conversão amplamente adotada:

Adiciona validação de CPF no formulário de cadastro

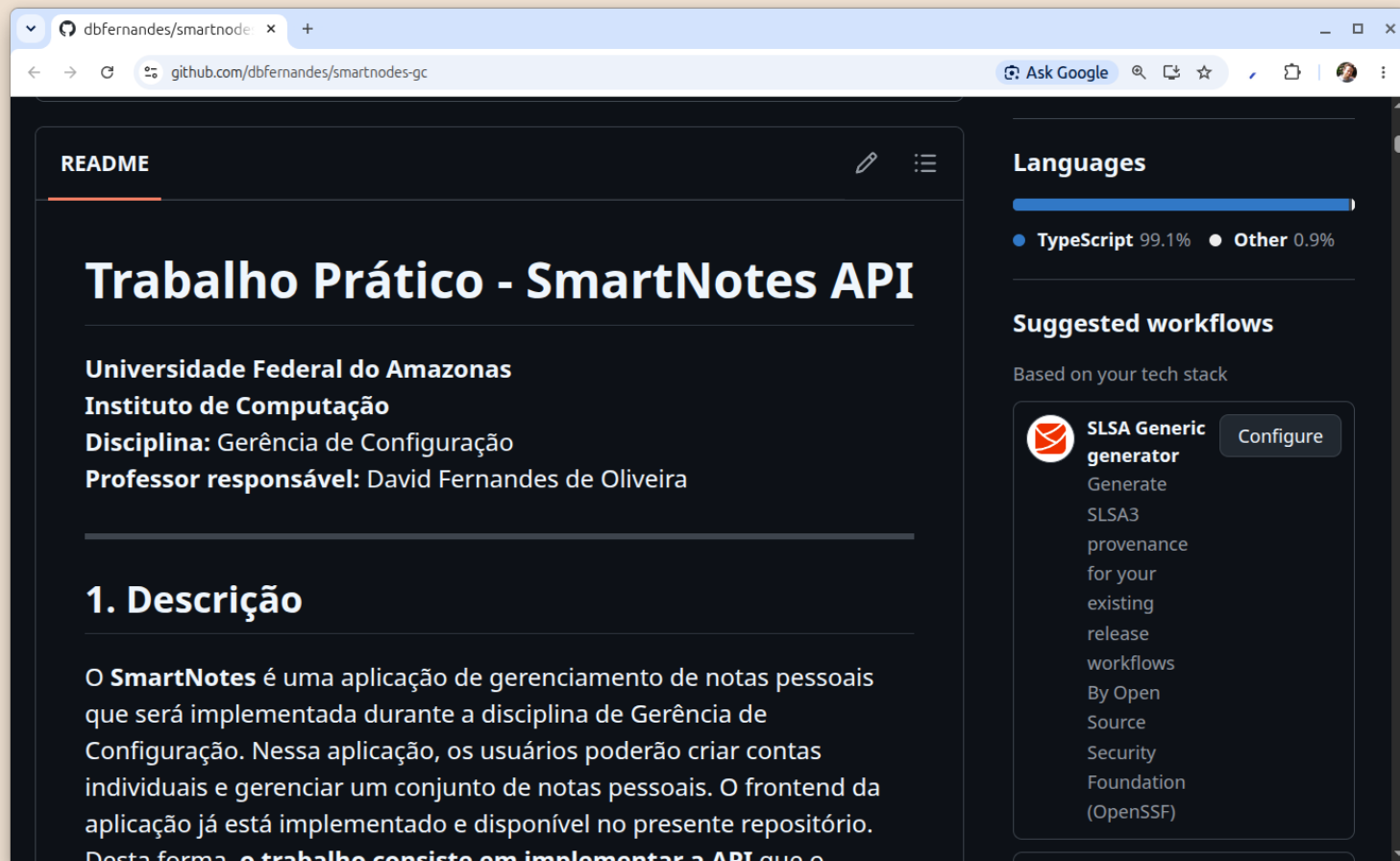
O formulário aceitava CPFs com dígito verificador inválido, o que gerava falha apenas na integração com a Receita. A validação passa a ocorrer no cliente e no servidor.

Refs #142



Exercício

- Adicione um arquivo **README.md** em seu projeto usando os orientações passadas nos slides anteriores



The screenshot shows a web browser displaying the GitHub repository page for 'dbfernandes/smartnodes-gc'. The main content is the README file, which has a title 'Trabalho Prático - SmartNotes API'. Below the title, it lists the institution 'Universidade Federal do Amazonas', the department 'Instituto de Computação', the discipline 'Disciplina: Gerência de Configuração', and the professor 'Professor responsável: David Fernandes de Oliveira'. The first section is '1. Descrição', which describes the 'SmartNotes' application as a personal note management tool. The right sidebar shows the 'Languages' section with 'TypeScript' at 99.1% and 'Other' at 0.9%, and a 'Suggested workflows' section with a recommendation for 'SLSA Generic generator'.

dbfernandes/smartnodes-gc

github.com/dbfernandes/smartnodes-gc

Ask Google

README

Trabalho Prático - SmartNotes API

Universidade Federal do Amazonas
Instituto de Computação
Disciplina: Gerência de Configuração
Professor responsável: David Fernandes de Oliveira

1. Descrição

O **SmartNotes** é uma aplicação de gerenciamento de notas pessoais que será implementada durante a disciplina de Gerência de Configuração. Nessa aplicação, os usuários poderão criar contas individuais e gerenciar um conjunto de notas pessoais. O frontend da aplicação já está implementado e disponível no presente repositório. Desta forma, o trabalho consiste em implementar a API que o

Languages

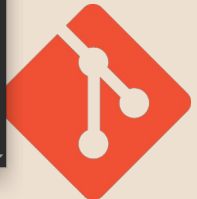
TypeScript 99.1% • Other 0.9%

Suggested workflows

Based on your tech stack

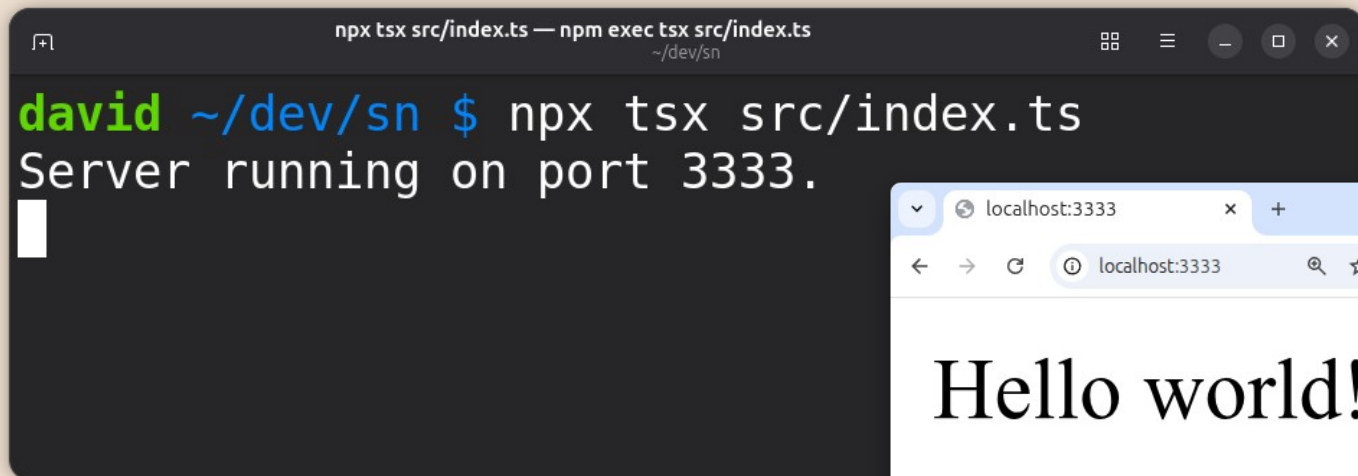
SLSA Generic generator Configure

Generate SLSA3 provenance for your existing release workflows By Open Source Security Foundation (OpenSSF)



Exercício

- Instale o pacote **dotenv** e crie um arquivo **.env** com uma variável com o número da porta do servidor
 - Para isso, siga as instruções dos slides sobre o SmartNotes, até a seção **Variáveis de Ambiente**



The image shows a terminal window and a web browser. The terminal window has a title bar that reads 'npx tsx src/index.ts — npm exec tsx src/index.ts' and a subtitle '~ /dev/sn'. The terminal prompt is 'david ~/dev/sn \$' and the command entered is 'npx tsx src/index.ts'. The output is 'Server running on port 3333.' followed by a blank line. The web browser window has a single tab titled 'localhost:3333'. The address bar shows 'localhost:3333'. The main content area of the browser displays 'Hello world!' in a large, black, serif font.

```
npx tsx src/index.ts — npm exec tsx src/index.ts
~/dev/sn
david ~/dev/sn $ npx tsx src/index.ts
Server running on port 3333.
```

Hello world!



Exercício

- Crie o arquivo **src/utils/getEnv.ts** para validar e recuperar as variáveis de ambiente do arquivo **.env**
 - Para isso, siga as instruções dos slides sobre o **SmartNotes**, até a seção **Validando as variáveis de ambiente**

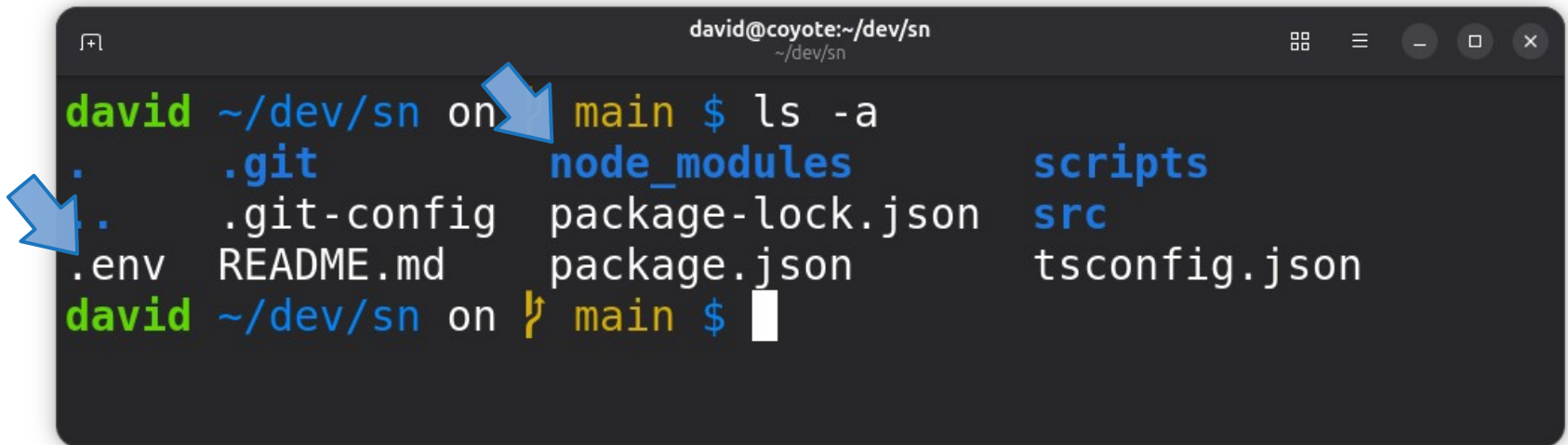
```
david@coyote:~/dev/sn
david ~/dev/sn on  main $ npx tsx src/index.ts
=====
Missing environment variables:
  NODE_ENV: undefined
=====

Exiting with error code 1
david ~/dev/sn on  main $
```



Ignorando Arquivos

- É muito comum que um dado projeto tenha arquivos que não devam ser versionados pelo git
 - Por exemplo, arquivos de logs, arquivos com variáveis de ambiente, códigos compilados, dependências de terceiros, etc



```
david@coyote:~/dev/sn
david ~/dev/sn on main $ ls -a
.      .git      node_modules  scripts
.      .git-config package-lock.json  src
.env   README.md  package.json    tsconfig.json
david ~/dev/sn on main $
```

The terminal window shows the command `ls -a` being executed in a directory `~/dev/sn`. The output lists files and directories: `.`, `.git`, `node_modules`, `scripts`, `.git-config`, `package-lock.json`, `src`, `.env`, `README.md`, `package.json`, and `tsconfig.json`. Two blue arrows are present: one pointing to the `node_modules` directory and another pointing to the `.env` file, indicating these are files that should be ignored by Git.



Ignorando Arquivos

- Por isso, criamos um arquivo **.gitignore** com uma lista de arquivos e diretórios que devem ser ignorados pelo Git

```
# Logs
logs/*
!logs/.gitkeep

# Node files
node_modules/

# Environment files
.env

# Build folders
dist/
build/
```

Ordem
importa



Ignorando Arquivos

- Após criar o **.gitignore** e adicioná-lo no **staging area**, o git passa a ignorar todos os arquivos listados em seu interior

```
david@coyote:~/dev/sn
david ~/dev/sn on ʘ main $ git add .gitignore
david ~/dev/sn on ʘ main $ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   .gitignore

david ~/dev/sn on ʘ main $ git commit -m "Create .gitignore"
[main 9123dfe] Create .gitignore
 1 file changed, 14 insertions(+)
 create mode 100644 .gitignore
david ~/dev/sn on ʘ main $ git status
On branch main
nothing to commit, working tree clean
david ~/dev/sn on ʘ main $
```



Ignorando Arquivos

- Após criar o **.gitignore** e adicioná-lo no **staging area**, o git passa a ignorar todos os arquivos listados em seu interior

```
david@coyote:~/dev/sn  
david ~/dev/sn on ʘ main $ git add .gitignore
```

Caso deseje adicionar um arquivo ou diretório já commitado ao **.gitignore**, é preciso remover ele do índice:

```
$ git rm --cached .env  
$ git add .gitignore  
$ git commit -m "Stop tracking the .env file"
```

```
[main 9123dfe] Create .gitignore  
1 file changed, 14 insertions(+)  
create mode 100644 .gitignore  
david ~/dev/sn on ʘ main $ git status  
On branch main  
nothing to commit, working tree clean  
david ~/dev/sn on ʘ main $
```



Ignorando Arquivos

- Após criar o **.gitignore** e adicioná-lo no **staging area**, o git passa a ignorar todos os arquivos listados em seu interior

```
david@coyote:~/dev/sn  
david ~/dev/sn on ʤ main $ git add .gitignore
```

O **--cached** é importante porque remove os arquivos apenas do índice do Git, mantendo-os no seu diretório de trabalho

Sem **--cached**, o **git rm** remove o arquivo do índice do Git e também do seu diretório de trabalho

O **git rm** apenas remove o arquivo do rastreamento do Git. Se precisar remover o arquivo de todos os commits anteriores, precisa reescrever o histórico com **git filter-repo**

```
nothing to commit, working tree clean  
david ~/dev/sn on ʤ main $
```



Exercício

- Adicione o arquivo **.gitignore** no seu projeto, deixando o arquivo **.env** não versionado
- Crie um arquivo **.env.example** para servir de base para a criação de arquivos **.env** em novos ambientes de execução
- Faça um commit com essas últimas alterações

```
david@coyote:~/dev/sn
david ~/dev/sn on  ↳ main $ ls -a
.          .git-config          package.json
..         .gitignore         scripts
.env       README.md         src
.env.example node_modules      tsconfig.json
.git       package-lock.json
david ~/dev/sn on  ↳ main $
```



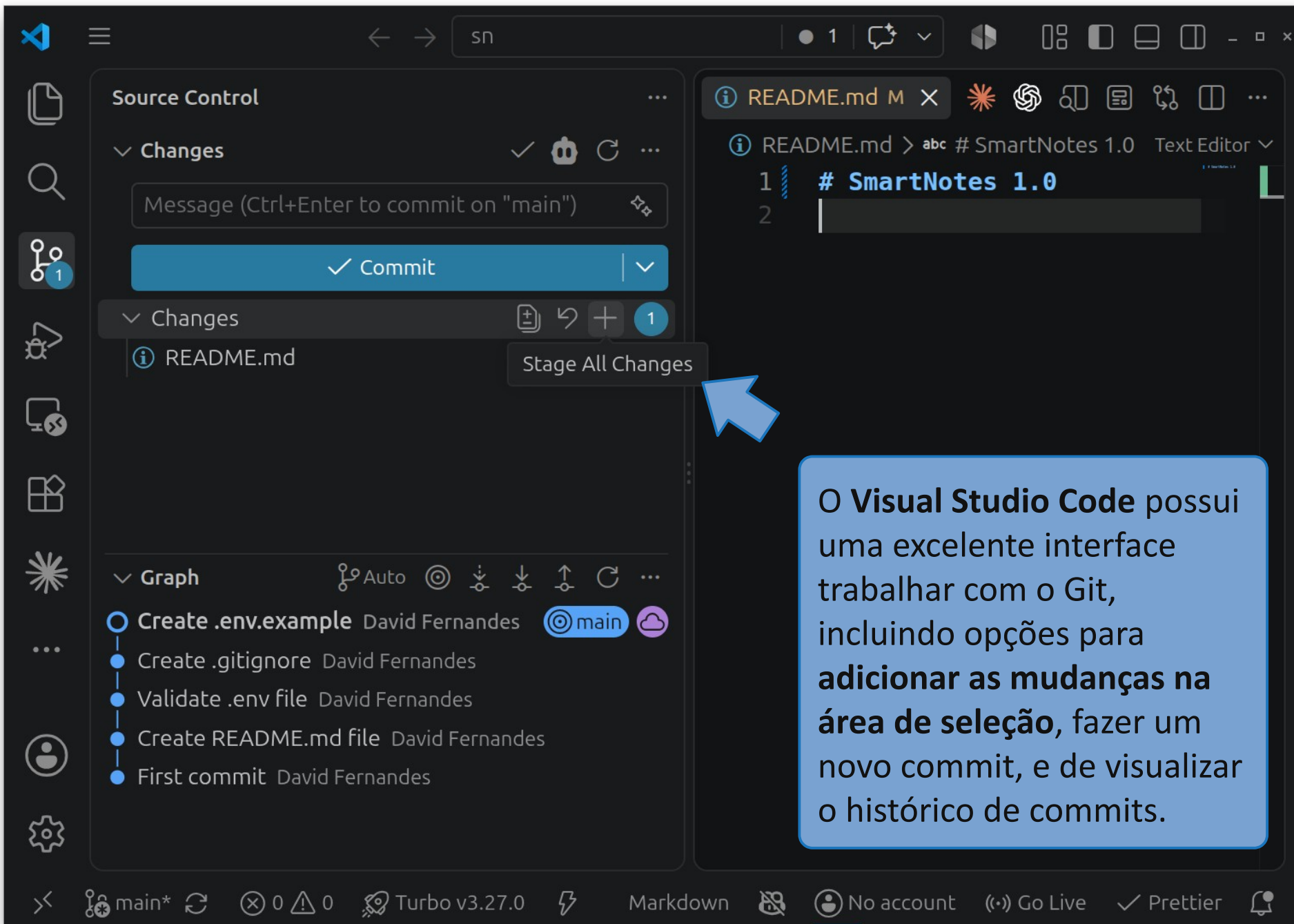
Alterando um arquivo

- Os arquivos também podem ser editados usando editores de texto ou outros recursos de seu SO
- Novamente, usamos a ferramenta **git status** para identificar os estados dos arquivos do repositório

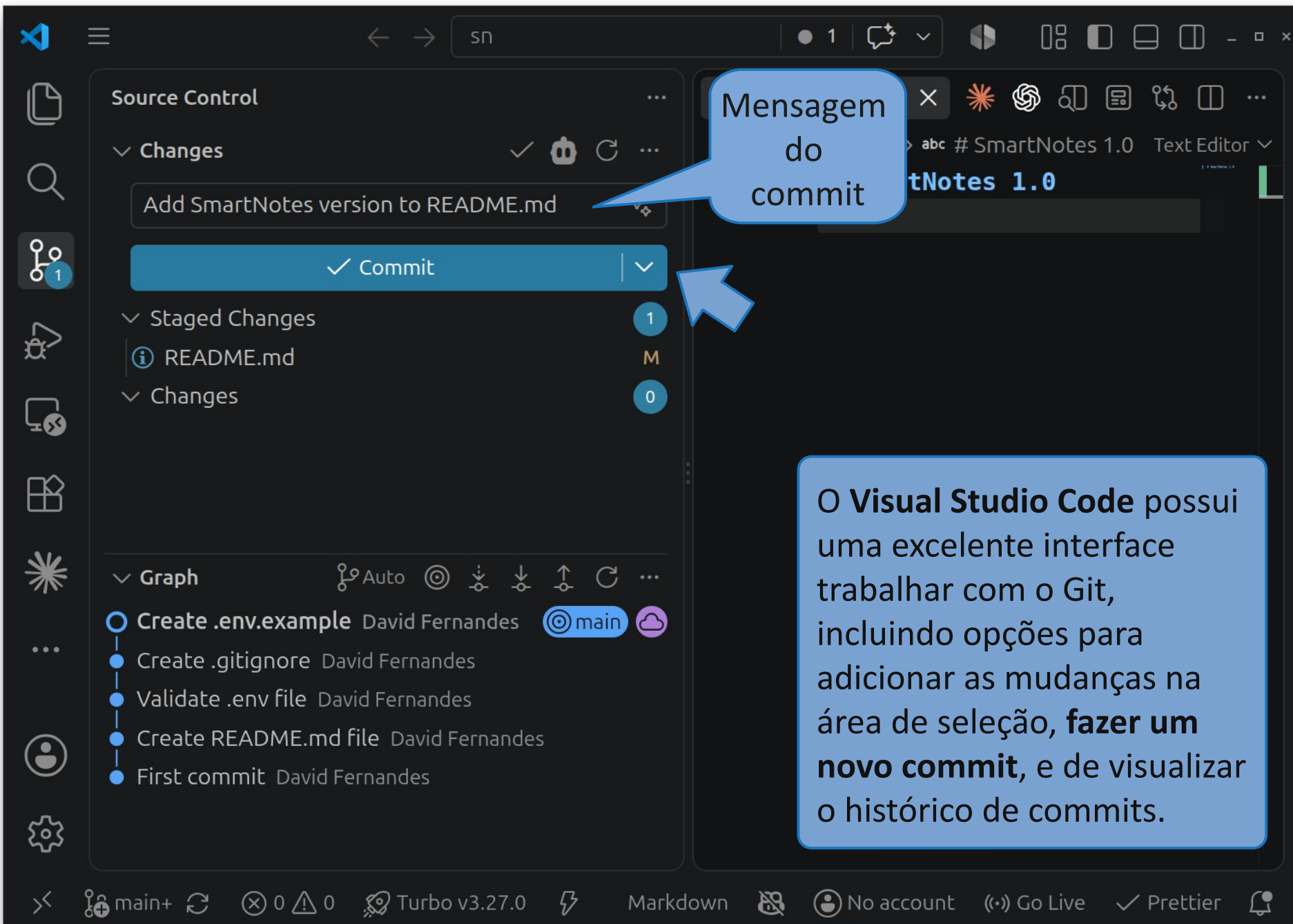
```
david@coyote:~/dev/sn
david ~/dev/sn on ʘ main $ echo "# SmartNotes 1.0" > README.md
david ~/dev/sn on ʘ main $ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

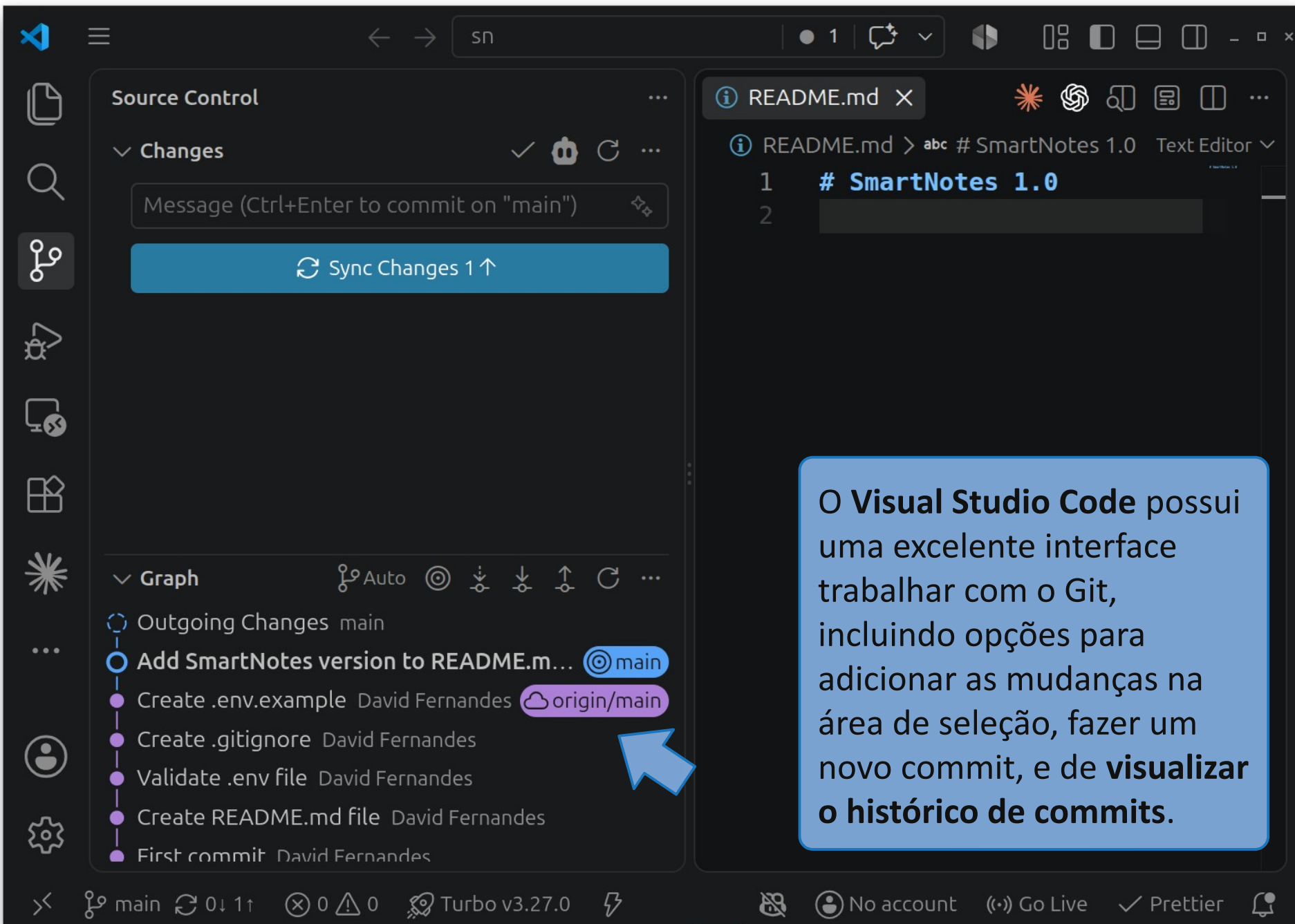
no changes added to commit (use "git add" and/or "git commit -a")
david ~/dev/sn on ʘ main $
```





O **Visual Studio Code** possui uma excelente interface trabalhar com o Git, incluindo opções para **adicionar as mudanças na área de seleção**, fazer um novo commit, e de visualizar o histórico de commits.





O **Visual Studio Code** possui uma excelente interface trabalhar com o Git, incluindo opções para adicionar as mudanças na área de seleção, fazer um novo commit, e de **visualizar o histórico de commits**.

Visualizando o Histórico de Commits

- Para ver o histórico de commits de um repositório, podemos utilizar o **git log**

```
david@coyote:~/dev/sn
david ~/dev/sn on ↵ main $ git log
commit 57f3e42da56efa5671ad2ead7a915eb165f39923 (HEAD -> main)
Author: David Fernandes <david@icomp.ufam.edu.br>
Date: Sun Aug 23 04:48:54 2026 -0400

    Add SmartNotes version to README.md

commit 28dc27999b075ac7945bb5274ab14de76c43cb4b
Author: David Fernandes <david@icomp.ufam.edu.br>
Date: Sun Aug 23 04:45:42 2026 -0400

    Create .env.example

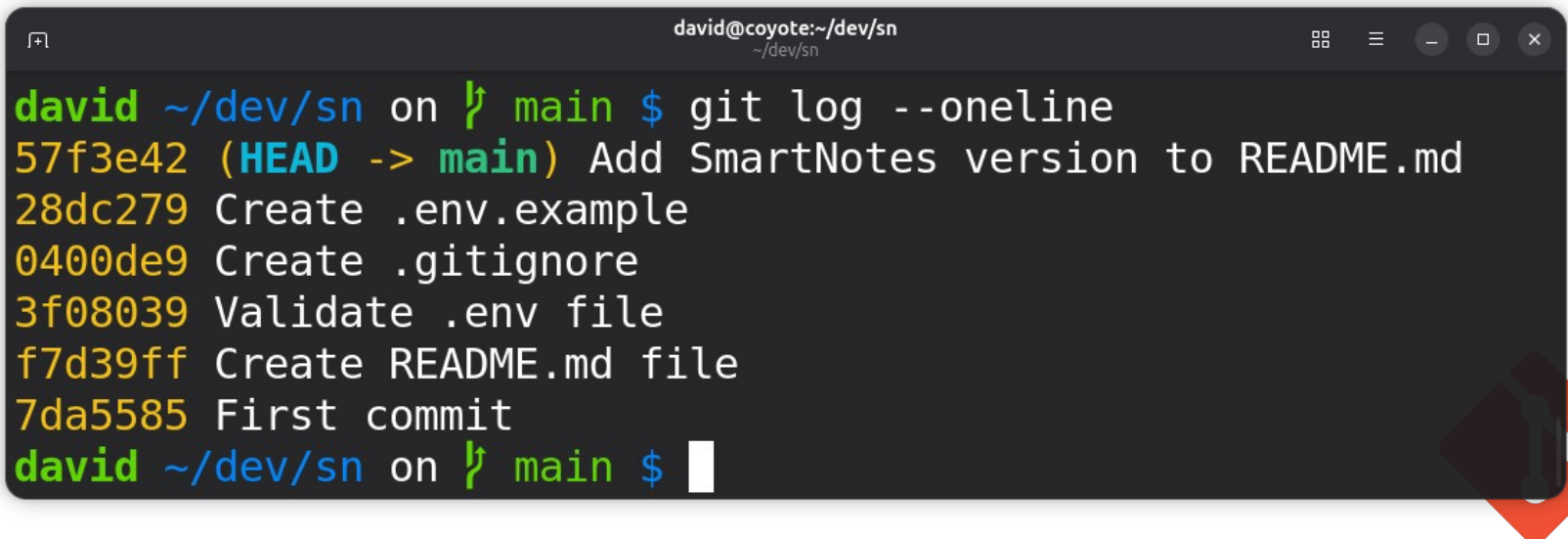
commit 0400de94c2f17c7b51e31e58aa55036b4a7a295b
Author: David Fernandes <david@icomp.ufam.edu.br>
Date: Sun Aug 23 04:43:02 2026 -0400

    Create .gitignore
```



Visualizando o Histórico de Commits

- O comando **git log** possui um conjunto de opções úteis:
 - **--oneline** - uma linha por commit
 - **--graph** - desenha a topologia dos branches
 - **--author** - filtra por autoria
 - **--<caminho>** - só commits que envolveram o arquivo
 - **-S <termo>** - commits que adicionaram ou removeram o termo nos arquivos
 - **-p** - mostra o diff de cada commit

A terminal window with a dark background and light-colored text. The window title is 'david@coyote:~/dev/sn'. The prompt is 'david ~/dev/sn on ̣ main \$'. The command 'git log --oneline' has been executed, showing a list of commits. The output is: '57f3e42 (HEAD -> main) Add SmartNotes version to README.md', '28dc279 Create .env.example', '0400de9 Create .gitignore', '3f08039 Validate .env file', 'f7d39ff Create README.md file', and '7da5585 First commit'. The prompt is now 'david ~/dev/sn on ̣ main \$' followed by a cursor.

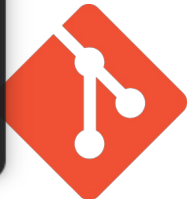
```
david@coyote:~/dev/sn
57f3e42 (HEAD -> main) Add SmartNotes version to README.md
28dc279 Create .env.example
0400de9 Create .gitignore
3f08039 Validate .env file
f7d39ff Create README.md file
7da5585 First commit
david ~/dev/sn on ̣ main $
```

Visualizando as Mudanças

- O comando **git status** informa quais arquivos foram modificados em um dado momento do fluxo de trabalho
- Para obter informações sobre o que foi modificado dentro dos arquivos você pode usar o **git diff**
- Por exemplo, para verificar as diferenças entre os arquivos do diretório de trabalho e a área de seleção:

```
david@coyote:~/dev/sn
~ /dev/sn

david ~/dev/sn on ̣ main $ echo "# SmartNotes 2.0" > README.md
david ~/dev/sn on ̣ main $ git diff
diff --git a/README.md b/README.md
index fc872a0..e0a5cc9 100644
--- a/README.md
+++ b/README.md
@@ -1,1 @@
-# SmartNotes 1.0
+# SmartNotes 2.0
david ~/dev/sn on ̣ main $
```



Visualizando as Mudanças

- O **git diff** também é usado para listar as diferenças entre os arquivos do diretório de trabalho e o último commit

```
david@coyote:~/dev/sn  
david ~/dev/sn on ʘ main $ git diff HEAD  
diff --git a/README.md b/README.md  
index fc872a0..e0a5cc9 100644  
--- a/README.md  
+++ b/README.md  
@@ -1,1 @@  
-# SmartNotes 1.0  
+# SmartNotes 2.0  
david ~/dev/sn on ʘ main $
```



Visualizando as Mudanças

git diff

Diretório de trabalho × área de seleção

git diff --staged

Área de seleção × último commit

git diff HEAD

Diretório de trabalho × último commit

git diff main develop

Diferença entre as branches main e develop

git show <hash>

Metadados e diff completo de um commit específico com o commit anterior.

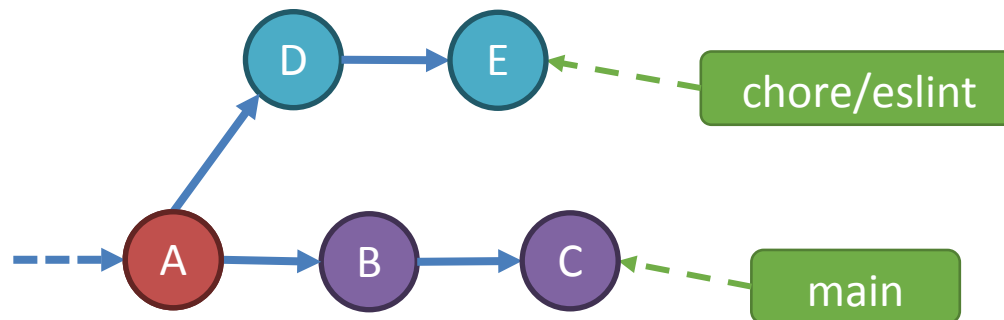
git blame <arquivo>

Para cada linha, o commit e o autor que a introduziu.

```
diff --git a/src/juros.ts b/src/juros.ts
@@ -12,7 +12,7 @@ export function calcular(valor: number) {
-   return valor * 0.02;
+   return valor * taxaVigente();
```

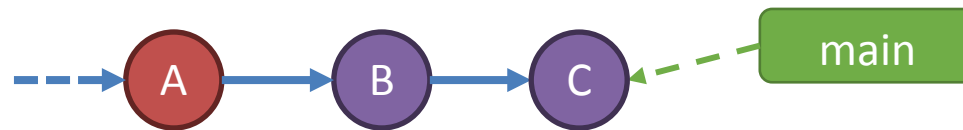
O que é uma branch?

- Uma **branch** é uma linha independente de desenvolvimento dentro do mesmo repositório
- Ela permite trabalhar em uma nova funcionalidade ou correção sem alterar imediatamente a versão principal do projeto
- Uma branch não é uma cópia completa do projeto, é apenas um **ponteiro para um commit**

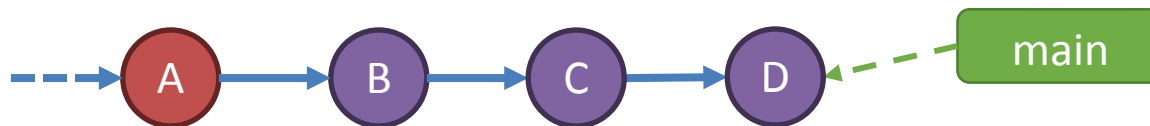


O que é uma branch?

- Quando um **novo commit** é criado, a branch atual passa a apontar para ele
- Antes do commit:



- Depois do commit:



- O Git armazena cada commit e move o ponteiro da branch conforme novos commits são criados



O que é uma branch?

- Quando um **novo commit** é criado, a branch atual passa a apontar para ele

```
david@coyote:~/dev/sn
b2d89ea (HEAD -> main, origin/main) Add SmartNotes version to README.md
51da999 Create .env.example
d128176 Create .gitignore
9f30563 Validate .env file
703dea3 Create README.md file
7d2a909 First commit
david ~/dev/sn on  main $
```



- O Git armazena cada commit e move o ponteiro da branch conforme novos commits são criados



Comandos para Branches

git branch

Lista branches locais

git branch -a

Lista branches locais e remotas

git branch nome-da-branch

Cria a branch, mas não muda para ela.

git switch -c nome-da-branch

Cria a branch e muda para ela

git switch nome-da-branch

Muda a branch atual (branch para onde o HEAD está apontando)

git branch -

Volta para a branch anterior

git branch -m novo-nome

Renomeia a branch atual

git branch -d nome-da-branch

Exclui a branch nome-da-branch



Comandos para Branches

git branch

Lista branches locais

git branch -a

Lista branches locais e remotas

```
david@coyote:~/dev/sn
david ~/dev/sn on  ↳ main $ git branch chore/eslint
david ~/dev/sn on  ↳ main $ git switch chore/eslint
Switched to branch 'chore/eslint'
david ~/dev/sn on  ↳ chore/eslint $
```

git switch nome-da-branch Muda a branch atual (branch para onde o HEAD está apontando)

git branch - Volta para a branch anterior

git branch -m novo-nome

Renomeia a branch atual

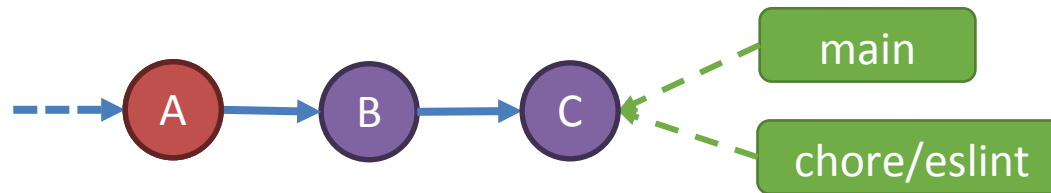
git branch -d nome-da-branch

Exclui a branch nome-da-branch



Criando um feature branch

- Ao criar uma branch **chore/eslint** a partir da branch **main**, as duas branches permanecem apontando para o mesmo commit



- Exemplos comuns de nomes para branches:
 - feat/login
 - chore/setup-orm
 - fix/update-user-validation
 - docs/update-readme
 - refactor/user-service



Criando um feature branch

- Ao criar uma branch **chore/eslint** a partir da branch main, as

```
david@coyote:~/dev/sn  
david ~/dev/sn on ✎ chore/eslint $ git log --oneline  
b2d89ea (HEAD -> chore/eslint, origin/main, main) Add SmartNotes  
version to README.md  
51da999 Create .env.example  
d128176 Create .gitignore  
9f30563 Validate .env file  
703dea3 Create README.md file  
7d2a909 First commit  
david ~/dev/sn on ✎ chore/eslint $
```

- fix/update-user-validation
- docs/update-readme
- refactor/user-service

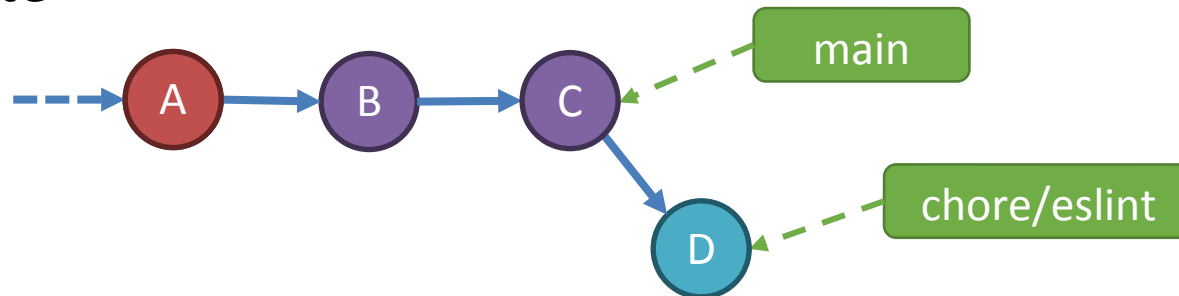


Criando um feature branch

- Após finalizar a instalação do ESLint no projeto, podemos criar um novo commit

```
david@coyote:~/dev/sn
david ~/dev/sn on ✎ chore/eslint $ git add .
david ~/dev/sn on ✎ chore/eslint $ git commit -m "Install ESLint"
[chore/eslint 897a605] Install ESLint
4 files changed, 390 insertions(+), 20 deletions(-)
david ~/dev/sn on ✎ chore/eslint $
```

- A partir disso, cada branch passa a apontar para um commit diferente

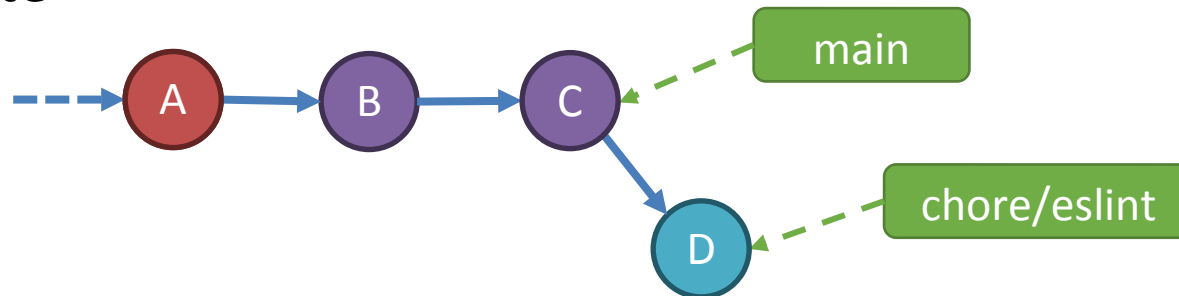


Criando um feature branch

- Após finalizar a instalação do ESLint no projeto, podemos

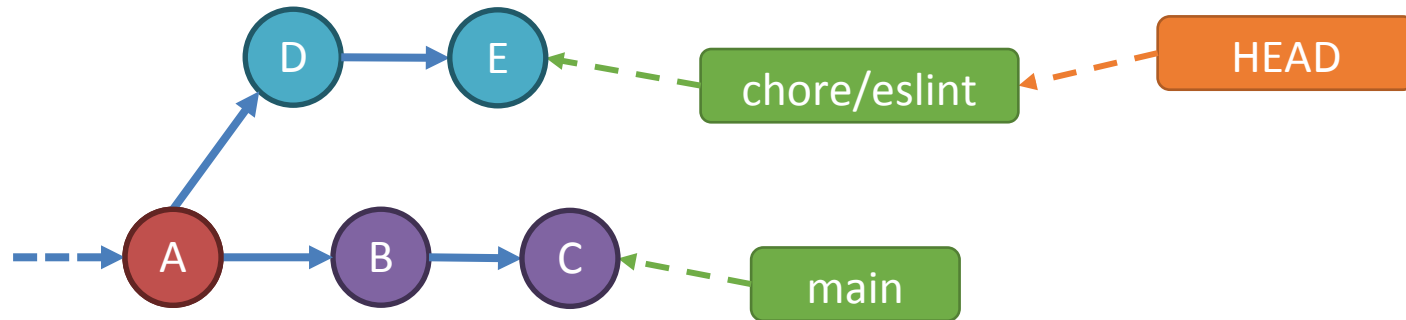
```
david@coyote:~/dev/sn
git log --oneline
7b42882 (HEAD -> chore/eslint) Install ESLint
b2d89ea (origin/main, main) Add SmartNotes version to README.md
51da999 Create .env.example
d128176 Create .gitignore
9f30563 Validate .env file
703dea3 Create README.md file
7d2a909 First commit
david@coyote:~/dev/sn on   chore/eslint $
```

A partir disso, cada branch passa a apontar para um commit diferente



O que é o HEAD?

- O HEAD indica onde o usuário está trabalhando atualmente, e normalmente é apenas um ponteiro para uma branch

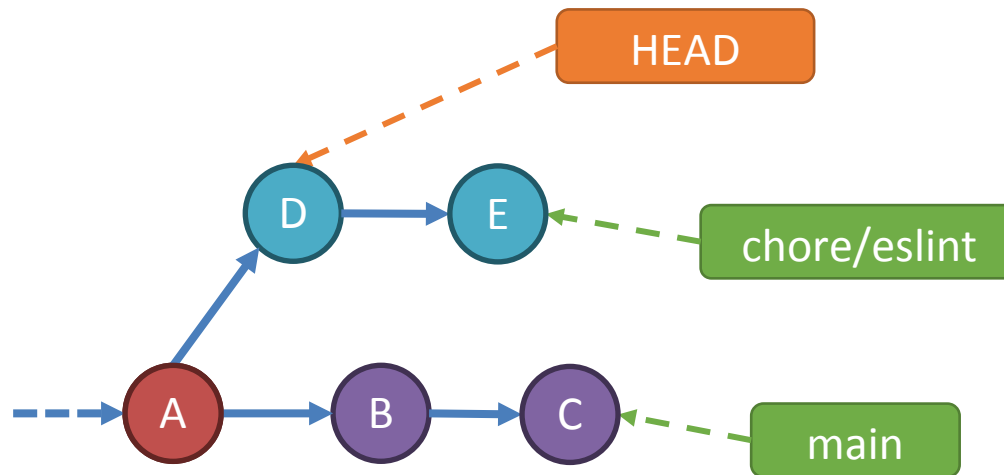


- Quando um commit é criado em uma branch:
 - o Git cria o novo commit
 - a branch atual passa a apontar para ele
 - o HEAD continua apontando para a branch



Detached HEAD

- Em algumas situações, podemos apontar o HEAD diretamente para um commit, e não para uma branch
 - Isso é feito com o comando **git switch --detach <commit>**



- Cuidado:** É possível criar commits nesse estado, mas eles não pertencem automaticamente a uma branch

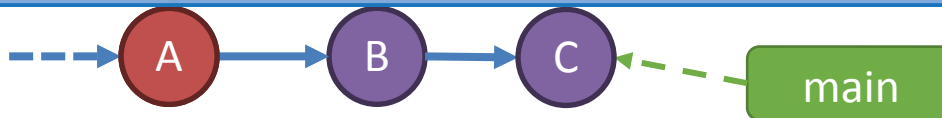


Detached HEAD

- Em algumas situações, podemos apontar o HEAD diretamente para um commit, e não para uma branch
 - Isso é feito com o comando **git switch --detach <commit>**

Detached HEAD é útil principalmente para:

- inspecionar uma versão antiga do projeto
- testar o código de um commit específico
- investigar quando um bug apareceu
- comparar o comportamento atual com uma versão anterior



- **Cuidado:** É possível criar commits nesse estado, mas eles não pertencem automaticamente a uma branch



Mesclando Branches

- Depois da instalação e configuração do ESLint, podemos fazer o merge entre as branches **chore/eslint** e **main**

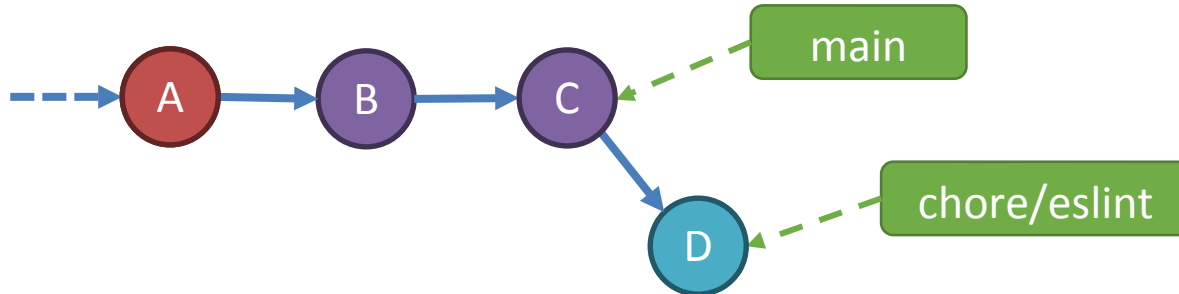
```
david@coyote:~/dev/sn
~/dev/sn
david ~/dev/sn on ✎ chore/eslint $ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
david ~/dev/sn on ✎ main $ git merge chore/eslint
Updating b2d89ea..5a08614
Fast-forward
 eslint.config.js |      9 +
 package-lock.json | 1219 ++++++
 package.json      |      6 +-
 3 files changed, 1227 insertions(+), 7 deletions(-)
 create mode 100644 eslint.config.js
david ~/dev/sn on ✎ main $
```

- Note que a branch que recebe as alterações deve estar ativa

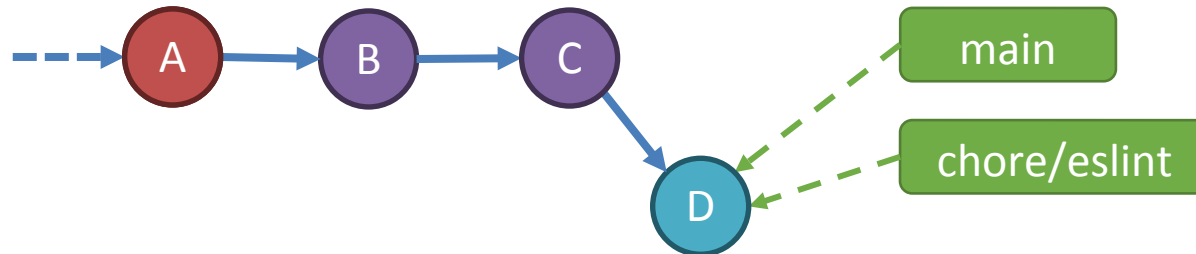


Merge fast-forward

- O **fast-forward** ocorre quando a branch de destino não recebeu novos commits depois da criação da outra branch
- Antes:



- Depois:

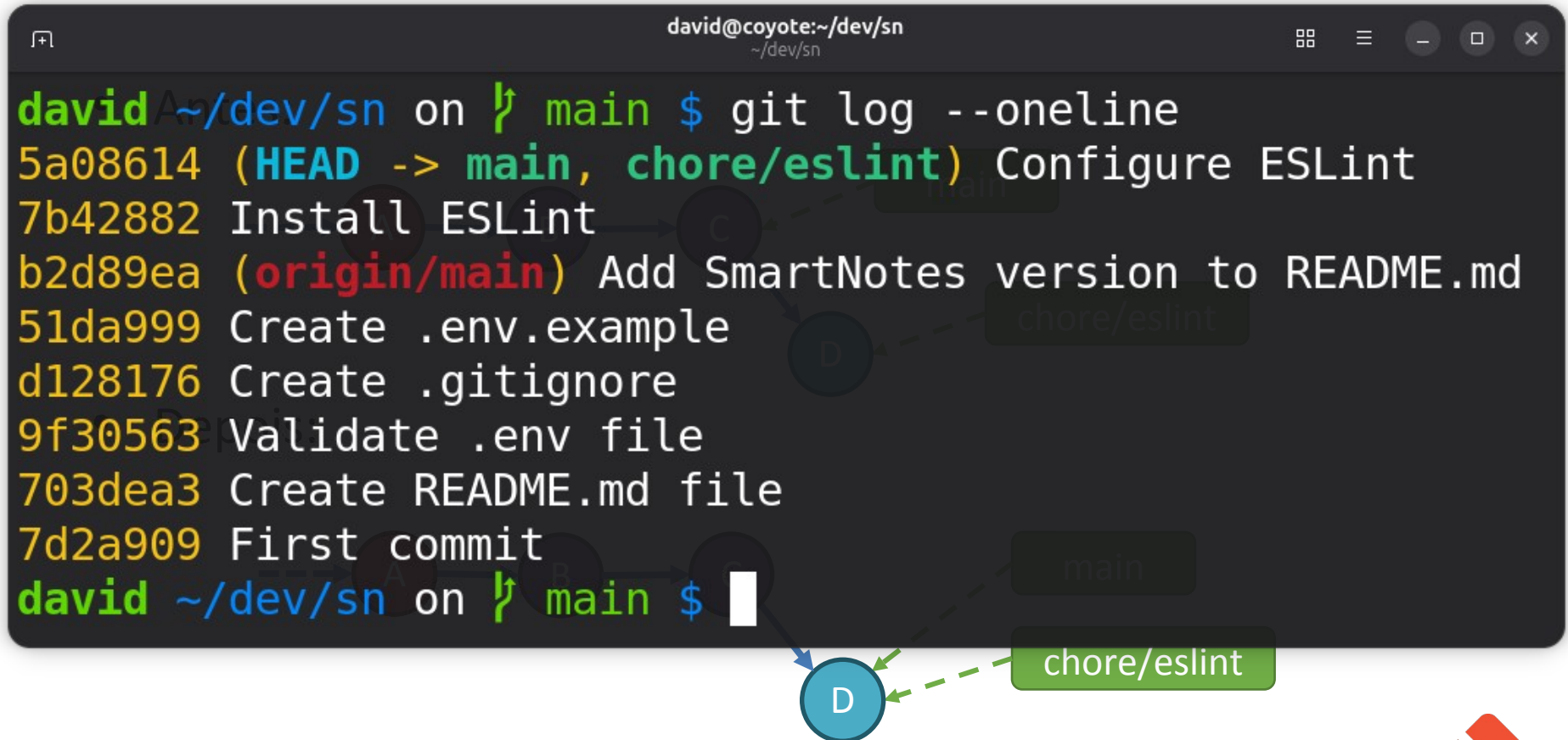


Merge fast-forward

- O **fast-forward** ocorre quando a branch de destino não

```
david@coyote:~/dev/sn
~ /dev/sn

david ~/dev/sn on ǃ main $ git log --oneline
5a08614 (HEAD -> main, chore/eslint) Configure ESLint
7b42882 Install ESLint
b2d89ea (origin/main) Add SmartNotes version to README.md
51da999 Create .env.example
d128176 Create .gitignore
9f30563 Validate .env file
703dea3 Create README.md file
7d2a909 First commit
david ~/dev/sn on ǃ main $
```

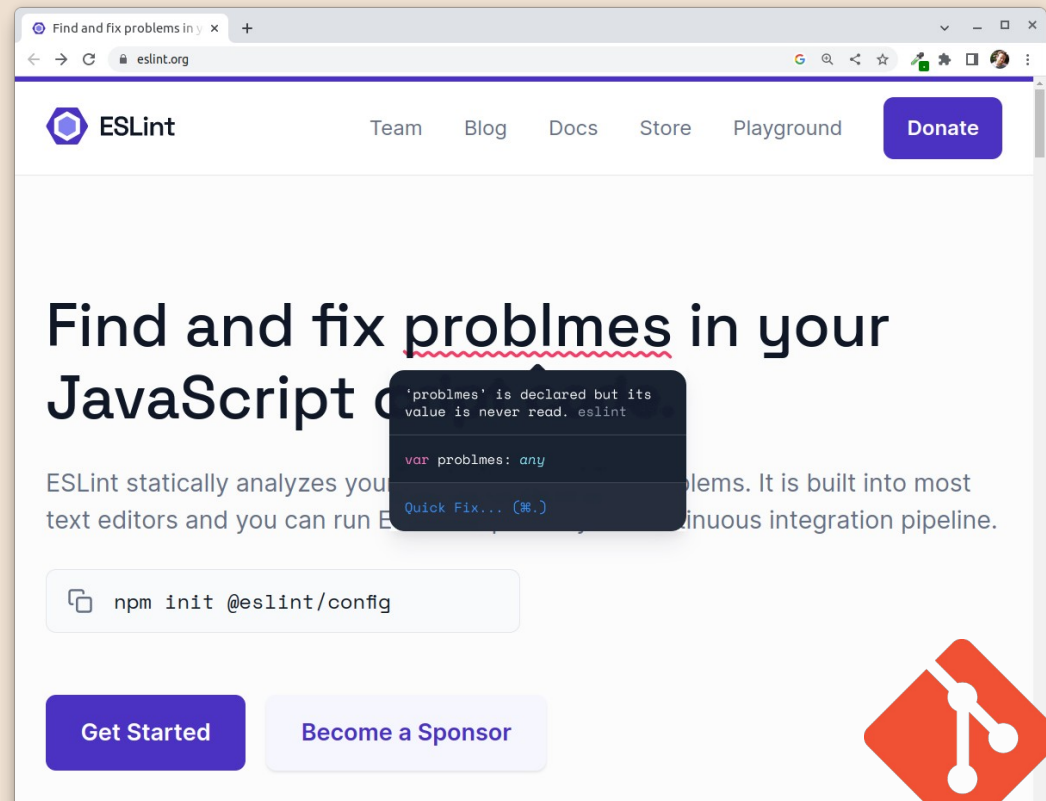


The diagram illustrates the Git commit history and branch structure. The commit history is shown as a vertical list of commits with their hashes and descriptions. The branches are represented by circles: 'main' (blue) and 'chore/eslint' (green). The 'chore/eslint' branch is shown as a fast-forward merge of the 'main' branch, indicated by a solid green arrow pointing from the 'main' branch to the 'chore/eslint' branch. The 'main' branch is labeled 'main' and the 'chore/eslint' branch is labeled 'chore/eslint'.



Exercício

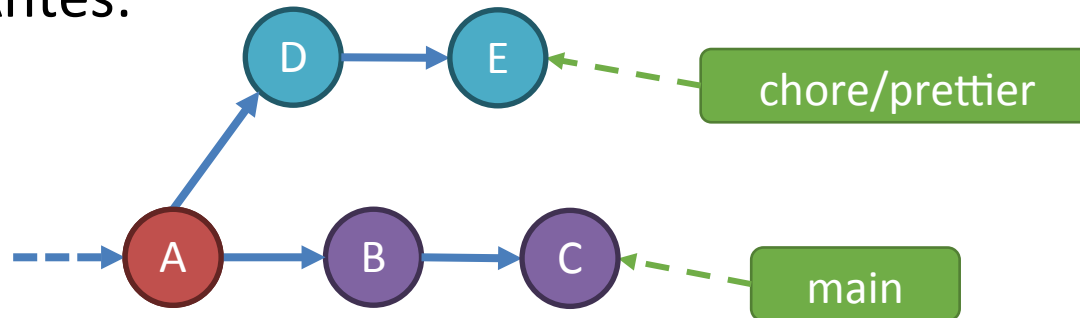
- Crie uma branch **chore/eslint** em seu projeto, e então instale e configure o **ESLint** nessa branch
- Após isso, faça o merge da branch **chore/eslint** na **main** usando **fast-forward**
 - Para isso, siga as instruções dos slides sobre o **SmartNotes**, até a seção **Adicionando o ESLint no projeto**



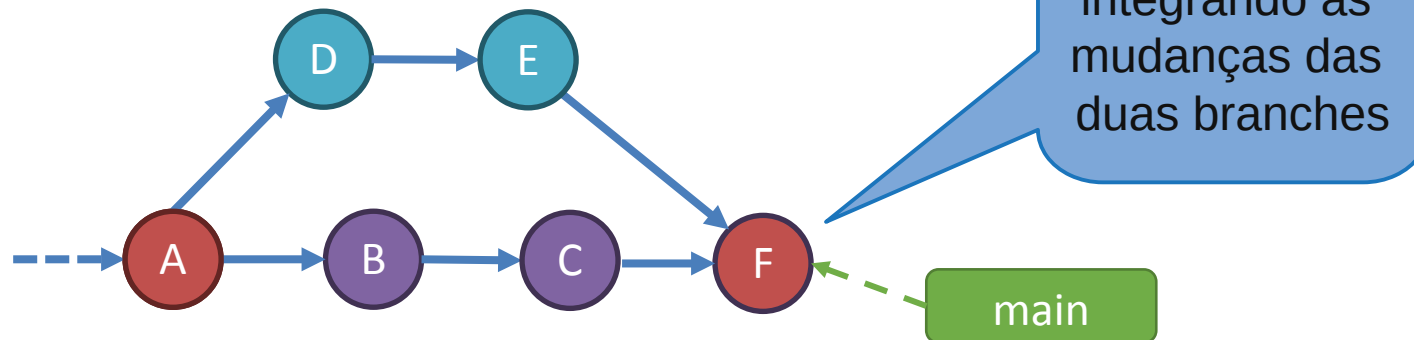
Merge commit

- Um **merge commit** é necessário quando as duas branches avançaram separadamente

- Antes:



- Depois:



Merge commit

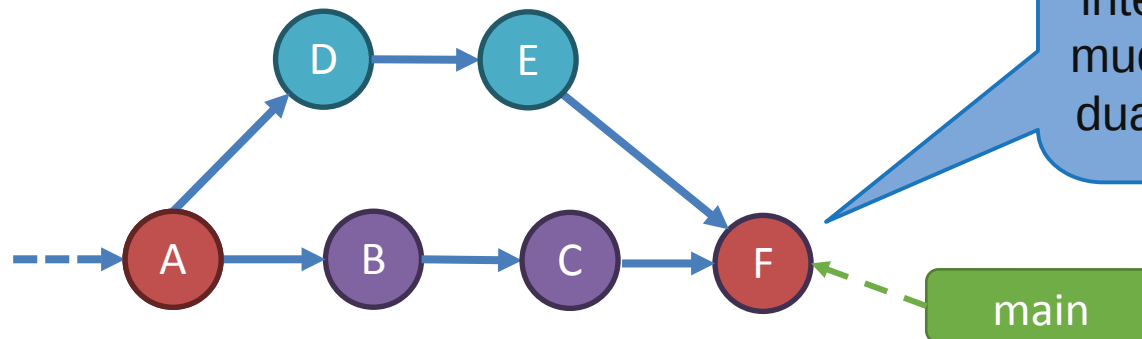
- Um **merge commit** é necessário quando as duas branches avançaram separadamente
- Antes:



O commit F possui dois commits pai: (i) o último commit da **main**, e (ii) o último commit da **chore/prettier**.

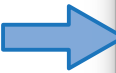
Ao fazer o merger, o Git abre um editor para confirmar a mensagem do novo commit: **Merge branch 'chore/prettier'**

- Depois.



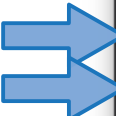
Exemplo de Merge commit

- Commits da branch **main**:



```
david@coyote:~/dev/sn
~/dev/sn
david ~/dev/sn on ̣ main $ git log --oneline -4
40ad73f (HEAD -> main) Update README
5a08614 (origin/main, chore/eslint) Configure ESLint
7b42882 Install ESLint
b2d89ea Add SmartNotes version to README.md
david ~/dev/sn on ̣ main $
```

- Commits da branch **chore/prettier**:



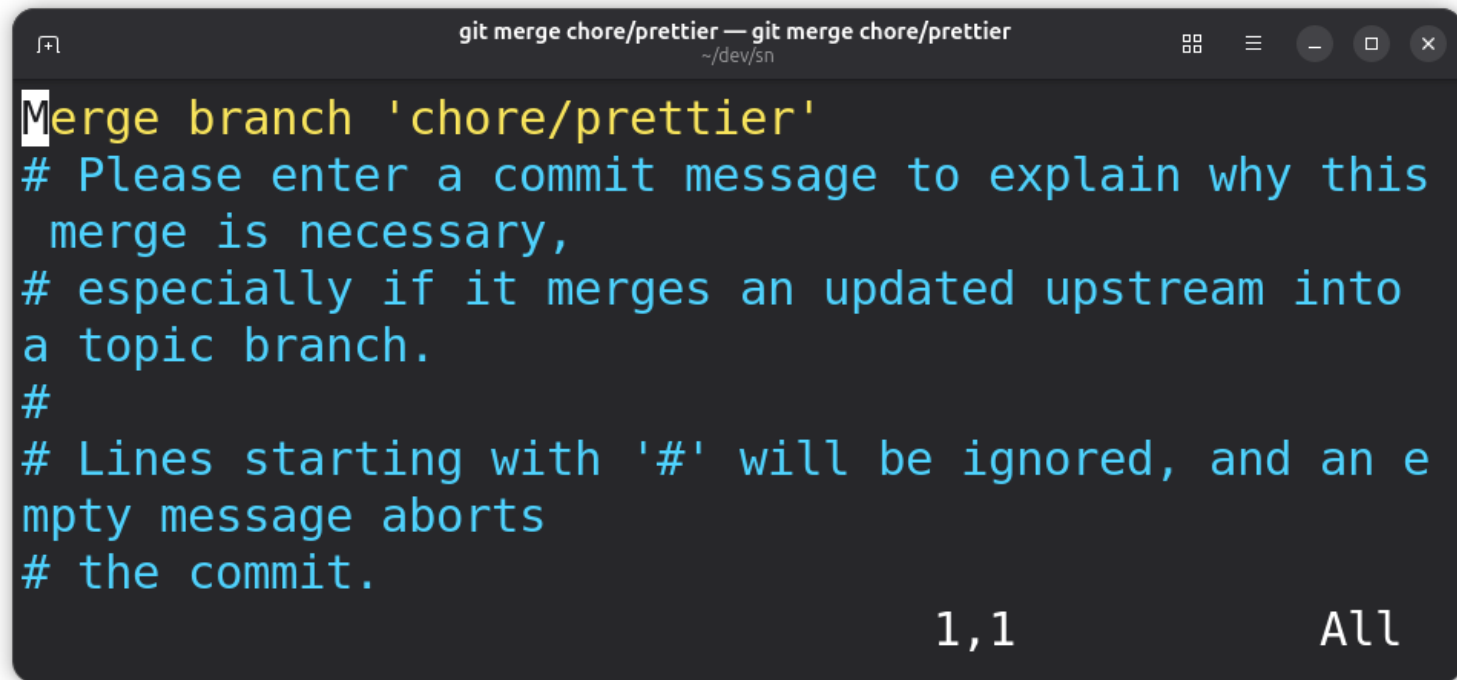
```
david@coyote:~/dev/sn
~/dev/sn
david ~/dev/sn on ̣ chore/prettier $ git log --oneline -4
9af3892 (HEAD -> chore/prettier) Configure Prettier
711d082 Install Prettier
5a08614 (origin/main, main, chore/eslint) Configure ESLint
7b42882 Install ESLint
david ~/dev/sn on ̣ chore/prettier $
```



Exemplo de Merge commit

- Ao executar o comando merge a partir de branch **main**, um editor é aberto para edição da mensagem do **merge commit**

```
$ git merge chore/prettier
```



The screenshot shows a terminal window titled "git merge chore/prettier — git merge chore/prettier" with the path "~/dev/sn". The window displays the Git merge commit message editor. The text inside the editor is as follows:

```
Merge branch 'chore/prettier'  
# Please enter a commit message to explain why this  
# merge is necessary,  
# especially if it merges an updated upstream into  
# a topic branch.  
#  
# Lines starting with '#' will be ignored, and an empty  
# message aborts the commit.
```

At the bottom right of the editor, the text "1,1" and "All" are visible.



Exemplo de Merge commit

- Ao executar o comando merge a partir de branch **main**, um

O nome **ORT**, **Ostensibly Recursive's Twin** (em português: “aparentemente, o gêmeo da Recursive”) é uma brincadeira com o nome da estratégia anterior, que era chamada de **Recursive**

```
Merge branch 'chore/prettier'
# david@coyote:~/dev/sn on 1/1 main $ git merge chore/prettier
Merge made by the 'ort' strategy.
# .prettierrc | 8 ++++++++
# package-lock.json | 17 ++++++++
# package.json | 1 +
mp 3 files changed, 26 insertions(+)
# create mode 100644 .prettierrc
david ~/dev/sn on 1/1 main $
```

took 42s

Ostensibly
Recursive's
Twin (ort)

Exemplo de Merge commit

- Como resultado, os commits da branch **chore/prettier** são mesclados com os commits da branch **main**

```
david@coyote:~/dev/sn
b57e369 (HEAD -> main) Merge branch 'chore/prettier'
40ad73f Update README
9af3892 (chore/prettier) Configure Prettier
711d082 Install Prettier
david ~/dev/sn on  main $
```

- Note que, além dos commits das duas branches, o HEAD está apontando para um commit adicional, criado a partir do **merge commit**

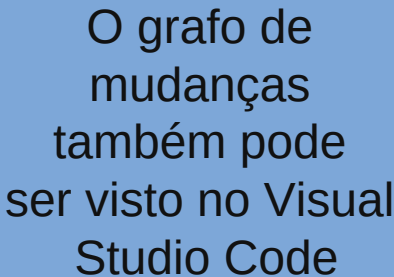


Exemplo de Merge commit

- A opção **--graph** do comando **git log** mostra os commits que participaram do merge entre duas branches

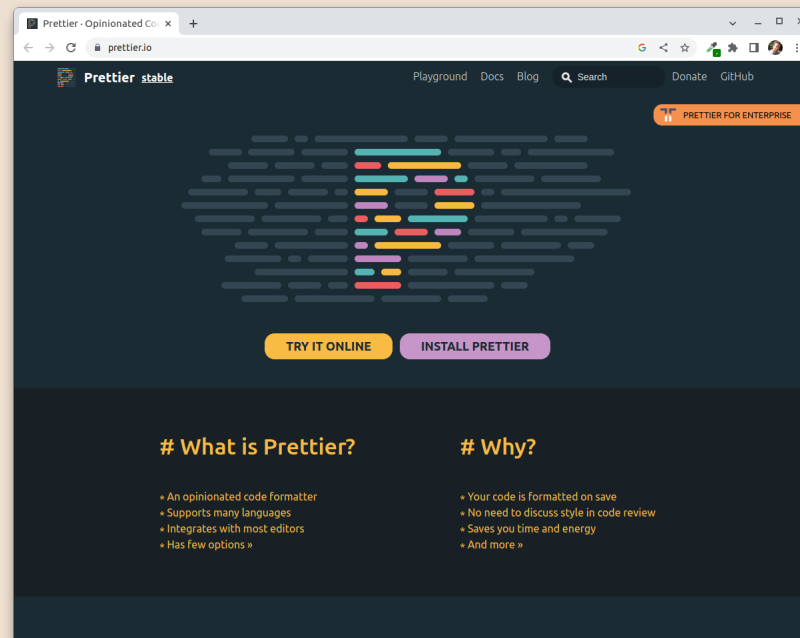
```
david@coyote:~/dev/sn
david ~/dev/sn on ̣ main $ git log --oneline --graph
* b57e369 (HEAD -> main) Merge branch 'chore/prettier'
| \
| * 9af3892 (chore/prettier) Configure Prettier
| * 711d082 Install Prettier
| * | 40ad73f Update README
| /
* 5a08614 (origin/main, chore/eslint) Configure ESLint
* 7b42882 Install ESLint
* b2d89ea Add SmartNotes version to README.md
* 51da999 Create .env.example
* d128176 Create .gitignore
* 9f30563 Validate .env file
* 703dea3 Create README.md file
* 7d2a909 First commit
david ~/dev/sn on ̣ main $
```





Exercício

- Crie uma branch **chore/prettier** em seu projeto, e então instale e configure o **Prettier** nessa branch
- Volte para a branch main, adicione algumas informações no README.md sobre o seu projeto, e faça um commit
- Após isso, faça o merge da branch **chore/prettier** na **main** usando **merge commit**
 - Para isso, siga as instruções dos slides sobre o **SmartNotes**, até a seção **Adicionando o Prettier no projeto**



Conflitos de Merge

- Um conflito ocorre quando o Git não consegue fazer o merge automático das alterações de duas branches
- Considere, por exemplo, as alterações abaixo, que ocorreram na mesma linha do README.md em duas branches distintas:
 - Na branch **main**:

Descrição

API para gerenciamento de notas pessoais.

- Na branch **feat/router**:

Descrição

API REST para criação e organização de notas pessoais.

- Ao fazer o merge de **feat/router** na **main**, o Git cancela o merge para que o conflito seja resolvido manualmente



Conflitos de Merge

- Um conflito ocorre quando o Git não consegue fazer o merge automático das alterações de duas branches diferentes.
- Considere, por exemplo, a seguinte situação: duas alterações ocorreram na mesma linha do `README.md` em branches distintas:
 - Na branch **main**:

Descrição

API para gerenciamento de notas pessoais.

- Na branch **feat/router**:

Descrição

API REST para criação e organização de notas pessoais.

- Ao fazer o merge de **feat/router** na **main**, o Git cancela o merge para que o conflito seja resolvido manualmente

Note que nenhuma mudança pode ser feita diretamente na `main`. Essa mudança deve ter ocorrido a partir do merge com outra branch



Conflitos de Merge

```
david@coyote:~/dev/sn  
~/dev/sn  
david ~/dev/sn on ✎ feat/router $ git log --oneline -4  
fc2dc41 (HEAD -> feat/router) Create the main API router  
b57e369 Merge branch 'chore/prettier'  
40ad73f Update README  
9af3892 (chore/prettier) Configure Prettier  
david ~/dev/sn on ✎ feat/router $
```

Descrição

```
david@coyote:~/dev/sn  
~/dev/sn  
david ~/dev/sn on ✎ main $ git log --oneline -4  
6ba934f (HEAD -> main) Add description to README.md  
b57e369 Merge branch 'chore/prettier'  
40ad73f Update README  
9af3892 (chore/prettier) Configure Prettier  
david ~/dev/sn on ✎ main $
```



Conflitos de Merge

- Um conflito ocorre quando o Git não consegue fazer o merge automático das alterações de duas branches

```
david@coyote:~/dev/sn
david ~/dev/sn on ̣ main $ git merge feat/router
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the
result.
david ~/dev/sn on ̣ main $
```



Conflicts de Merge

```
david@coyote:~/dev/sn
~ /dev/sn

david ~/dev/sn on ʘ main $ git status
On branch main
Your branch is ahead of 'origin/main' by 5 commits.
(use "git push" to publish your local commits)

You have unmerged paths.
(fix conflicts and run "git commit")
(use "git merge --abort" to abort the merge)

Changes to be committed:
  modified:   src/index.ts
  new file:   src/router/router.ts

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   README.md

david ~/dev/sn on ʘ main $
```



Conflitos de Merge

- O Git modifica o arquivo README.md e adiciona **marcadores**

```
david@coyote:~/dev/sn
~/dev/sn
david ~/dev/sn on ȳ main $ cat README.md
# SmartNotes 2.0

## Descrição

<<<<<<< HEAD
API para gerenciamento de notas pessoais.
=====
API REST para criação e organização de notas pessoais.
>>>>>>> feat/router
david ~/dev/sn on ȳ main $
```

- Esses marcadores não podem permanecer no código final, e para resolver o conflito o arquivo deve ser editado, escolhendo ou combinando as alterações



Resolvendo Conflitos

- Para resolver o conflito, deve-se seguir esses passos:
 - Identificar os arquivos em conflito com **git status**
 - Abrir cada arquivo que deu conflito e localizar os marcadores:

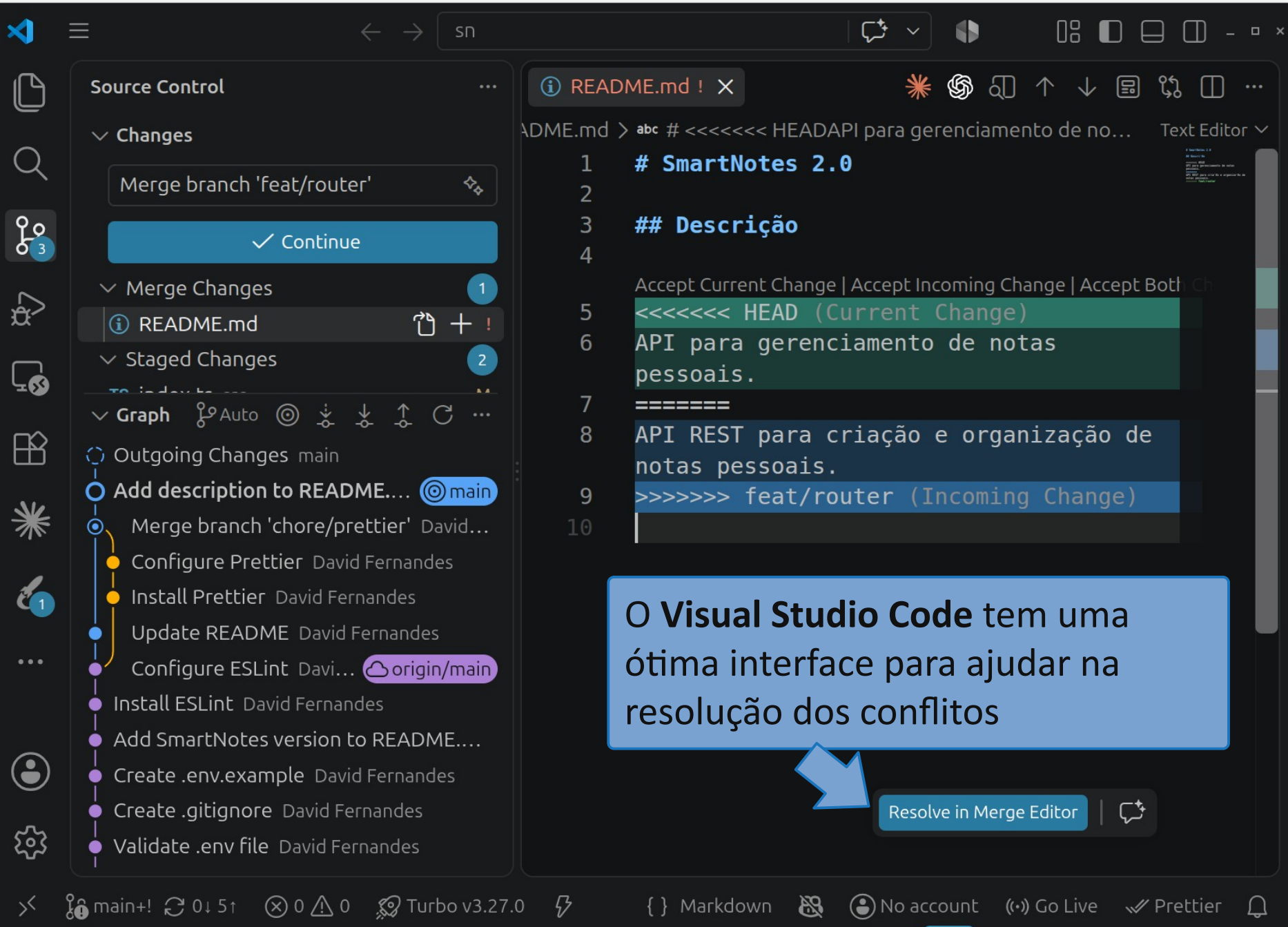
```
<<<<<<  
=====  
>>>>>>
```

- Escolher ou combinar as alterações

```
API REST para criação e organização de notas pessoais.
```

- Remover os marcadores
- Marcar o conflito como resolvido com o comando **git add**
- Concluir o merge com **git commit**





1

3

1

☰

← → sn

🔗

🌐

📄

🔍

📁

🔧

🔌

🔔

👤

⚙️

🔔 README.md !

🔔 Merging: README.md !

🔔 README.md > abc # SmartNotes 2.0

Text Editor ▼

Incoming ⚡ fc2dc41 • refs/heads/feat/router

Current ⚡ 6ba934f • refs/heads/main

1

2

3

4

5

6

SmartNotes 2.0

Accept Incoming | Accept Combination (Incoming First)

Descrição

API REST para criação e organização de notas pessoais.

1

2

3

4

5

6

SmartNotes 2.0

Accept Current | Accept Combination (Current First)

Descrição

API para gerenciamento de notas pessoais.

Result README.md

1 # SmartNotes 2.0

2 No Changes Accepted

maining ↶

Complete Merge

< 🔔 main+! 🔄 0 ↓ 5 ↑ 🚫 0 ⚠️ 0 🚀 Turbo v3.27.0 ⚡

{ } Markdown 🚫 👤 No account 🌐 Go Live ✓ Prettier 🔔

Se desejar a versão vindo da branch **feat/router**, clique em **Accept Incoming**. Se desejar manter a versão que está na branch **main**, clique em **Accept Current**.





 README.md ! ●

 Merging: README.md ! ●

README.md > abc # SmartNotes 2.0

Incoming φ fc2dc41 • refs/heads/feat/router

1 # SmartNotes 2.0

2

3 ## Descrição

4

5 API REST para criação e organização de notas pessoais.

6

Current φ 6ba934f • refs/heads/main

1 # SmartNotes 2.0

Accept Current | Accept Combination (Current First)

2

3 ## Descrição

4

5 API para gerenciamento de notas pessoais.

6

Result README.md

0 Conflicts Remaining

1 # SmartNotes 2.0

Incoming | Remove Incoming

2

3 ## Descrição

4

5 API REST para criação e organização de notas pessoais.

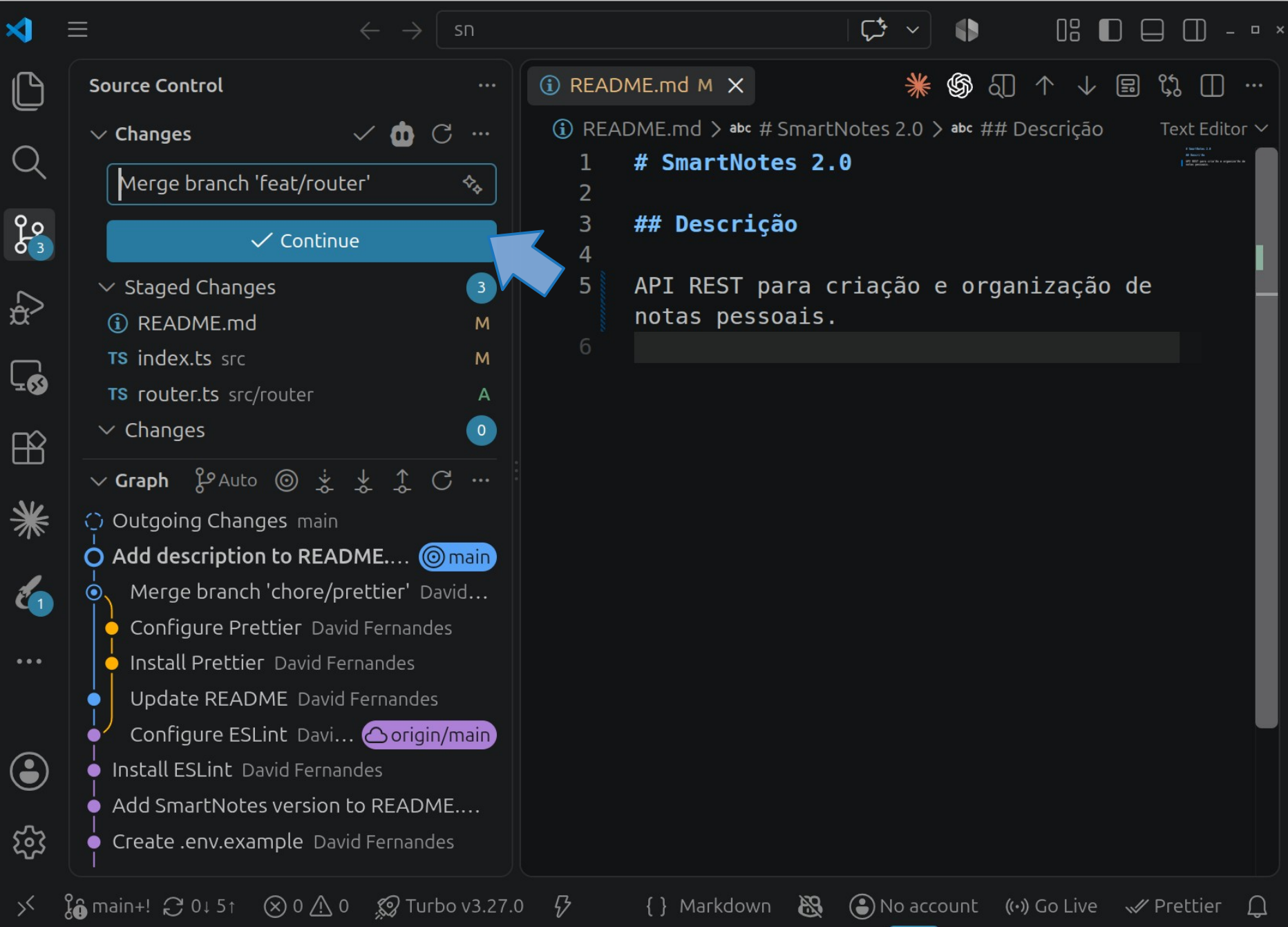
6



Complete Merge



main+! 0↓ 5↑ 0 0 Turbo v3.27.0 {} Markdown No account Go Live Prettier





sn



Source Control



Changes

Message (Ctrl+Enter to commi...

Sync Changes 7 ↑

Graph



Auto



Outgoing Changes main

Merge branch 'feat/router'... main

Create the main API router David Fe...

Add description to README.md Da...

Merge branch 'chore/prettier' David...

Configure Prettier David Fernandes

Install Prettier David Fernandes

Update README David Fernandes

Configure ESLint Davi... origin/main

Install ESLint David Fernandes

README.md X



README.md > abc # SmartNotes 2.0 > abc ## Descrição

Text Editor

1 # SmartNotes 2.0

2

3 ## Descrição

4

5 API REST para criação e organização de
notas pessoais.

6



main



0 ↓ 7 ↑



0 0



Turbo

v3.27.0



LF



{ }

Markdown



No account



Go Live



Prettier





sn

david@coyote:~/dev/sn
~/dev/sn

```
david ~/dev/sn on ȳ main $ git status 2.0
On branch main
Your branch is ahead of 'origin/main' by 7 commits.
(use "git push" to publish your local commits)
```

```
nothing to commit, working tree clean
```

```
david ~/dev/sn on ȳ main $
```

david@coyote:~/dev/sn
~/dev/sn

```
david ~/dev/sn on ȳ main $ git log --oneline -4
75604e5 (HEAD -> main) Merge branch 'feat/router'
6ba934f Add description to README.md
fc2dc41 (feat/router) Create the main API router
b57e369 Merge branch 'chore/prettier'
```

```
david ~/dev/sn on ȳ main $
```



main 0 7↑



0 0



Turbo v3.27.0



LF



Markdown



No account



Go Live



Prettier



Cancelando um merge

- Caso não queira continuar a resolução, o comando abaixo tenta restaurar o repositório ao estado anterior ao merge

```
git merge --abort
```

- Com o merge abortado, pode-se tentar fazer novos commits nas branches para desfazer as causas dos conflitos



Exercício (parte 1)

- Reproduza o cenário apresentado nos slides anteriores, provocando e resolvendo um conflito no arquivo README.md:
 - A partir da branch **main**, crie a branch **docs/update**, adicione a seção **Descrição** com uma única linha no arquivo README.md, e faça um commit
 - Volte para a main e, antes de fazer o merge da **docs/update** com a branch **main**, crie a branch **feat/router**
 - Na branch **feat/router**:
 - Crie o arquivo **src/router/router.ts** com as rotas da API, seguindo os slides do SmartNotes até a seção **Arquivo Separado de Rotas**
 - Adicione a seção Descrição no **README.md**, com um conteúdo diferente daquele escrito em **docs/update**
 - Faça um commit com essas alterações



Exercício (parte 2)

- Continuação...
 - Volte para a branch main e faça o merge de **docs/update**
 - Em seguida, ainda na main, faça o merge de **feat/router**
 - O Git deverá detectar um conflito no arquivo README.md
 - Resolva o conflito utilizando a interface do **Visual Studio Code**, escolhendo qual alteração manter ou combinando as duas versões. Depois, conclua o merge.
- Ao final, use git **log --oneline --graph** para verificar o histórico das branches e dos merges



Repositórios Remotos

- Um **repositório remoto** é um repositório hospedado em algum servidor, como GitHub, GitLab ou servidor privado
 - Conforme já dito anteriormente, o Git é **distribuído**: cada clone, remoto ou local, possui seu próprio histórico completo
- **origin** é o nome convencional dado ao repositório remoto principal, e é basicamente uma URL

```
david@coyote:~/dev/sn
david ~/dev/sn on ↳ main $ git remote -v
origin  https://github.com/dbfernandes/smartnotes.git (fetch)
origin  https://github.com/dbfernandes/smartnotes.git (push)
david ~/dev/sn on ↳ main $
```



Comandos para Remotes

git remote

Lista os repositórios remotos configurados.

git remote -v

Lista os repositórios remotos de forma detalhada (v = verbose), mostrando também as URLs usadas para fetch e push.

git remote show origin

Exibe informações detalhadas sobre o remoto origin, incluindo branches rastreadas e configuração de push/pull.

git remote add origin URL

Adiciona um novo repositório remoto chamado origin, associado à URL informada.

git remote set-url origin URL

Altera a URL associada ao repositório remoto origin. Pode ser usado com a opção --push, para configurar apenas a URL dos pushes.

git remote remove origin

Remove o repositório remoto chamado origin da configuração local.



Comandos para Remotes

git remote

Lista os repositórios remotos configurados.

git remote -v

Lista os repositórios remotos de forma detalhada (v = verbose), mostrando também as URLs usadas para fetch e push.

```

david@coyote:~/dev/sn
david ~/dev/sn on  main $ git remote show origin
* remote origin
Fetch URL: https://github.com/dbfernandes/smartnotes.git
Push URL: https://github.com/dbfernandes/smartnotes.git
HEAD branch: main
Remote branch:
  main tracked
Local branch configured for 'git pull':
  main merges with remote main
Local ref configured for 'git push':
  main pushes to main (up to date)
david ~/dev/sn on  main $
```



Git Clone

- O comando **git clone** cria uma cópia local completa de um repositório remoto existente
 - O comando configura automaticamente o remoto **origin**
 - Também cria a branch local principal e configura o upstream da branch remota

```

david@coyote:~
lucy ~ $ git clone https://github.com/dbfernandes/smartnotes
Cloning into 'smartnotes'...
remote: Enumerating objects: 62, done.
remote: Counting objects: 100% (62/62), done.
remote: Compressing objects: 100% (31/31), done.
remote: Total 62 (delta 23), reused 62 (delta 23), pack-reused
Receiving objects: 100% (62/62), 29.35 KiB | 29.35 MiB/s, done
Resolving deltas: 100% (23/23), done.
lucy ~ $
```



Git Clone

- Após clonar um repositório, o Git cria automaticamente referências locais como **main** e **origin/main**, onde:
 - **main** é a branch local na qual o usuário trabalha
 - **origin/main** representa o último estado conhecido da branch main no repositório remoto origin
- **origin/main** não é consultada continuamente pela internet; ela é atualizada por comandos como **git fetch** e **git pull**

```
david@coyote:~/smartnotes
~/smartnotes

lucy ~/smartnotes on ʘ main $ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
lucy ~/smartnotes on ʘ main $
```



Exercício

- Faça um clone do seu repositório em um diretório diferente do que você tem usado
- Nesse diretório, crie uma branch **feat/helmet**, instale e configure o **Helmet** nessa branch, e então crie um commit
 - Para isso, siga as instruções dos slides sobre o **SmartNotes**, até a seção **Segurança da API**
 - Não envie essa branch para o servidor remoto, ainda



HELMET



Primeiro push de uma branch

- Usamos o comando **git push** para enviar os commits de uma branch local para uma branch remota no repositório origin
 - Exemplo: **git push -u origin feat/helmet**
- No primeiro push de uma branch (feat/helmet), deve-se usar a opção -u para criar o **upstream** (origin/feat/helmet)
 - Upstream é uma referência local à uma branch remota, usada como referência padrão para operações como git push e git pull
 - No nosso exemplo:
 - branch local: feat/helmet
 - upstream configurado: origin/feat/helmet
 - branch real no servidor remoto: feat/helmet
- Depois disso, normalmente basta usar **git push** e **git pull** sem informar origin e o nome da branch



Primeiro push de uma branch

- Usamos o comando **git push** para enviar os commits de uma

```
david@coyote:~/smartnotes
lucy ~/smartnotes on ̣ feat/helmet $ git push -u origin feat/helmet
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 16 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 791 bytes | 791.00 KiB/s, done.
Total 6 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
remote:
remote: Create a pull request for 'feat/helmet' on GitHub by visiting:
remote:   https://github.com/dbfernandes/smartnotes/pull/new/feat/helmet
remote:
To https://github.com/dbfernandes/smartnotes
 * [new branch] feat/helmet -> feat/helmet
branch 'feat/helmet' set up to track 'origin/feat/helmet'.
lucy ~/smartnotes on ̣ feat/helmet $
```

- Depois disso, normalmente basta usar **git push** e **git pull** sem informar origin e o nome da branch



dbfernandes/smarnotes

github.com/dbfernandes/smarnotes

dbfernandes

smartnotes

<> Code

Issues

Pull requests

Agents

Actions

Projects

More

smartnotes

Private

Watch 0

Fork 0

Star 0

feat/helmet

had recent pushes 1 minute ago

Compare & pull request

main

Go to file

+

<> Code

dbfernandes

Merge branch 'feat/router'

f153b94 · 2 days ago

scripts	First commit	4 days ago
src	Create the main API router	3 days ago
.env.example	Create .env.example	4 days ago
.git-config	First commit	4 days ago
.gitignore	Create .gitignore	4 days ago
.prettierrc	Configure Prettier	3 days ago
README.md	Create the main API router	3 days ago
eslint.config.js	Configure ESLint	4 days ago
package-lock.json	Configure Prettier	3 days ago

About

No description, website, or topics provided.

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Contributors 1

lucy
Enume
Count
Delta
Compr
Writi
Total
remot
remot
remot
remot
To ht
* [n
branc
lucy

na

a
a
da
pull
:
helmet

Comparing main...feat/h...x

github.com/dbfernandes/smartnotes/compare/feat/helmet?expand=1

dbfernandes / smartnotes

Search

+

🔄

🔗

📄

📧

👤

<> Code

Issues

Pull requests

Agents

Actions

Projects

More

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#).
[Learn more about diff comparisons here](#).

🔗

base: main

←

compare: feat/helmet

✓ **Able to merge.** These branches can be automatically merged.



Add a title *

Install and configure helmet

🔒

Add a description

Write

Preview

H

B

I

≡

<>

🔗

≡

≡

...

Add your description here...

📄 Markdown is supported

📎 Paste, drop, or click to add files

Create pull request

⌵

Reviewers

No reviews

⚙️

Assignees

No one—[assign yourself](#)

⚙️

Labels

None yet

⚙️

Projects

None yet

⚙️

Milestone

No milestone

⚙️

Development

Remember, contributions to this repository should follow our [GitHub Community Guidelines](#).

lucy
Enum
Count
Delta
Compr
Writi
Total
remot
remot
remot
remot
To ht
* [n
branc
lucy

na

na

na

na

na

na

na

na

na

na

na

na

na

na

na

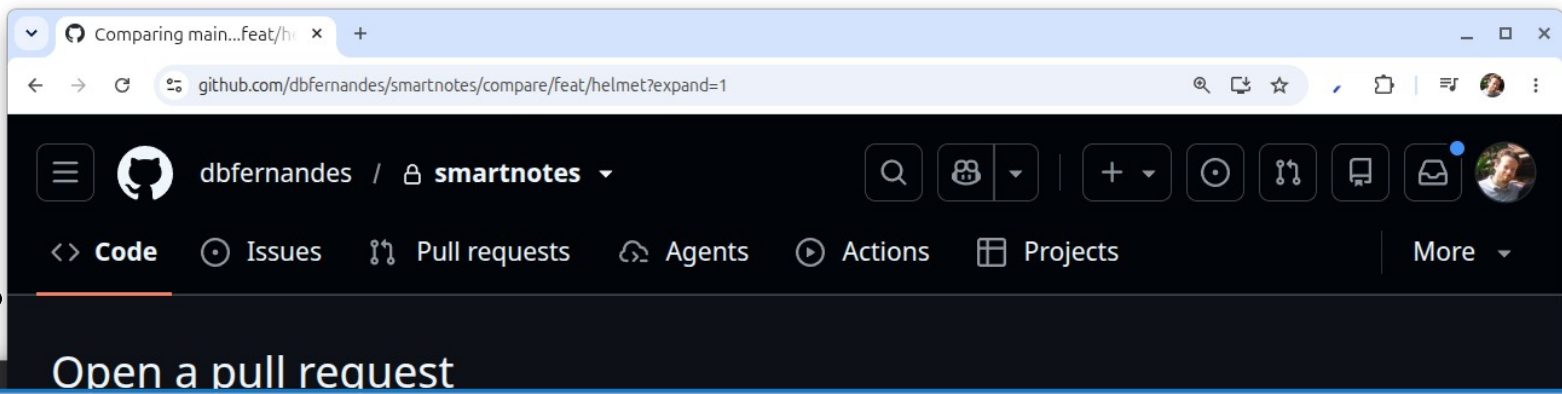
na

na

na

na

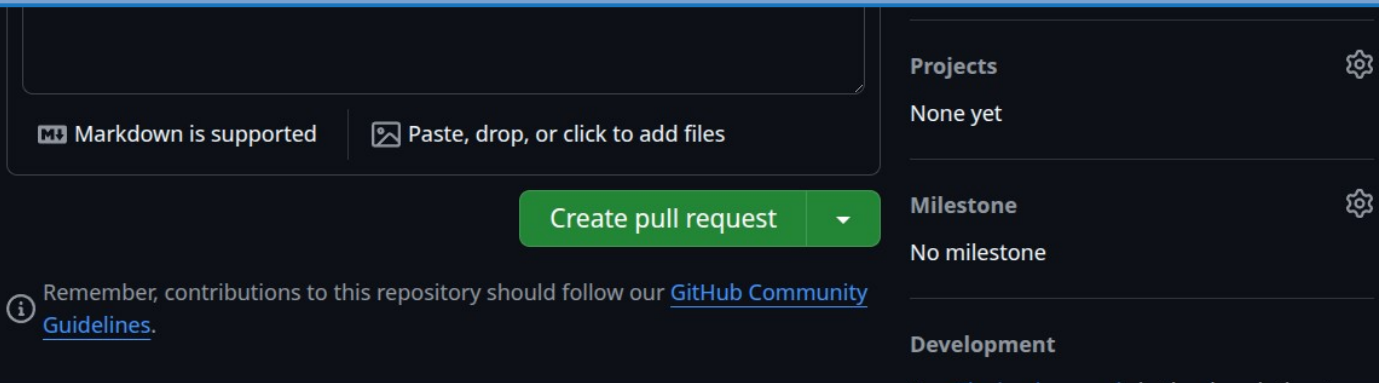
na

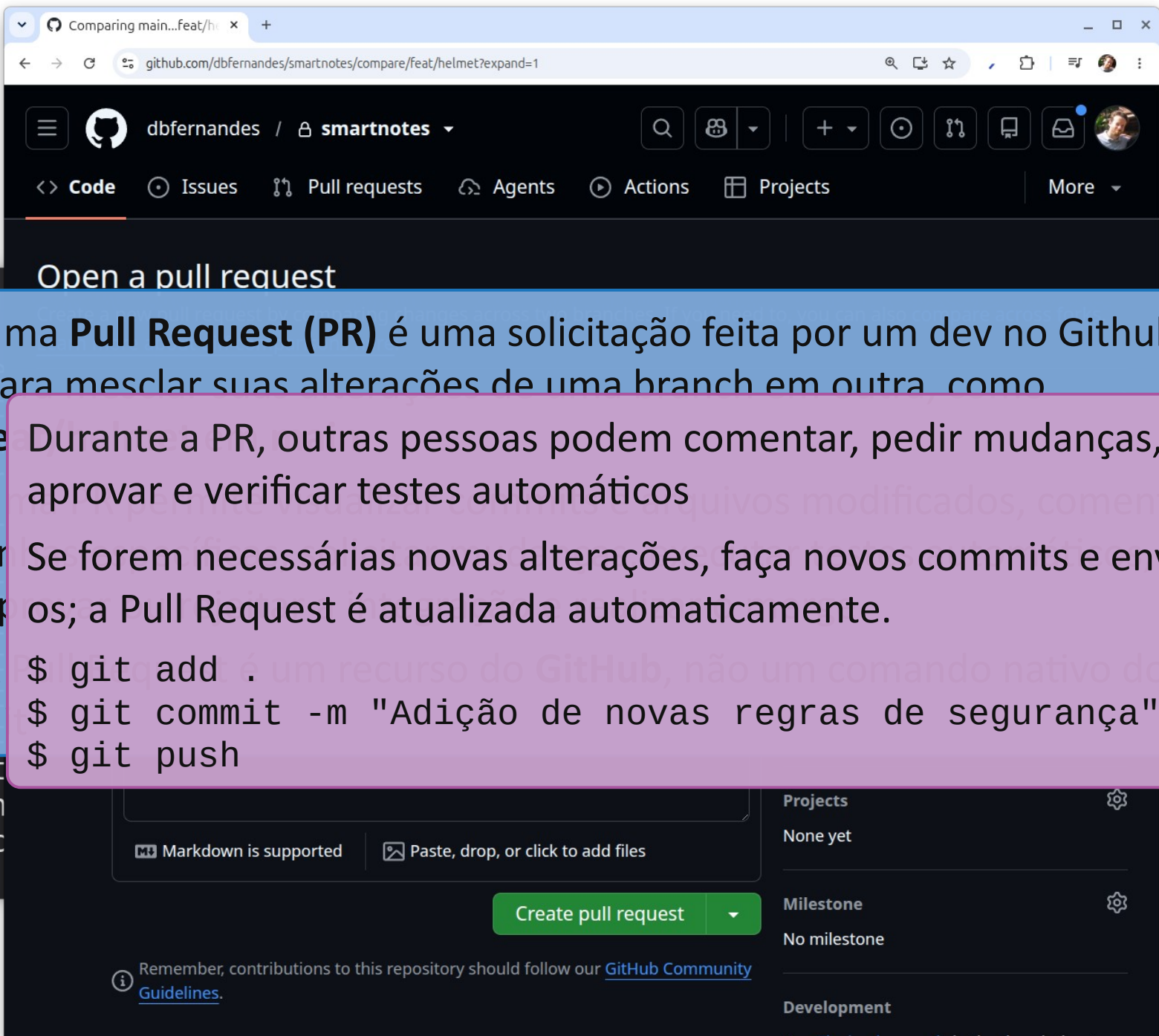


Uma **Pull Request (PR)** é uma solicitação feita por um dev no Github para mesclar suas alterações de uma branch em outra, como **feat/helmet** em **main**

Uma PR permite visualizar commits e arquivos modificados, comentar linhas específicas, solicitar mudanças, executar testes automáticos, aprovar ou rejeitar a integração e realizar o merge

A Pull Request é um recurso do **Git****Hub**, não um comando nativo do **Git**





Comparing main...feat/h... x +

github.com/dbfernandes/smartnotes/compare/feat/helmet?expand=1

dbfernandes / smartnotes

Code Issues Pull requests Agents Actions Projects More

Open a pull request

Uma **Pull Request (PR)** é uma solicitação feita por um dev no Github para mesclar suas alterações de uma branch em outra, como

Durante a PR, outras pessoas podem comentar, pedir mudanças, aprovar e verificar testes automáticos

Se forem necessárias novas alterações, faça novos commits e envie-os; a Pull Request é atualizada automaticamente.

```
$ git add .  
$ git commit -m "Adição de novas regras de segurança"  
$ git push
```

Markdown is supported Paste, drop, or click to add files

Create pull request

Remember, contributions to this repository should follow our [GitHub Community Guidelines](#).

Projects
None yet

Milestone
No milestone

Development



Install and configure helm

github.com/dbfernandes/smartnotes/pull/1

dbfernandes / smartnotes

🔍

🔒

+

🔄

🔗

📁

📧

<> Code

🕒 Issues

🔗 Pull requests 1

🚧 Agents

🔄 Actions

📁 Projects

More ▾

🟢 Ready to merge

Code ▾

Install and configure helmet #1

🔗 Open

dbfernandes wants to merge 1 commit into `main` from `feat/helmet`

💬 Conversation 0

🔗 Commits 1

📄 Checks 0

📄 Files changed 3

+18 -2

dbfernandes commented now

No description provided.

👍

🔗 Install and configure helmet efaf40c

🟢 **No conflicts with base branch**
Merging can be performed automatically.

Merge pull request ▾

You can also merge this with the command line.
[View command line instructions.](#)

Reviewers

No reviews

Still in progress? [Convert to draft](#)

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet

Milestone

No milestone

lucy
Enum
Count
Delta
Compr
Writi
Total
remot
remot
remot
remot
To ht
* [n
branc
lucy

na

a

la

pull

]:

helmet

A red GitHub logo, which is a stylized octocat (a cat with eight legs) inside a red diamond shape.

Install and configure helm

github.com/dbfernandes/smartnotes/pull/1

Open

Install and configure helm #1

dbfernandes wants to merge 1 commit into main from feat/helmet

Commit message

Merge pull request #1 from dbfernandes/feat/helmet

Extended description

Install and configure helm

Commit email

david@icomp.ufam.edu.br

Confirm merge

Cancel

Still in progress? [Convert to draft](#)

Add a comment

Write

Preview

H

B

I

≡

<>

↗

≡

≡

⋮

Add your comment here...

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet

Notifications

Customize

Unsubscribe

You're receiving notifications because you authored the thread.

1 participant

Lock conversation

Archive pull request

lucy
Enum
Count
Delta
Compr
Writi
Total
remot
remot
remot
remot
To ht
* [n
branc
lucy

na
a
pull
:
helmet

A red octagonal logo with a white branching diagram inside, representing the GitHub logo.

Install and configure helm x +

github.com/dbfernandes/smartnotes/pull/1

dbfernandes / smartnotes

Code Issues Pull requests 1 Agents Actions Projects More

Code

Install and configure helmet #1

Merged dbfernandes merged 1 commit into main from feat/helmet now

Conversation 0 Commits 1 Checks 0 Files changed 3 +18 -2

dbfernandes commented 2 minutes ago

No description provided.

Install and configure helmet efaf40c

dbfernandes merged commit 2a89d03 into main now

Pull request successfully merged and closed

You're all set — the `feat/helmet` branch can be safely deleted.

Reviewers

No reviews

Still in progress? [Convert to draft](#)

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet

Milestone

No milestone



Com a Pull Request mesclada à **origin/main**, podemos atualizar o repositório **main** local

```
lucy ~/smartnotes on ❷ feat/helmet $ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
lucy ~/smartnotes on ❷ main $ git pull
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (1/1), 917 bytes | 917.00 KiB/s, done.
From https://github.com/dbfernandes/smartnotes
   f153b94..2a89d03  main       -> origin/main
Updating f153b94..2a89d03
Fast-forward
 package-lock.json | 15 ++++++
 package.json      |  3 ++-
 src/index.ts      |  2 ++
 3 files changed, 18 insertions(+), 2 deletions(-)
lucy ~/smartnotes on ❷ main $
```

Pull request successfully merged and closed

You're all set — the `feat/helmet` branch can be safely deleted.

Milestone

No milestone

Exercício

- No repositório local criado no exercício anterior, execute o comando **git push -u origin feat/helmet**
 - Como vimos, esse comando irá salvar **feat/helmet** no servidor remoto e configurar o upstream **origin/feat/helmet**
- Seguindo os passos dos slides anteriores, crie um **pull request** e faça o merge de **feat/helmet** com a branch **main** do Github
- Depois, ainda no repositório local criado no exercício anterior, use o **git pull** para atualizar a branch main local com a branch main do Github

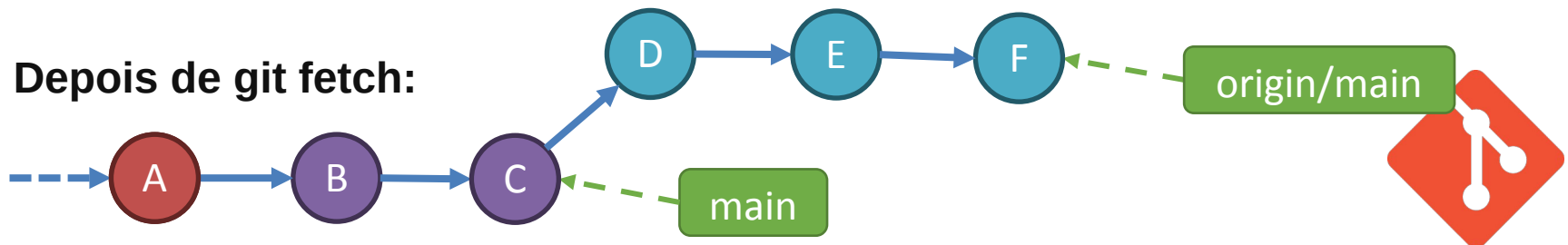
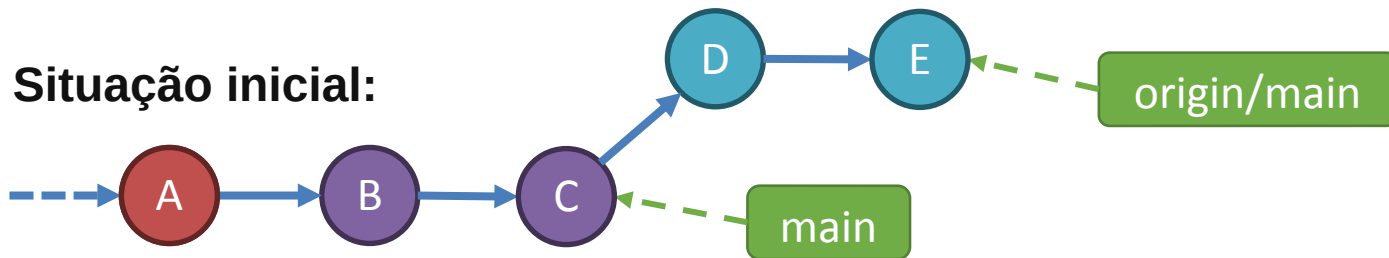


HELMET



Git Pull e Git Fetch

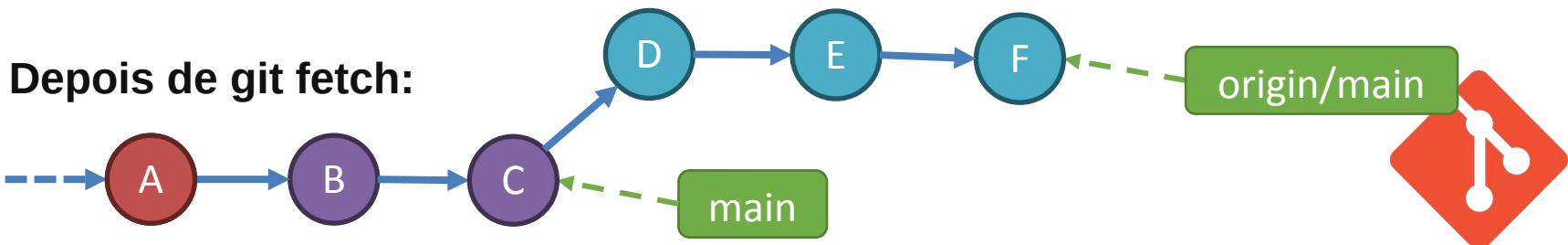
- Nos últimos slides, a dev Lucy usou o **git pull** para atualizar a branch main local com a branch main do Github
 - O **git pull** baixa as alterações do repositório remoto, atualiza o upstream e faz o merge desse upstream com a branch local
- Além do git pull, também podemos usar o **git fetch**, que atualiza o upstream mas não faz o merge com a branch local



Git Pull e Git Fetch

- Nos últimos slides, a dev Lucy usou o **git pull** para atualizar a branch main local com a branch main do Github

```
david@coyote:~/dev/sn
david ~/dev/sn on main$ git fetch
remote: Enumerating objects: 12, done.
remote: Counting objects: 100% (12/12), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 7 (delta 5), reused 5 (delta 4), pack-reused 0 (from 0)
Unpacking objects: 100% (7/7), 1.54 KiB | 1.54 MiB/s, done.
From https://github.com/dbfernandes/smarnotes
 f153b94..2a89d03  main          -> origin/main
david ~/dev/sn on main$
```



david@coyote:~/dev/sn
~/dev/sn

```
david ~/dev/sn on ̣ main $ git log --oneline origin/main
2a89d03 (origin/main, origin/HEAD) Merge pull request #1 from
dbfernandes/feat/helmet
efaf40c Install and configure helmet
f153b94 (HEAD -> main, docs/update) Merge branch 'feat/router'
6ba934f Add description to README.md
fc2dc41 (feat/router) Create the main API router
b57e369 Merge branch 'chore/prettier'
40ad73f Update README
9af3892 (chore/prettier) Configure Prettier
711d082 Install Prettier
5a08614 (chore/eslint) Configure ESLint
david 7b42882 Install ESLint
b2d89ea Add SmartNotes version to README.md
51da999 Create .env.example
d128176 Create .gitignore
9f30563 Validate .env file
703dea3 Create README.md file
7d2a909 First commit
david ~/dev/sn on ̣ main $
```

main

origin/main

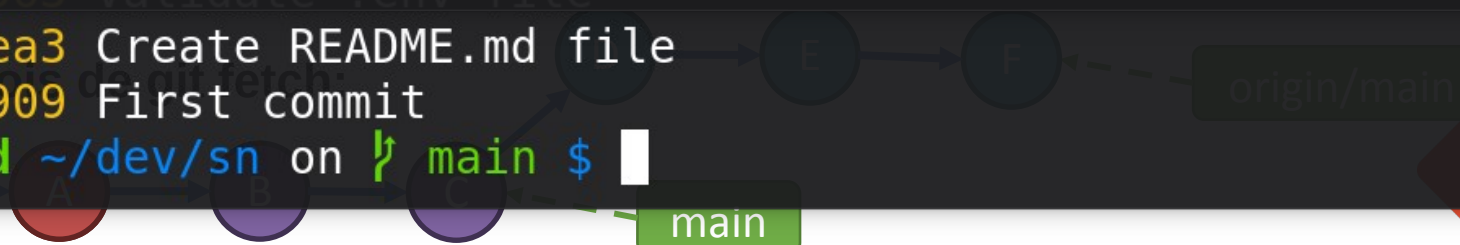
```
david@coyote:~/dev/sn
~/dev/sn

david ~/dev/sn on  main $ git log --oneline origin/main
2a89d03 (origin/main, origin/HEAD) Merge pull request #1 from
dbfernandes/feat/helmet
efaf40c Install and configure helmet
f153b94 (HEAD -> main, docs/update) Merge branch 'feat/router'
6ba934f Add description to README.md
fc2dc41 (feat/router) Create the main API router

david@coyote:~/dev/sn
~/dev/sn

david ~/dev/sn on  main $ git merge origin/main
Updating f153b94..2a89d03
Fast-forward
 package-lock.json | 15 ++++++
 package.json      |  3 ++-
 src/index.ts      |  2 ++
 3 files changed, 18 insertions(+), 2 deletions(-)
david ~/dev/sn on  main $

703dea3 Create README.md file
7d2a909 First commit
david ~/dev/sn on  main $
```



A diagram illustrating the Git commit history. It shows a sequence of commits represented by circles labeled A, B, C, D, E, and F. Commit A is the root. Commit B is a child of A. Commit C is a child of B. Commit D is a child of C. Commit E is a child of D. Commit F is a child of E. A box labeled 'main' has an arrow pointing to commit C. A box labeled 'origin/main' has an arrow pointing to commit F. The diagram shows a linear history of commits on the 'main' branch.

Exercício

- No repositório local original, use o **git fetch** para atualizar o **origin/main** com as alterações da main do Github
- Caso não esteja na branch main, use o comando **git switch** para mudar o HEAD para essa branch
- Como o comando **git fetch** não faz o merge entre **origin/main** e o repositório **main** local, use o comando **git merge origin/main** para atualizar a branch main com o conteúdo do repositório remoto

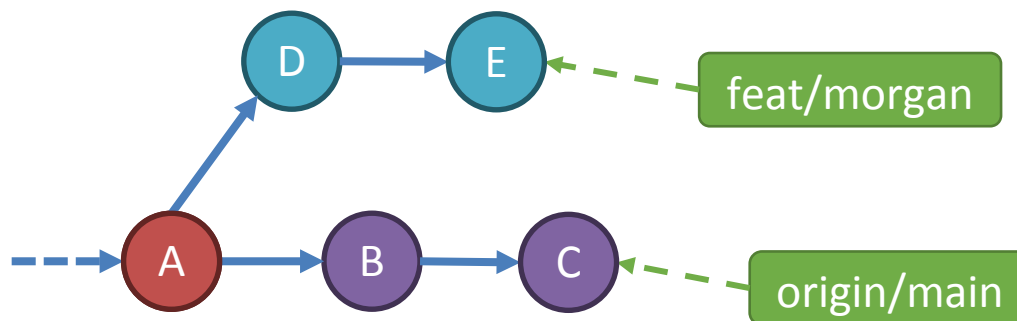


HELMET



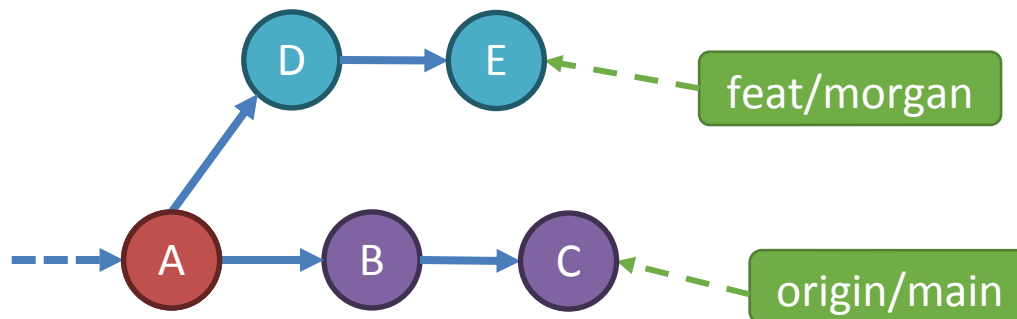
Atualização de Branches

- Durante o desenvolvimento, outros desenvolvedores podem atualizar a branch **main** do Github com:
 - correções de bugs
 - novas funcionalidades
 - refatorações
 - mudanças em arquivos usados pela feature
- No exemplo abaixo, a branch **feat/morgan** foi criada a partir do commit A, mas a **origin/main** já recebeu os commits B e C



Atualização de Branches

- Quanto mais tempo a feature demorar para ser concluída, mais conflitos podem se acumular ao integrá-la com a main
- Por isso, uma prática comum é trazer periodicamente as mudanças da main remota para a branch da nova feature
 - Atualizar a feature durante o desenvolvimento ajuda a descobrir incompatibilidades antes da integração final

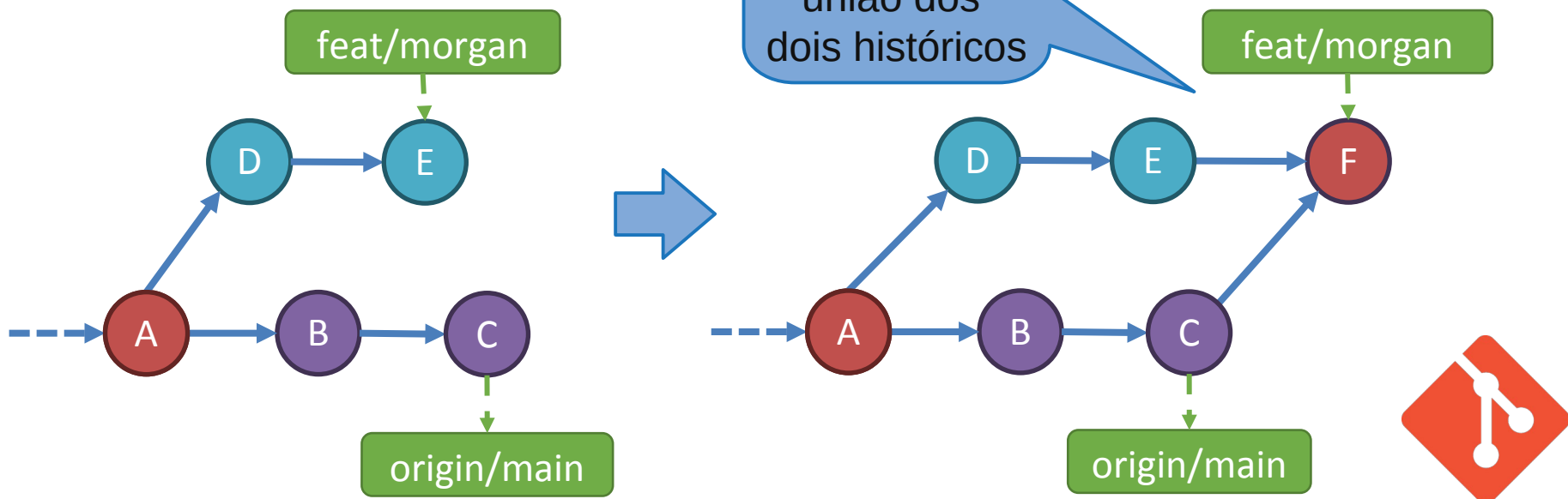


Atualização de Branches

- Para trazer as mudanças mais recentes da main do repositório remoto para a branch local da feature:

```
$ git switch feat/morgan  
$ git fetch origin  
$ git merge origin/main
```

- Resultado:



Atualização de Branches

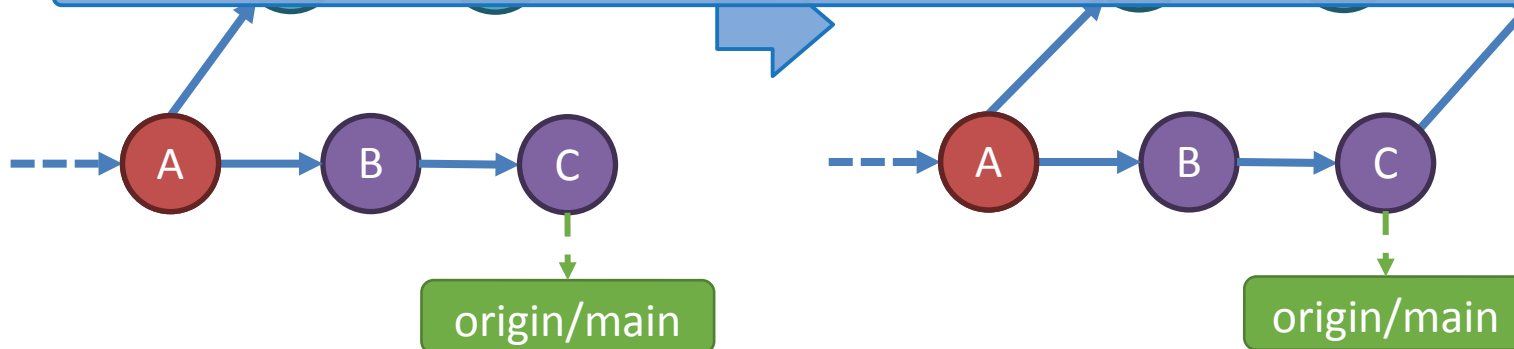
- Para trazer as mudanças mais recentes da main do repositório remoto para a branch local da feature:

```
$ git switch feat/morgan  
$ git fetch origin  
$ git merge origin/main
```

- Resultado:

O commit F
representa a
união dos

É preferível usar **origin/main** após executar **git fetch**, pois isso garante que a integração será feita com o estado mais recente da main no repositório remoto

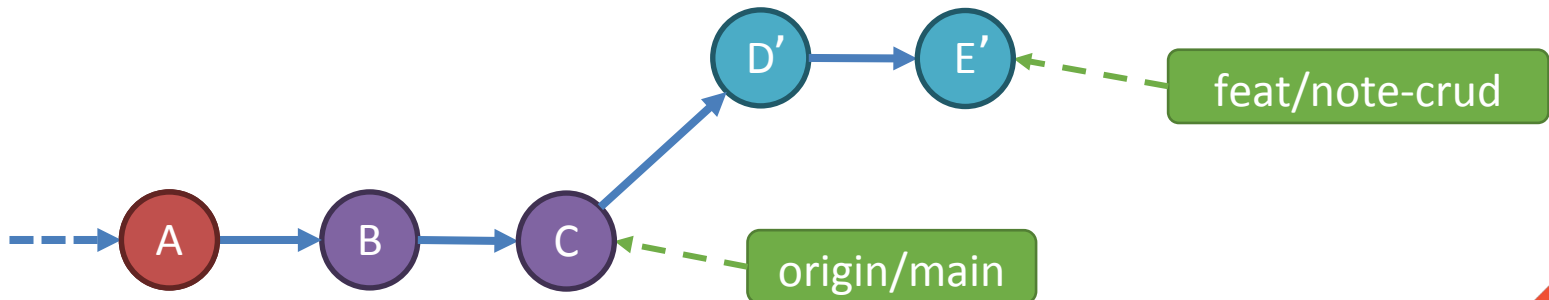


Atualização com Rebase

- Outra possibilidade é reposicionar os commits da feature sobre a versão mais recente da main, usando o **rebase**:

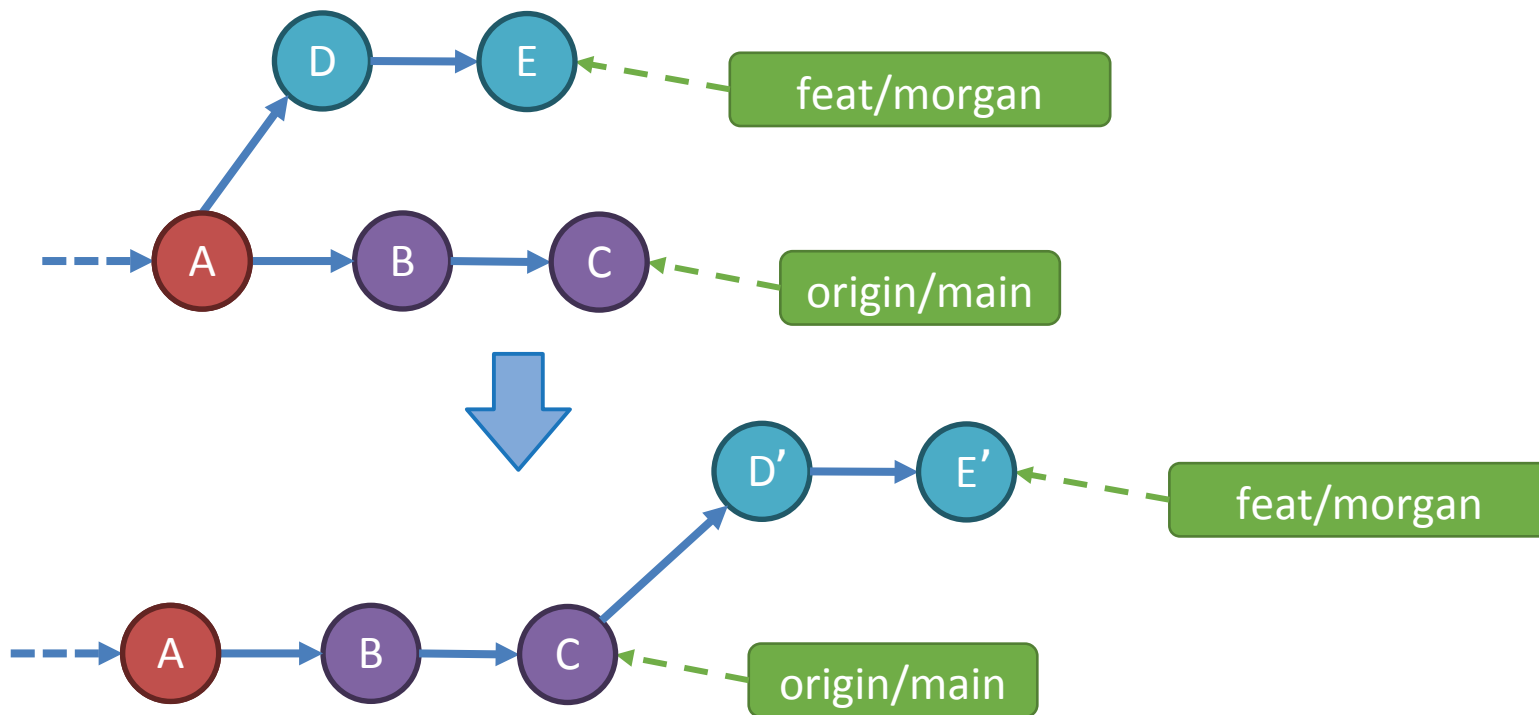
```
$ git switch feat/morgan  
$ git fetch origin  
$ git rebase origin/main
```

- O rebase reaplica os commits da feature sobre o commit mais recente de **origin/main**, criando novos commits e produzindo um histórico mais linear que o merge



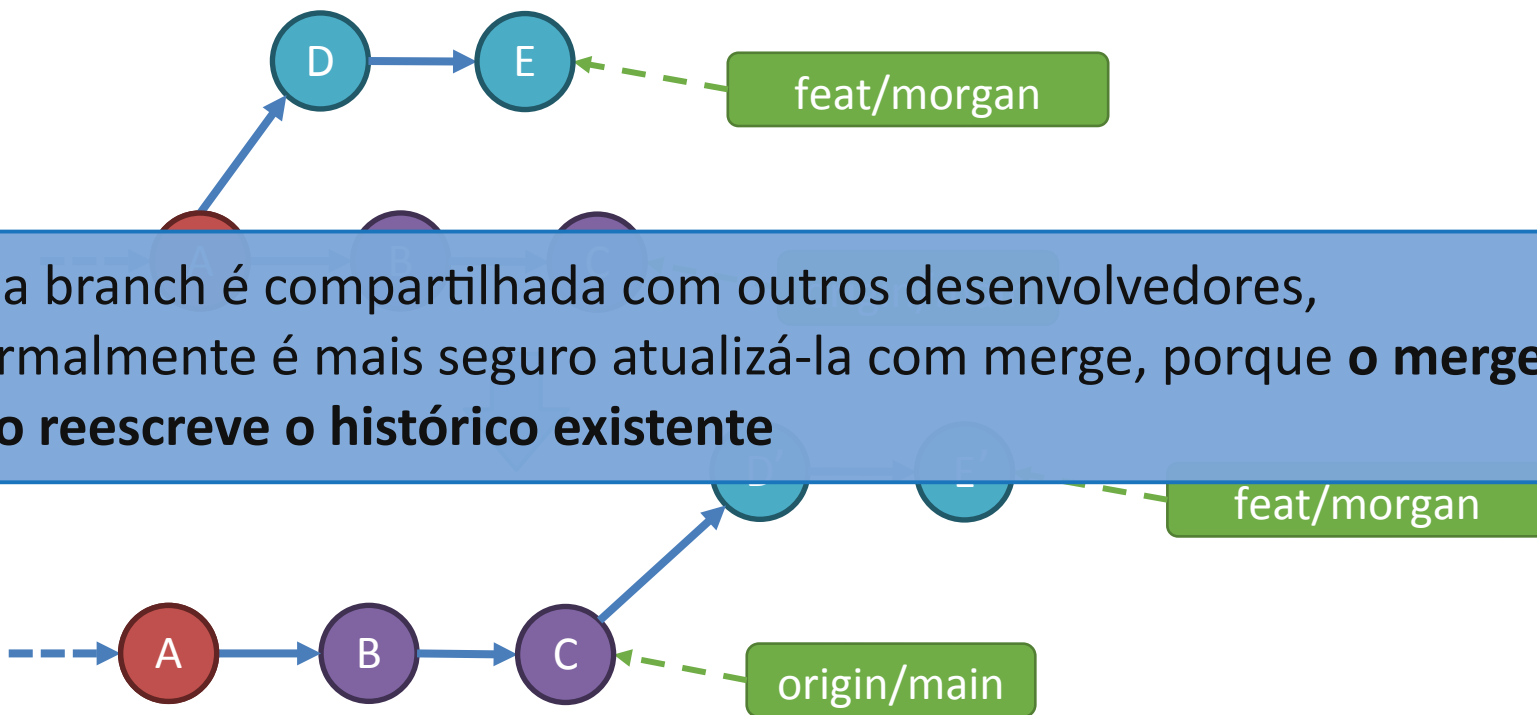
Atualização com Rebase

- Note que os commits D e F são recriados, e por isso o **rebase** não é recomendado quando os commits que já foram compartilhados com outros desenvolvedores



Atualização com Rebase

- Note que os commits D e F são recriados, e por isso o **rebase** não é recomendado quando os commits que já foram compartilhados com outros desenvolvedores



Merge ou Rebase?

- Prefira **merge** quando:
 - quiser preservar o histórico completo
 - estiver integrando uma branch compartilhada
 - quiser evitar reescrever commits
- Prefira **rebase** quando:
 - quiser produzir um histórico linear
 - os commits ainda não tiverem sido compartilhados
 - quiser organizar o histórico antes de abrir uma Pull Request



Install and configura mor x +

github.com/dbfernandes/smartnotes/pull/2

Ask Google

Install and configura morgan #2

Open dbfernandes wants to merge 1 commit into **main** from **feat/morgan**

Conversation 0 Commits 1 Checks 0 Files changed 4 +85 -3

dbfernandes commented 3 minutes ago

No description provided.

✓ Create a merge commit
All commits from this branch will be added to the base branch via a merge commit.

Squash and merge
The 1 commit from this branch will be added to the base branch.

Rebase and merge
The 1 commit from this branch will be rebased and added to the base branch.

✓ No conflicts with base
Merging can be automatic.

Merge pull request

You can also merge this with the command line. [View command line instructions.](#)

Still in progress? [Convert to draft](#)

Reviewers
No reviews
Still in progress

Assignees
No one assigned

Projects
None yet

Milestone
No milestone

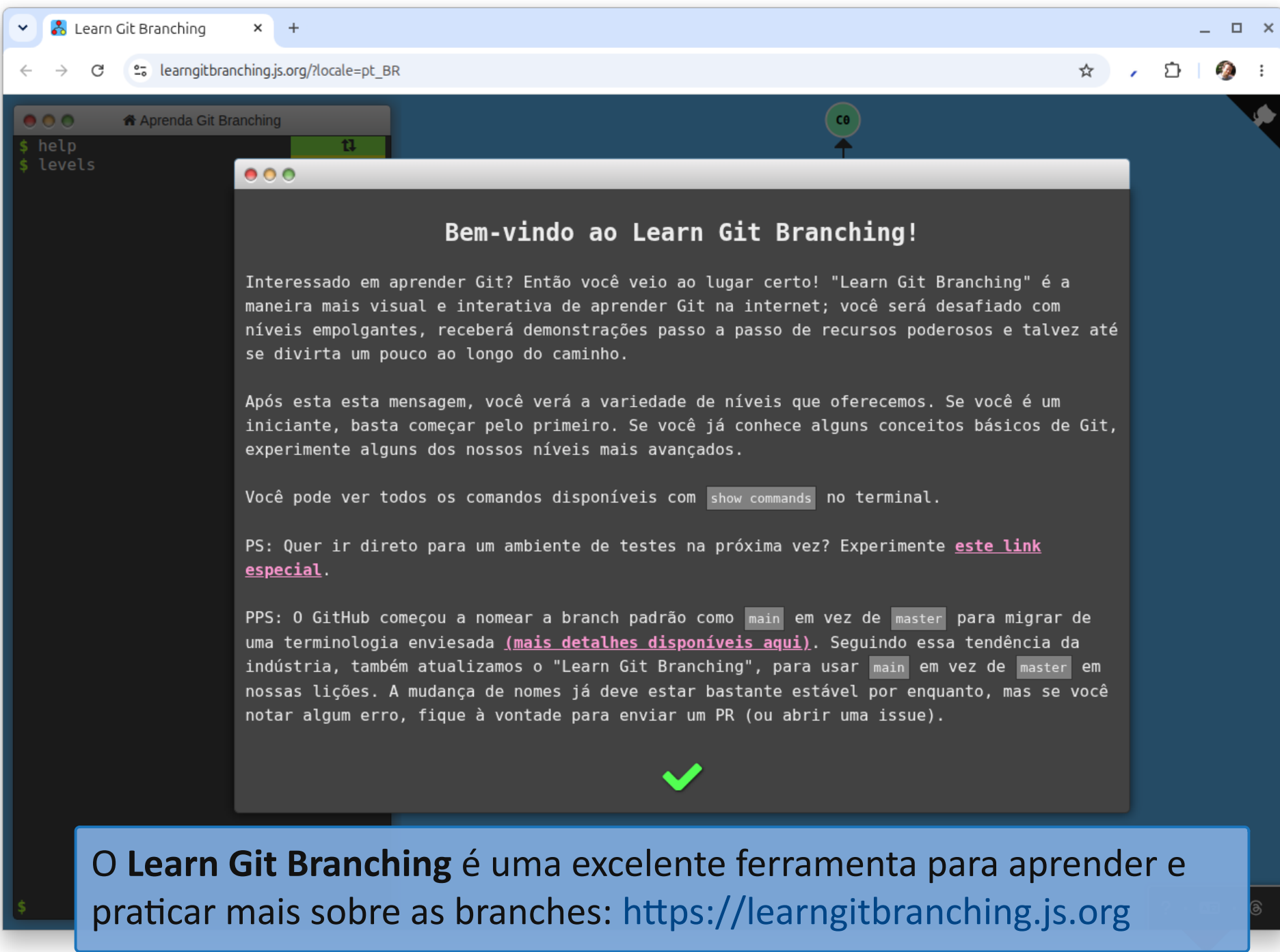
Development

Em um Pull Request, também podemos fazer o **rebase** dos commits antes de integrá-los à branch main

Exercício

- Crie uma branch **feat/morgan** em seu projeto e, nessa branch, instale e configure o Morgan
 - Para isso, siga as instruções dos slides sobre o SmartNotes, até a seção **Registro das Requisições**
- Após concluir a configuração, envie a branch **feat/morgan** para o GitHub, configurando o upstream
- Crie um **Pull Request** para integrar feat/morgan à branch main e depois escolha a opção **Rebase and merge**
 - Os commits de feat/morgan serão reaplicados sobre a ponta atual da main, sem a criação de um merge commit
- Após a integração, atualize sua branch main local com as alterações do repositório remoto





The image shows a web browser window with the URL `learngitbranching.js.org/?locale=pt_BR`. The page title is "Learn Git Branching". In the background, a terminal window is visible with the text "Aprenda Git Branching" and commands `$ help` and `$ levels`. A large, semi-transparent modal window is centered on the screen, displaying a welcome message in Portuguese. The modal has a title "Bem-vindo ao Learn Git Branching!" and contains several paragraphs of text. At the bottom of the modal is a large green checkmark icon. A blue banner at the bottom of the image contains text in Portuguese and a URL.

Bem-vindo ao Learn Git Branching!

Interessado em aprender Git? Então você veio ao lugar certo! "Learn Git Branching" é a maneira mais visual e interativa de aprender Git na internet; você será desafiado com níveis empolgantes, receberá demonstrações passo a passo de recursos poderosos e talvez até se divirta um pouco ao longo do caminho.

Após esta esta mensagem, você verá a variedade de níveis que oferecemos. Se você é um iniciante, basta começar pelo primeiro. Se você já conhece alguns conceitos básicos de Git, experimente alguns dos nossos níveis mais avançados.

Você pode ver todos os comandos disponíveis com `show commands` no terminal.

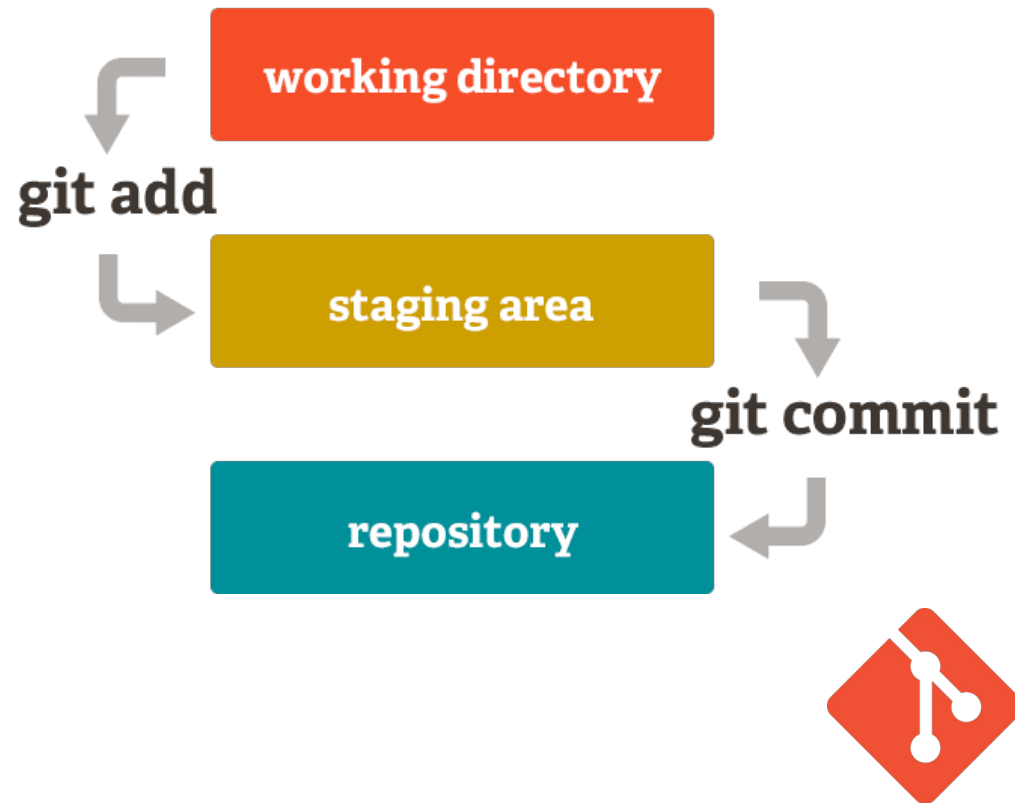
PS: Quer ir direto para um ambiente de testes na próxima vez? Experimente [este link especial](#).

PPS: O GitHub começou a nomear a branch padrão como `main` em vez de `master` para migrar de uma terminologia enviesada ([mais detalhes disponíveis aqui](#)). Seguindo essa tendência da indústria, também atualizamos o "Learn Git Branching", para usar `main` em vez de `master` em nossas lições. A mudança de nomes já deve estar bastante estável por enquanto, mas se você notar algum erro, fique à vontade para enviar um PR (ou abrir uma issue).

O **Learn Git Branching** é uma excelente ferramenta para aprender e praticar mais sobre as branches: <https://learngitbranching.js.org>

Desfazendo alterações no Git

- O Git oferece diferentes comandos para desfazer alterações, dependendo de onde a alteração está:
 - Diretório de trabalho
 - Staging area
 - Histórico de commits
 - Repositório remoto
- Principais comandos:
 - `git restore`
 - `git reset`
 - `git revert`
 - `git commit --amend`

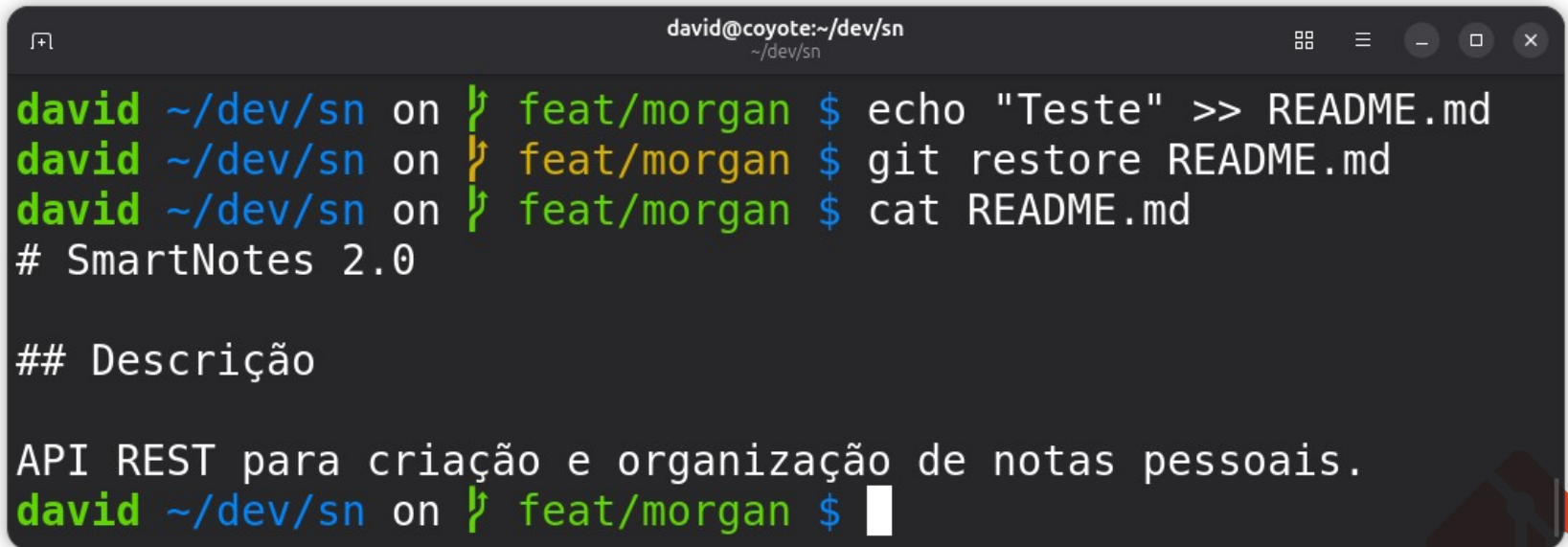


Desfazendo alterações em arquivo

- Para descartar alterações feitas no diretório de trabalho:

```
$ git restore arquivo.txt
```

- Com isso, o arquivo volta para a versão registrada na **staging area**, e as alterações feitas no arquivo são perdidas

A terminal window with a dark background and light-colored text. The window title is 'david@coyote:~/dev/sn'. The terminal shows a sequence of commands and their output. First, 'echo "Teste" >> README.md' is executed. Then, 'git restore README.md' is executed. Finally, 'cat README.md' is executed, showing the content of the file. The content includes a title '## Descrição' and a description 'API REST para criação e organização de notas pessoais.'.

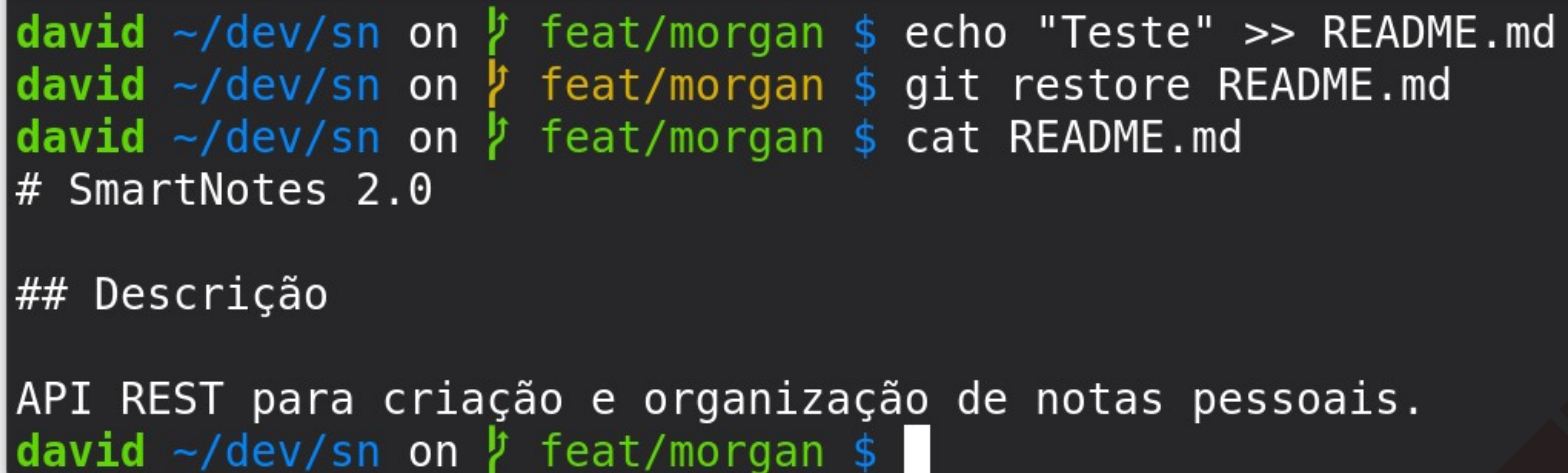
```
david@coyote:~/dev/sn  
david ~/dev/sn on feat/morgan $ echo "Teste" >> README.md  
david ~/dev/sn on feat/morgan $ git restore README.md  
david ~/dev/sn on feat/morgan $ cat README.md  
# SmartNotes 2.0  
  
## Descrição  
  
API REST para criação e organização de notas pessoais.  
david ~/dev/sn on feat/morgan $
```

Desfazendo alterações em arquivo

- Para descartar alterações feitas no diretório de trabalho:

```
$ git restore arquivo.txt
```

- Com isso, o arquivo volta para a versão registrada na **staging area**. Para restaurar **todos** os arquivos modificados, usamos **git restore .**



A terminal window with a dark background and light-colored text. The window title is "david@dev: ~/dev/sn". The terminal shows a sequence of commands and their output. The first command is "echo 'Teste' >> README.md". The second command is "git restore README.md". The third command is "cat README.md", which outputs the content of the README file. The content starts with "# SmartNotes 2.0" and "## Descrição", followed by a description of the API REST. The prompt "david ~/dev/sn on ↳ feat/morgan \$" is visible at the bottom.

```
david ~/dev/sn on ↳ feat/morgan $ echo "Teste" >> README.md
david ~/dev/sn on ↳ feat/morgan $ git restore README.md
david ~/dev/sn on ↳ feat/morgan $ cat README.md
# SmartNotes 2.0

## Descrição

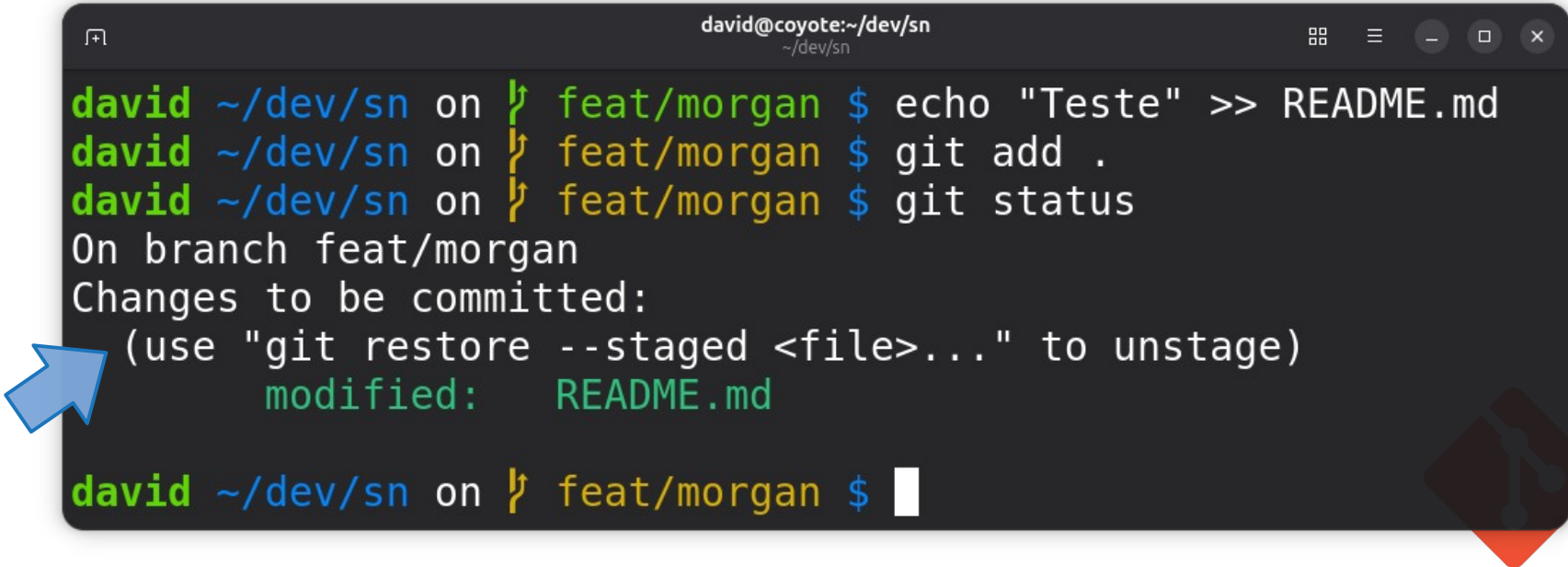
API REST para criação e organização de notas pessoais.
david ~/dev/sn on ↳ feat/morgan $
```

Removendo arquivo da staging area

- Para remover as alterações de um arquivo da staging area, mantendo suas modificações no diretório de trabalho:

```
$ git restore --staged arquivo.txt
```

- As modificações no arquivo **não são perdidas**, apenas deixam de fazer parte do próximo commit



```
david@coyote:~/dev/sn
david ~/dev/sn on ↳ feat/morgan $ echo "Teste" >> README.md
david ~/dev/sn on ↳ feat/morgan $ git add .
david ~/dev/sn on ↳ feat/morgan $ git status
On branch feat/morgan
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md
david ~/dev/sn on ↳ feat/morgan $
```

Removendo arquivo da staging area

```
david@coyote:~/dev/sn
david ~/dev/sn on  feat/morgan $ echo "Teste" >> README.md
david ~/dev/sn on  feat/morgan $ git add .
david ~/dev/sn on  feat/morgan $ git status
On branch feat/morgan
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md

david ~/dev/sn on  feat/morgan $ git restore --staged README.md
david ~/dev/sn on  feat/morgan $ git status
On branch feat/morgan
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
david ~/dev/sn on  feat/morgan $
```



Exercício

- Na branch **feat/morgan**, acrescente a string **Teste** no final do arquivo README.md e execute o **git status**
- Descarte a alteração com **git restore README.md** e confirme que o arquivo voltou à versão anterior
- Acrescente novamente a linha Teste, execute **git add README.md** e consulte o status do repositório
- Remova o arquivo da staging area com **git restore --staged README.md**, sem perder a modificação
- Confirme o resultado com **git status**. Depois, adicione novamente o arquivo à staging area e crie o commit:
 - **git commit -m "Practice git restore"**



Desfazendo o último commit

- Para os próximos slides, vamos executar o código bash abaixo, que cria 4 commits na branch **feat/morgan**

```
$ for letra in A B C D; do  
  echo "$letra" >> README.md  
  git add .  
  git commit -m "$letra"  
done
```

A terminal window with a dark background. The title bar shows 'david@coyote:~/dev/sn' and standard window controls. The prompt is 'david ~/dev/sn on ʘ feat/morgan'. The command 'git log --oneline -5' has been executed, showing five commits. The first commit is '9990fe3 (HEAD -> feat/morgan) D'. The next three are '5352dbd C', 'eed750 B', and '24b7cd4 A'. The fifth commit is 'b68ac70 (origin/feat/morgan) Install and configure morgan'. The prompt is now 'david ~/dev/sn on ʘ feat/morgan \$' with a cursor.

```
david@coyote:~/dev/sn  
david ~/dev/sn on ʘ feat/morgan $ git log --oneline -5  
9990fe3 (HEAD -> feat/morgan) D  
5352dbd C  
eed750 B  
24b7cd4 A  
b68ac70 (origin/feat/morgan) Install and configure morgan  
david ~/dev/sn on ʘ feat/morgan $
```



Desfazendo o último commit

- Para os próximos slides, vamos executar o código bash abaixo, que cria 4 commits na branch **feat/morgan**

```
$ for letra in A B C D; do  
  echo "$letra" >> README.md  
  git add .  
  git commit -m "$letra"  
done
```

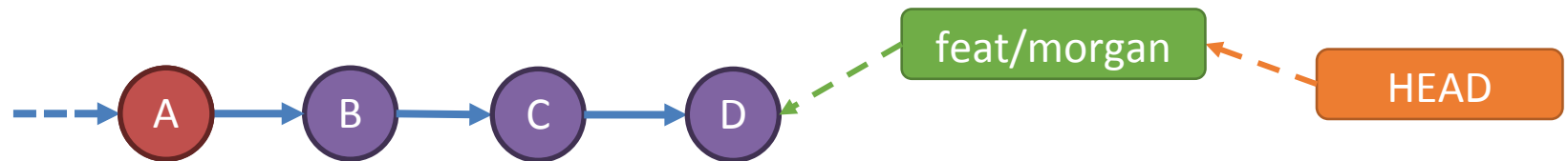


The image shows a terminal window with a dark background. The prompt is 'david@coyote:~/dev/sn'. The user has run the command 'tail -n4 README.md'. The output shows the last four commits on the 'feat/morgan' branch. The commits are listed in reverse chronological order, with the most recent at the top. The commit messages are 'A', 'B', 'C', and 'D'. The commit hashes are '2dbd', 'd750', '7cd4', and 'ac70'. The commit 'ac70' is the initial commit on the branch, with the message '(origin/feat/morgan) Install and configure morgan'.

```
david@coyote:~/dev/sn  
david@coyote:~/dev/sn  
david ~/dev/sn on ʝ feat/morgan $ tail -n4 README.md  
5 A 2dbd C  
e B d750 B  
2 C 7cd4 A  
b D ac70 (origin/feat/morgan) Install and configure morgan  
d david ~/dev/sn on ʝ feat/morgan $
```

Desfazendo o último commit

- Após o último comando, os commits da branch **feat/morgan** passam a seguir a disposição abaixo



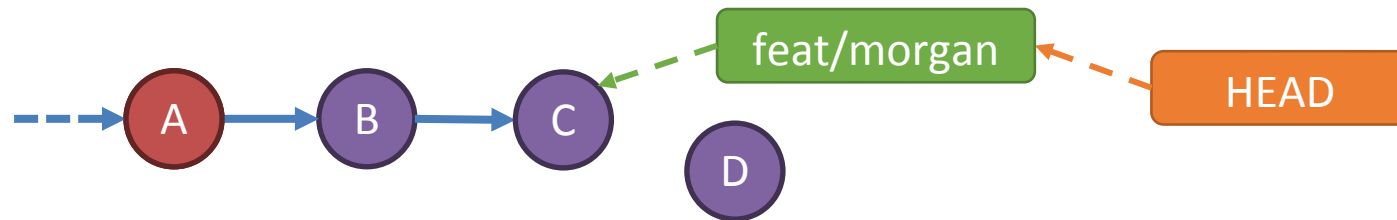
```
david@coyote:~/dev/sn
david ~/dev/sn on ̣ feat/morgan $ git log --oneline -5
9990fe3 (HEAD -> feat/morgan) D
5352dbd C
eed750 B
24b7cd4 A
b68ac70 (origin/feat/morgan) Install and configure morgan
david ~/dev/sn on ̣ feat/morgan $
```



Desfazendo o último commit

- O comando **git reset** permite mover a branch atual para um commit a partir de seu HASH

```
$ git reset 5352dbd
```

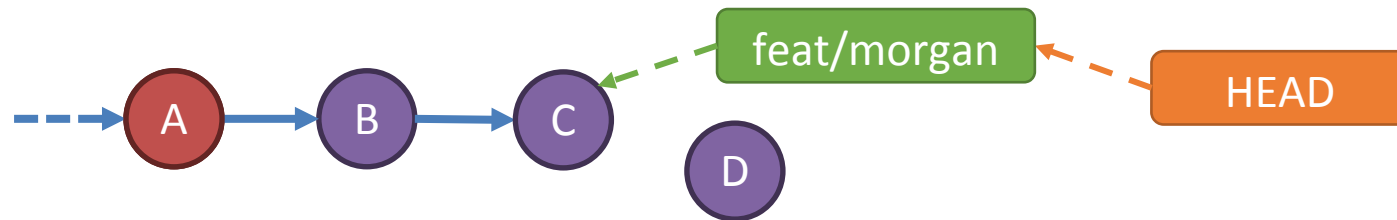


```
david@coyote:~/dev/sn  
david ~/dev/sn on  ↳ feat/morgan $ git log --oneline -4  
5352dbd (HEAD -> feat/morgan) C  
eed750 B  
24b7cd4 A  
b68ac70 (origin/feat/morgan) Install and configure morgan  
david ~/dev/sn on  ↳ feat/morgan $
```

Desfazendo o último commit

- O comando **git reset** permite mover a branch atual para um commit a partir de seu HASH

```
$ git reset 5352dbd
```



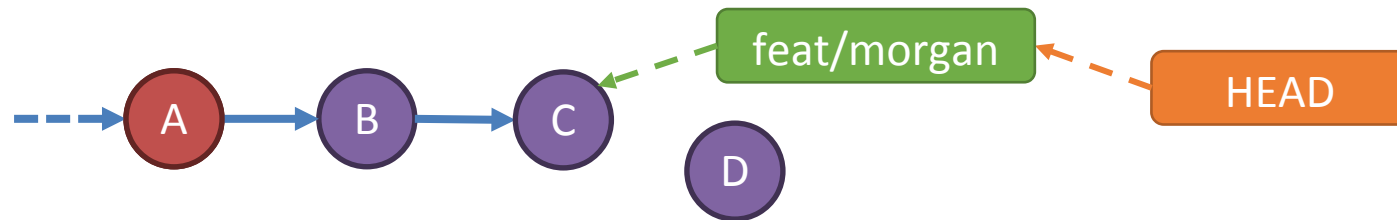
```
david@coyote:~/dev/sn  
david ~/dev/sn on ȳ feat/morgan $ git log --oneline -4  
5352dbd (HEAD -> feat/morgan) C  
eed750 B
```

Após o reset, a branch **feat/morgan** passa a apontar para **5352dbd** (C), e os commits posteriores deixam de fazer parte da branch

Desfazendo o último commit

- O comando **git reset** permite mover a branch atual para um commit a partir de seu HASH

```
$ git reset 5352dbd
```



```
david@coyote:~/dev/sn  
david ~/dev/sn on ↳ feat/morgan $ git log --oneline -4  
5352dbd (HEAD -> feat/morgan) C  
eed750 B
```

As alterações de D permanecem no diretório de trabalho e os commits posteriores deixam de fazer parte da branch

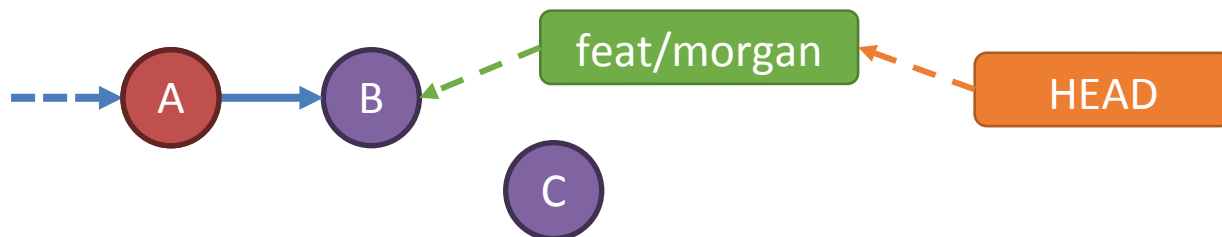
Desfazendo o último commit

- Também podemos usar **HEAD~1** para referenciar o pai de HEAD, **HEAD~2** para referenciar o avô, e assim por diante



```
$ git reset HEAD~1
```

- Depois do reset, o commit C é removido do histórico da branch e as alterações de C continuam no diretório de trabalho



Desfazendo o último commit

- O **git reset** move a referência da branch para outro commit

```
git reset <commit>
```

- Mas existem diferentes maneiras de tratar as mudanças nos arquivos dos commits removidos:
 - soft
 - mixed
 - hard

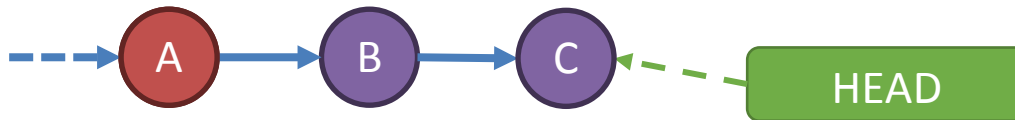


Git Reset Soft

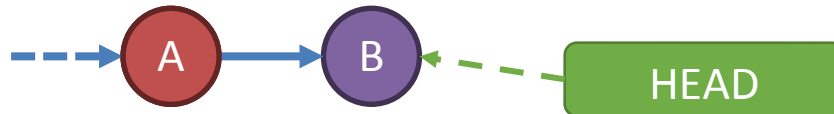
- Usando a opção **soft**, podemos mover a branch para um commit anterior, mas matendo as alterações na **staging area**

```
git reset --soft HEAD~1
```

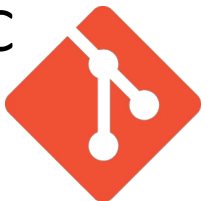
Antes:



Depois:



- Nesse caso, todas as mudanças feitas no commit C são mantidas na **staging área**, e podemos refazer o commit C

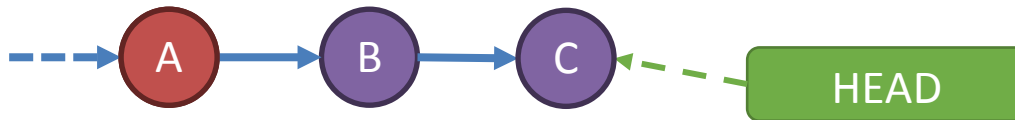


Git Reset Mixed

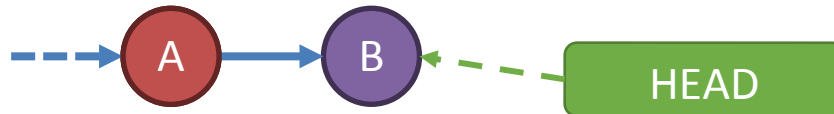
- Usando a opção **mixed**, movemos a branch para um commit anterior, mas matendo as alterações no **diretório de trabalho**

```
git reset --mixed HEAD~1
```

Antes:



Depois:



- As alterações do commit removido permanecem no **diretório de trabalho**, mas saem da staging area
- Essa é a opção padrão do comando **git reset**

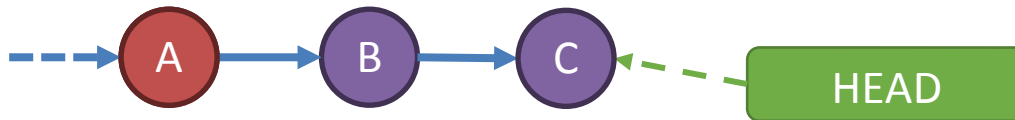


Git Reset Hard

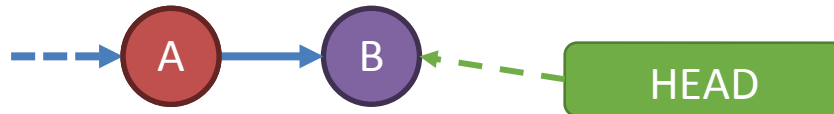
- Usando a opção **hard**, movemos a branch para um commit anterior, mas **descartando todas as alterações nos arquivos**

```
git reset --hard HEAD~1
```

Antes:



Depois:



- Com a opção **hard**, as alterações feitas em C desaparecem do diretório de trabalho, sendo a forma mais destrutiva de reset



Histórico dos movimentos do HEAD

- O git reflog registra os movimentos recentes do HEAD e das referências locais

```
$ git reflog
```

```
david@coyote:~/dev/sn
david ~/dev/sn on ʒ feat/morgan $ git reset --hard HEAD~1
HEAD is now at eeed750 B
david ~/dev/sn on ʒ feat/morgan $ git log --oneline -3
eeed750 (HEAD -> feat/morgan) B
24b7cd4 A
b68ac70 (origin/feat/morgan) Install and configure morgan
david ~/dev/sn on ʒ feat/morgan $
```

- O reflog mostra onde o HEAD estava ao longo do tempo, mesmo que um commit não apareça mais no git log



git log × git reflog

- O **git log** mostra os commits alcançáveis pelo histórico atual

```
david@coyote:~/dev/sn
david ~/dev/sn on ʘ feat/morgan $ git log --oneline -4
5352dbd (HEAD -> feat/morgan) C
eed750 B
24b7cd4 A
b68ac70 (origin/feat/morgan) Install and configure morgan
david ~/dev/sn on ʘ feat/morgan $
```

- O **git reflog** mostra por onde o HEAD passou

```
david@coyote:~/dev/sn
david ~/dev/sn on ʘ feat/morgan $ git reflog -5
5352dbd (HEAD -> feat/morgan) HEAD@{0}: reset: moving to 5352dbd
9990fe3 HEAD@{1}: commit: D
5352dbd (HEAD -> feat/morgan) HEAD@{2}: commit: C
eed750 HEAD@{3}: commit: B
24b7cd4 HEAD@{4}: commit: A
david ~/dev/sn on ʘ feat/morgan $
```

git log × git reflog

- O **git log** mostra os commits alcançáveis pelo histórico atual

```
david@coyote:~/dev/sn
david ~/dev/sn on ̣ feat/morgan $ git log --oneline -4
5352dbd (HEAD -> feat/morgan) C
eed750 B
24b7cd4 A
b68ac70 (origin/feat/morgan) Install and configure morgan
david ~/dev/sn on ̣ feat/morgan $
```

- O **git reflog** mostra por onde o HEAD

```
david@coyote:~/dev/sn
david ~/dev/sn on ̣ feat/morgan $ git reflog
5352dbd (HEAD -> feat/morgan) HEAD@{1}: commit: D
9990fe3 HEAD@{1}: commit: D
5352dbd (HEAD -> feat/morgan) HEAD@{2}: commit: C
eed750 HEAD@{3}: commit: B
24b7cd4 HEAD@{4}: commit: A
david ~/dev/sn on ̣ feat/morgan $
```

O commit **9990fe3** desapareceu do **git log**, mas ainda pode ser encontrado pelo **git reflog**



Recuperando um commit

- Para recuperar um commit <hash> eliminado através de um **git reset --hard**, podemos usar **reset --hard <hash>**

```
david@coyote:~/dev/sn
david ~/dev/sn on ↳ feat/morgan $ git reset --hard 9990fe3
HEAD is now at 9990fe3 D
david ~/dev/sn on ↳ feat/morgan $ git log --oneline -4
9990fe3 (HEAD -> feat/morgan) D
5352dbd C
eed750 B
24b7cd4 A
david ~/dev/sn on ↳ feat/morgan $
```



Recuperando um commit

Além da opção com o **git reset**, também podemos criar nova uma branch apontando para o commit recuperado

```
david@coyote:~/dev/sn
david ~/dev/sn on ❷ feat/morgan $ git reset --hard HEAD~1
HEAD is now at 5352dbd C
david ~/dev/sn on ❷ feat/morgan $ git branch recuperacao 9990fe3
david ~/dev/sn on ❷ feat/morgan $ git switch recuperacao
Switched to branch 'recuperacao'
david ~/dev/sn on ❷ recuperacao $ git log --oneline -3
9990fe3 (HEAD -> recuperacao) D
5352dbd (feat/morgan) C
eed750 B
david ~/dev/sn on ❷ recuperacao $
```



Exercício (parte I)

- Na branch **feat/morgan**, execute o laço apresentado para acrescentar A, B, C e D ao README.md e criar um commit para cada letra
- Anote os hashes de C e D. Execute **git reset <hash-de-C>**. Confirme que D saiu do git log e permaneceu no diretório de trabalho
- Volte a D com **git reset --hard <hash-de-D>**. Execute **git reset --soft HEAD~1** e confirme que as alterações de D estão na staging area
- Volte novamente a D. Execute **git reset --hard HEAD~1** e confirme que D desapareceu do git log e do README.md



Exercício (parte II)

- Compare **git log --oneline -4** com **git reflog -5** e localize o hash de D no reflog
- Recupere D na **feat/morgan** com **git reset --hard <hash-de-D>**. Verifique o histórico e o conteúdo do README.md
- Execute novamente **git reset --hard HEAD~1**. Crie a branch recuperacao apontando para D e mude para ela
- Na branch recuperacao, gere o arquivo **evidencia-reset-reflog.txt**. Faça o commit Practice git reset and reflog e envie a branch ao GitHub
 - **git log --oneline --all --decorate --graph > evidencia-reset-reflog.txt**
 - **git reflog -10 >> evidencia-reset-reflog.txt**



Corrigindo o último commit

- O comando **git reset HEAD~1** remove o último commit
- Mas se quisermos corrigir o último commit sem removê-lo, podemos usar o comando **git commit --amend**

```
$ git add arquivo-esquecido.txt  
$ git commit --amend
```

- Também podemos apenas alterar a mensagem do último commit:

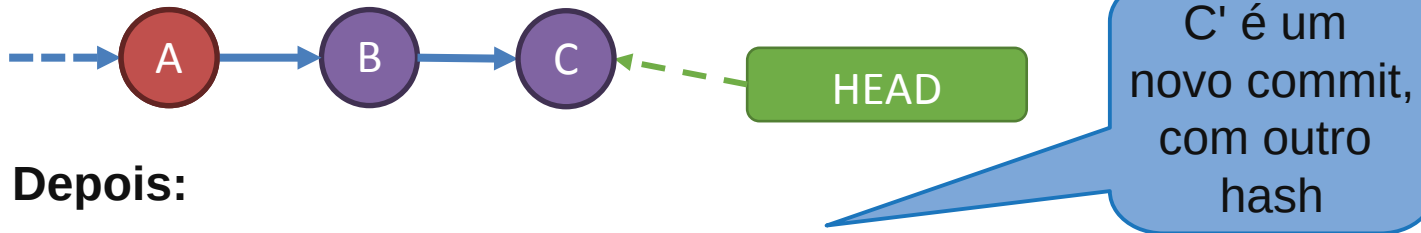
```
$ git add arquivo-esquecido.txt  
$ git commit --amend -m "Nova mensagem"
```



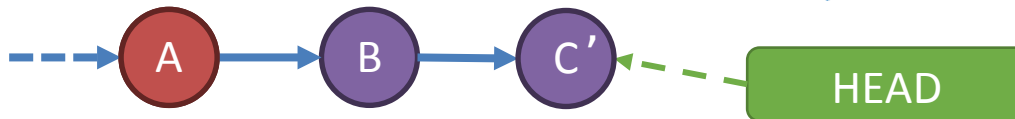
Corrigindo o último commit

- O comando **git commit --amend** reescreve o histórico, pois o último commit é apagado e substituído por um novo

Antes:



Depois:



- Desta forma, a opção **--amend** não deve ser usada em commits que já foram enviados para o repositório remoto

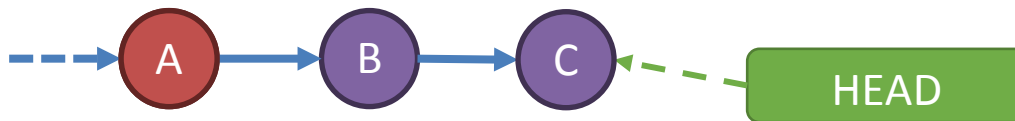


E se o commit já está no GitHub?

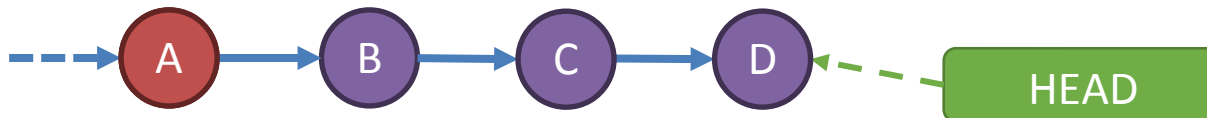
- Quando um commit já foi compartilhado, normalmente não queremos apagar ou reescrever o histórico
- Nesses casos, podemos usar o **git revert** para criar um novo commit que desfaz as alterações de outro commit

```
$ git revert HEAD
```

Antes:



Depois:



D desfaz as alterações realizadas por C, e o C continua no histórico



Git Stash

- O **git stash** guarda temporariamente alterações que ainda não estão prontas para um commit
- É útil quando você está trabalhando no código, precisa mudar de branch e não quer criar um commit incompleto

```
$ git add .  
$ git commit -m "Work In Progress"  
$ git switch main
```



```
$ git stash  
$ git switch main
```



- Depois de executar o comando **git stash**, o diretório de trabalho volta ao estado do último commit
 - As alterações ficam armazenadas em uma pilha temporária



Git Stash

- Após o comando **git stash**, pode-se trocar de branch e trabalhar normalmente nela
- Ao voltar para a branch anterior, pode-se restaurar as alterações do último git stash usando a opção pop

```
$ git switch -  
$ git stash pop
```



Comandos principais do Stash

`git stash`

Guardar alterações

`git stash push -m "mensagem"` Guardar com descrição

`git stash -u`

Incluir arquivos untracked

`git stash list`

Listar stashes

`git stash pop`

Restaurar e remover

`git stash apply`

Restaurar e manter

`git stash clear`

Remover todos os stashes armazenados

`git stash drop`

Remover um stash

